

Human review packet: GN21DriverSurgePricing

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GN21DriverSurgePricing

Paper id: GN21DriverSurgePricing

Source version: Management Science 2022 / arXiv v4 (2021); byte-pinned publication source record

Source record: audit/paper_statement_map.json

Rows: 16

Generated: 2026-09-07

Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GN21DriverSurgePricing --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- [Governing Lean declarations \(221; supporting context\)](#)
- [Paper-specific semantic prerequisites \(13; not paper claims\)](#)
 - [GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel](#)
 - [GN21DriverSurgePricing.GN21DynamicIIDCycleModel](#)
 - [GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint](#)
 - [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
 - [GN21DriverSurgePricing.Lemma6EndpointDerivativeData](#)
 - [GN21DriverSurgePricing.PaperInterface.review_definition_incentive_compatibleSpec](#)
 - [GN21DriverSurgePricing.PaperInterface.review_definition_surge_stateSpec](#)
 - [GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization](#)
 - [GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation](#)
 - [GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar](#)
 - [GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar](#)
 - [GN21DriverSurgePricing.paper_lemma1_stochastic_dynamic_reward_decomposition_of_actual_calendar](#)
 - [GN21DriverSurgePricing.paper_lemma3_stochastic_time_fraction_formula_of_actual_calendar](#)
- [Appendix source claims \(1\)](#)
 - [review_lemma10_nonsurge_derivative_positive_of_acceptAll_boundsSpec](#)
- [Main-text source claims \(3\)](#)
 - [review_lemma1_measured_dynamic_reward_decompositionSpec](#)
 - [review_lemma2_switch_probability_formulaSpec](#)
 - [review_lemma3_measured_time_fraction_formulaSpec](#)
- [Appendix source claims \(continued\) \(5\)](#)
 - [review_lemma4_single_state_threshold_uniquenessSpec](#)
 - [review_lemma6_upper_endpoint_derivative_formulaSpec](#)
 - [review_lemma7_affine_positive_additive_response_quasi_convexSpec](#)
 - [review_lemma8_affine_negative_additive_response_quasi_concaveSpec](#)

- review_lemma9_surge_derivative_positive_of_acceptAll_boundsSpec
- Main-text source claims (continued) (3)
 - review_proposition3_1_affine_single_state_icSpec
 - review_theorem1_single_state_threshold_best_responseSpec
 - review_theorem2_multiplicative_positive_finite_cutoff_not_ic_both_statesSpec
- Appendix source claims (continued 2) (1)
 - review_theorem4_full_structural_policy_forms_directSpec
- Main-text source claims (continued 2) (1)
 - review_theorem2_multiplicative_policy_shape_source_claimSpec
- Appendix source claims (continued 3) (1)
 - lemma5_full_variational_policy_formsSpec
- Main-text source claims (continued 3) (1)
 - review_theorem3_structured_pricingSpec

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.Probability.IIDStream.coordinate

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → ℕ → (ℕ → α) → α
value:
fun {α : Type u_1} (n : ℕ) (a : ℕ → α) => a n
```

AppliedModelingLib.Probability.IIDStream.externalIndexTail

Declaration location: reusable library

Lean declaration body

```
type:
{σ : Type u_1} → {α : Type u_2} → (σ → ℕ) → σ × (ℕ → α) → ℕ → α
value:
fun {σ : Type u_1} {α : Type u_2} (index : σ → ℕ) (a : σ × (ℕ → α)) (a_1 : ℕ) =>
  AppliedModelingLib.Probability.IIDStream.coordinate (index a.1 + a_1) a.2
```

Direct retained dependencies

- [AppliedModelingLib.Probability.IIDStream.coordinate](#)

AppliedModelingLib.Probability.PoissonProcess.exponentialInterarrivalMeasure

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → MeasureTheory.Measure (ℕ → ℝ)
value:
fun (rate : ℝ) => MeasureTheory.Measure.infinitePi fun (x : ℕ) => ProbabilityTheory.expMeasure rate
```

AppliedModelingLib.Probability.PoissonProcess.interarrival

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → (ℕ → ℝ) → ℝ
value:
fun (n : ℕ) (a : ℕ → ℝ) => a n
```

AppliedModelingLib.Probability.PoissonProcess.arrivalTime

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → (ℕ → ℝ) → ℝ
value:
fun (n : ℕ) (a : ℕ → ℝ) => ∑ i ∈ Finset.range (n + 1), AppliedModelingLib.Probability.PoissonProcess.interarrival i a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.interarrival](#)

AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow (\mathbb{N} \rightarrow \mathbb{R}) \rightarrow \mathbb{N}$ 

value:
fun (t :  $\mathbb{R}$ ) (a :  $\mathbb{N} \rightarrow \mathbb{R}$ ) =>
  if h :  $\exists (n : \mathbb{N}), t < \text{AppliedModelingLib.Probability.PoissonProcess.arrivalTime } n \ a$  then Nat.find h else 0
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.arrivalTime](#)

AppliedModelingLib.Probability.PoissonProcess.residualTail

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow (\mathbb{N} \rightarrow \mathbb{R}) \rightarrow \mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (s :  $\mathbb{R}$ ) ( $\omega$  :  $\mathbb{N} \rightarrow \mathbb{R}$ ) (a :  $\mathbb{N}$ ) =>
  match a with
  | 0 =>
    AppliedModelingLib.Probability.PoissonProcess.arrivalTime
      (AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount s  $\omega$ )  $\omega$  -
      s
  | k.succ =>
    AppliedModelingLib.Probability.PoissonProcess.interarrival
      (AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount s  $\omega$  + (k + 1)  $\omega$ )
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.arrivalTime](#)
- [AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount](#)
- [AppliedModelingLib.Probability.PoissonProcess.interarrival](#)

AppliedModelingLib.Probability.PoissonProcess.externalTimeResidualTail

Declaration location: reusable library

Lean declaration body

```
type:
 $\{\beta : \text{Type } u\_1\} \rightarrow (\beta \rightarrow \mathbb{R}) \rightarrow \beta \times (\mathbb{N} \rightarrow \mathbb{R}) \rightarrow \mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun  $\{\beta : \text{Type } u\_1\}$  (time :  $\beta \rightarrow \mathbb{R}$ ) (a :  $\beta \times (\mathbb{N} \rightarrow \mathbb{R})$ ) (a_1 :  $\mathbb{N}$ ) =>
  AppliedModelingLib.Probability.PoissonProcess.residualTail (time a.1) a.2 a_1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.residualTail](#)

AppliedModelingLib.Probability.PoissonProcess.externalTimeMarkedResidualTail

Declaration location: reusable library

Lean declaration body

```
type:
 $\{\beta : \text{Type } u\_1\} \rightarrow \{\alpha : \text{Type } u\_2\} \rightarrow [\text{MeasurableSpace } \beta] \rightarrow (\beta \rightarrow \mathbb{R}) \rightarrow (\beta \times (\mathbb{N} \rightarrow \mathbb{R})) \times (\mathbb{N} \rightarrow \alpha) \rightarrow (\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \alpha)$ 

value:
fun  $\{\beta : \text{Type } u\_1\}$   $\{\alpha : \text{Type } u\_2\}$  [MeasurableSpace  $\beta$ ] (time :  $\beta \rightarrow \mathbb{R}$ ) (a :  $(\beta \times (\mathbb{N} \rightarrow \mathbb{R})) \times (\mathbb{N} \rightarrow \alpha)$ ) =>
  (AppliedModelingLib.Probability.PoissonProcess.externalTimeResidualTail time a.1,
   AppliedModelingLib.Probability.IIDStream.externalIndexTail
     (fun (h :  $\beta \times (\mathbb{N} \rightarrow \mathbb{R})$ ) => AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount (time h.1) h.2) a)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.IIDStream.externalIndexTail](#)
- [AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount](#)
- [AppliedModelingLib.Probability.PoissonProcess.externalTimeResidualTail](#)

AppliedModelingLib.Probability.PoissonProcess.twoStreamAfterActiveHead

Declaration location: reusable library

Lean declaration body

```
type:
Fin 2 → (ℕ → ℝ) × (ℕ → ℝ) → ((ℕ → ℝ) × (ℕ → ℝ)) × ℝ

value:
fun (initial : Fin 2) (x : (ℕ → ℝ) × (ℕ → ℝ)) =>
  if initial = 0 then ((fun (n : ℕ) => x.1 (n + 1), x.2), x.1 0) else ((x.1, fun (n : ℕ) => x.2 (n + 1)), x.2 0)
```

AppliedModelingLib.Probability.TwoStateSwitching.otherState

Declaration location: reusable library

Lean declaration body

```
type:
Fin 2 → Fin 2

value:
fun (i : Fin 2) => if i = 0 then 1 else 0
```

AppliedModelingLib.Probability.TwoStateSwitching.stateAfterSwitches

Declaration location: reusable library

Lean declaration body

```
type:
Fin 2 → ℕ → Fin 2

value:
fun (initial : Fin 2) (n : ℕ) =>
  if Even n then initial else AppliedModelingLib.Probability.TwoStateSwitching.otherState initial
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.otherState](#)

AppliedModelingLib.Probability.TwoStateSwitching.alternatingGap

Declaration location: reusable library

Lean declaration body

```
type:
Fin 2 → (Fin 2 → ℕ → ℝ) → ℕ → ℝ

value:
fun (initial : Fin 2) (switchGaps : Fin 2 → ℕ → ℝ) (n : ℕ) =>
  switchGaps (AppliedModelingLib.Probability.TwoStateSwitching.stateAfterSwitches initial n) (n / 2)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.stateAfterSwitches](#)

AppliedModelingLib.Probability.TwoStateSwitching.alternatingGaps

Declaration location: reusable library

Lean declaration body

```
type:
Fin 2 → (Fin 2 → ℕ → ℝ) → ℕ → ℝ

value:
fun (initial : Fin 2) (a : Fin 2 → ℕ → ℝ) (a_1 : ℕ) =>
  AppliedModelingLib.Probability.TwoStateSwitching.alternatingGap initial a a_1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.alternatingGap](#)

AppliedModelingLib.Probability.TwoStateSwitching.stateAt

Declaration location: reusable library

Lean declaration body

```
type:
Fin 2 → (Fin 2 → ℕ → ℝ) → ℝ → Fin 2

value:
fun (initial : Fin 2) (switchGaps : Fin 2 → ℕ → ℝ) (t : ℝ) =>
  AppliedModelingLib.Probability.TwoStateSwitching.stateAfterSwitches initial
    (AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount t
      (AppliedModelingLib.Probability.TwoStateSwitching.alternatingGaps initial switchGaps))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount](#)
- [AppliedModelingLib.Probability.TwoStateSwitching.alternatingGaps](#)
- [AppliedModelingLib.Probability.TwoStateSwitching.stateAfterSwitches](#)

AppliedModelingLib.Probability.TwoStateSwitching.switchGapsOfAlternating

Declaration location: reusable library

Lean declaration body

```
type:
Fin 2 → (ℕ → ℝ) → Fin 2 → ℕ → ℝ

value:
fun (initial : Fin 2) (gaps : ℕ → ℝ) (a : Fin 2) (a_1 : ℕ) => if a = initial then gaps (2 * a_1) else gaps (2 * a_1 + 1)
```

AppliedModelingLib.Probability.TwoStateSwitching.switchGapsAfterElapsed

Declaration location: reusable library

Lean declaration body

```
type:
Fin 2 → (Fin 2 → ℕ → ℝ) → ℝ → Fin 2 → ℕ → ℝ

value:
fun (initial : Fin 2) (switchGaps : Fin 2 → ℕ → ℝ) (t : ℝ) (a : Fin 2) (a_1 : ℕ) =>
  AppliedModelingLib.Probability.TwoStateSwitching.switchGapsOfAlternating
    (AppliedModelingLib.Probability.TwoStateSwitching.stateAt initial switchGaps t)
    (AppliedModelingLib.Probability.PoissonProcess.residualTail t
      (AppliedModelingLib.Probability.TwoStateSwitching.alternatingGaps initial switchGaps))
  a a_1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.residualTail](#)
- [AppliedModelingLib.Probability.TwoStateSwitching.alternatingGaps](#)
- [AppliedModelingLib.Probability.TwoStateSwitching.stateAt](#)
- [AppliedModelingLib.Probability.TwoStateSwitching.switchGapsOfAlternating](#)

AppliedModelingLib.Probability.exponentialRaceMinimum

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (p :  $\mathbb{R} \times \mathbb{R}$ ) => min p.1 p.2
```

AppliedModelingLib.Probability.regenerativeTail

Declaration location: reusable library

Lean declaration body

```
type:
{State : Type u_4} → {Summary : Type u_5} → (State → Summary × State) → State → State

value:
fun {State : Type u_4} {Summary : Type u_5} (step : State → Summary × State) (a : State) => (step a).2
```

AppliedModelingLib.Probability.regenerativeSummary

Declaration location: reusable library

Lean declaration body

```
type:
{State : Type u_4} → {Summary : Type u_5} → (State → Summary × State) →  $\mathbb{N}$  → State → Summary

value:
fun {State : Type u_4} {Summary : Type u_5} (step : State → Summary × State) (n :  $\mathbb{N}$ ) (a : State) =>
  (step ((AppliedModelingLib.Probability.regenerativeTail step)^[n] a)).1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.regenerativeTail](#)

AppliedModelingLib.positiveRealAcceptAll

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \text{Prop}$ 

value:
fun (a :  $\mathbb{R}$ ) => Set.lci 0 a
```

AppliedModelingLib.acceptsAllPositiveReal

Declaration location: reusable library

Lean declaration body

```
type:
Set  $\mathbb{R} \rightarrow \text{Prop}$ 

value:
fun ( $\sigma$  : Set  $\mathbb{R}$ ) => AppliedModelingLib.positiveRealAcceptAll  $\subseteq \sigma$ 
```

Direct retained dependencies

- [AppliedModelingLib.positiveRealAcceptAll](#)

AppliedModelingLib.twoStateCtmcSwitchProb

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (lambdalJ lambdajl t :  $\mathbb{R}$ ) => lambdalJ / (lambdalJ + lambdajl) * (1 - Real.exp (-(lambdalJ + lambdajl) * t))
```

GN21DriverSurgePricing.GN21EndpointVector

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \text{Type}$ 

value:
fun (n :  $\mathbb{N}$ ) => Fin (2 * n) → ENNReal
```

GN21DriverSurgePricing.GN21Lemma5EndpointVector

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \text{Type}$ 

value:
fun (extra :  $\mathbb{N}$ ) => GN21DriverSurgePricing.GN21EndpointVector (extra + 1)
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21EndpointVector](#)

GN21DriverSurgePricing.MarkedRestart.Step

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
 $\mathbb{R} \times \text{Bool}$ 
```

GN21DriverSurgePricing.MarkedRestart.accepted

Declaration location: paper-local

Lean declaration body

```
type:
( $\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.MarkedRestart.Step}$ ) →  $\mathbb{N} \rightarrow \text{Prop}$ 

value:
fun (omega :  $\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.MarkedRestart.Step}$ ) (n :  $\mathbb{N}$ ) => (omega n).2 = true
```

Direct retained dependencies

- [GN21DriverSurgePricing.MarkedRestart.Step](#)

GN21DriverSurgePricing.MarkedRestart.firstSuccess

Declaration location: paper-local

Lean declaration body

```
type:
( $\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.MarkedRestart.Step}$ ) →  $\mathbb{N}$ 

value:
fun (omega :  $\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.MarkedRestart.Step}$ ) =>
  if h :  $\exists (n : \mathbb{N}), \text{GN21DriverSurgePricing.MarkedRestart.accepted omega n}$  then Nat.find h else 0
```


Direct retained dependencies

- [GN21DriverSurgePricing.MarkedRestart.Step](#)
- [GN21DriverSurgePricing.MarkedRestart.accepted](#)

GN21DriverSurgePricing.RawTwoStateCTMC.switchClockIndex

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → Fin 4

value:
fun (a : Fin 2) =>
  match a with
  | 0 => 2
  | 1 => 3
```

GN21DriverSurgePricing.TripLength

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
ℝ
```

GN21DriverSurgePricing.GN21PostThinningRaceSeed

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
ℝ × ℝ × GN21DriverSurgePricing.TripLength
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.GN21RawCycleSeed

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
(Fin 4 → ℕ → ℝ) × (Fin 2 → ℕ → GN21DriverSurgePricing.TripLength)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.GN21RawPostThinningRaceHistory

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
ℝ ⊕ Fin 2 × ℝ × GN21DriverSurgePricing.TripLength
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.GN21SwitchFirstJointContinuation

Declaration location: paper-local

Lean declaration body

type:
Type

value:
 $((\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.TripLength})) \times ((\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \mathbb{R})) \times (\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.TripLength})$

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.GN21SwitchFirstJointContinuationInput

Declaration location: paper-local

Lean declaration body

type:
Type

value:
 $((\mathbb{R} \times ((\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \mathbb{R})) \times (\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.TripLength})) \times (\mathbb{N} \rightarrow \mathbb{R})) \times (\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.TripLength})$

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.GN21SwitchFirstSplitInput

Declaration location: paper-local

Lean declaration body

type:
Type

value:
 $(((((\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \mathbb{R})) \times \mathbb{R}) \times (\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.TripLength})) \times (\mathbb{N} \rightarrow \mathbb{R}) \times (\mathbb{N} \rightarrow \text{GN21DriverSurgePricing.TripLength}))$

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.PricingFunction

Declaration location: paper-local

Lean declaration body

type:
Type

value:
 $\text{GN21DriverSurgePricing.TripLength} \rightarrow \mathbb{R}$

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.ProofBridge.GN21PayoutFunction

Declaration location: paper-local

Lean declaration body

```
field toPricingFunction:
GN21DriverSurgePricing.ProofBridge.GN21PayoutFunction → GN21DriverSurgePricing.PricingFunction

field nonnegative_on_trip:
∀ (self : GN21DriverSurgePricing.ProofBridge.GN21PayoutFunction) (tau : GN21DriverSurgePricing.TripLength),
0 < tau → 0 ≤ self.toPricingFunction tau
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.TripPolicy

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
Set GN21DriverSurgePricing.TripLength
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.DynamicReward

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
(Fin 2 → GN21DriverSurgePricing.TripPolicy) → ℝ
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.GN21SymmDiffContinuousAt

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
(GN21DriverSurgePricing.TripPolicy → ℝ) → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (Rhat : GN21DriverSurgePricing.TripPolicy → ℝ)
(σ : GN21DriverSurgePricing.TripPolicy) =>
∀ (ε : ℝ),
0 < ε →
∃ (δ : ENNReal), δ ≠ 0 ∧ ∀ (τ : GN21DriverSurgePricing.TripPolicy), μ (symmDiff σ τ) < δ → |Rhat τ - Rhat σ| < ε
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.SingleStateReward

Declaration location: paper-local

Lean declaration body

```
type:
Type

value:
GN21DriverSurgePricing.TripPolicy → ℝ
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.acceptsAllTrips

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (σ : GN21DriverSurgePricing.TripPolicy) => AppliedModelingLib.acceptsAllPositiveReal σ
```

Direct retained dependencies

- [AppliedModelingLib.acceptsAllPositiveReal](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.affinePricing

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → GN21DriverSurgePricing.TripLength → ℝ

value:
fun (m a : ℝ) (a_1 : GN21DriverSurgePricing.TripLength) => m * a_1 + a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.completeThresholdPolicy

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.PricingFunction → ℝ → GN21DriverSurgePricing.TripLength → Prop

value:
fun (w : GN21DriverSurgePricing.PricingFunction) (c : ℝ) (a : GN21DriverSurgePricing.TripLength) =>
(Set.lol 0 n {τ : GN21DriverSurgePricing.TripLength | c ≤ w τ / τ}) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.dynamicOptimal

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.DynamicReward → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → Prop

value:
fun (R : GN21DriverSurgePricing.DynamicReward) (σ : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
  ∀ (ρ : Fin 2 → GN21DriverSurgePricing.TripPolicy), R ρ ≤ R σ
```

Direct retained dependencies

- [GN21DriverSurgePricing.DynamicReward](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21AcceptanceMark

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → Bool

value:
fun (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.TripLength) =>
  if a ∈ sigma then true else false
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.AcceptedTripSelection.acceptancePairStream

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy →
  (ℕ → GN21DriverSurgePricing.TripLength) → ℕ → GN21DriverSurgePricing.MarkedRestart.Step

value:
fun (sigma : GN21DriverSurgePricing.TripPolicy) (a : ℕ → GN21DriverSurgePricing.TripLength) (a_1 : ℕ) =>
  (0, GN21DriverSurgePricing.gn21AcceptanceMark sigma (a a_1))
```

Direct retained dependencies

- [GN21DriverSurgePricing.MarkedRestart.Step](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21AcceptanceMark](#)

GN21DriverSurgePricing.AcceptedTripSelection.firstAcceptedIndex

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy → (ℕ → GN21DriverSurgePricing.TripLength) → ℕ

value:
fun (sigma : GN21DriverSurgePricing.TripPolicy) (a : ℕ → GN21DriverSurgePricing.TripLength) =>
  (GN21DriverSurgePricing.MarkedRestart.firstSuccess ◦
    GN21DriverSurgePricing.AcceptedTripSelection.acceptancePairStream sigma)
a
```

Direct retained dependencies

- [GN21DriverSurgePricing.AcceptedTripSelection.acceptancePairStream](#)
- [GN21DriverSurgePricing.MarkedRestart.Step](#)
- [GN21DriverSurgePricing.MarkedRestart.firstSuccess](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.AcceptedTripSelection.firstAcceptedTripMark

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy → (ℕ → GN21DriverSurgePricing.TripLength) → GN21DriverSurgePricing.TripLength

value:
fun (sigma : GN21DriverSurgePricing.TripPolicy) (a : ℕ → GN21DriverSurgePricing.TripLength) =>
  a (GN21DriverSurgePricing.AcceptedTripSelection.firstAcceptedIndex sigma a)
```

Direct retained dependencies

- [GN21DriverSurgePricing.AcceptedTripSelection.firstAcceptedIndex](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21AggregateDynamicReward

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
fun (Qi Qj Ti Tj Wi Wj : ℝ) => (Qi * Wj + Qj * Wi) / (Qi * Tj + Qj * Ti)
```

GN21DriverSurgePricing.gn21ArrivalClockIndex

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → Fin 4

value:
fun (a : Fin 2) =>
  match a with
  | 0 => 0
  | 1 => 1
```

GN21DriverSurgePricing.gn21ComponentUpperPath

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy →
  GN21DriverSurgePricing.TripLength →
  GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripLength → Prop

value:
fun (sigma : GN21DriverSurgePricing.TripPolicy) (lower upper x a : GN21DriverSurgePricing.TripLength) =>
  (sigma \ Set.loo lower upper u Set.loo lower x) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21CrossSubcycleProb

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (arrivalRate acceptProb switchRate exitWeight :  $\mathbb{R}$ ) => exitWeight / (arrivalRate * acceptProb + switchRate)
```

GN21DriverSurgePricing.gn21CycleClockRate

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Fin } 4 \rightarrow \mathbb{R}$ 

value:
fun (arrivalI arrivalJ switchIJ switchJI :  $\mathbb{R}$ ) (a : Fin 4) =>
  match a with
  | 0 => arrivalI
  | 1 => arrivalJ
  | 2 => switchIJ
  | 3 => switchJI
```

GN21DriverSurgePricing.gn21CycleMarkLaw

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength

value:
fun (mul muj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (a : Fin 2) =>
  match a with
  | 0 => mul
  | 1 => muj
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.gn21DerivativeSignKernel

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (q u wi Qi Qj Ti Tj Wi Wj :  $\mathbb{R}$ ) => q * (Wj * Ti - Tj * Wi) + wi * (Qi * Tj + Qj * Ti) - u * (Qi * Wj + Qj * Wi)
```

GN21DriverSurgePricing.gn21DynamicRewardFormula

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (timeFractionI rewardRateI timeFractionJ rewardRateJ :  $\mathbb{R}$ ) =>
  timeFractionI * rewardRateI + timeFractionJ * rewardRateJ
```

GN21DriverSurgePricing.gn21EndpointQiPath

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (arrivalRate switchRate lowerEndpoint :  $\mathbb{R}$ ) (density switchProb :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (x :  $\mathbb{R}$ ) =>
  switchRate + arrivalRate *  $\int (\tau : \mathbb{R})$  in lowerEndpoint..x, switchProb  $\tau$  * density  $\tau$ 
```

GN21DriverSurgePricing.gn21EndpointTiPath

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (arrivalRate lowerEndpoint :  $\mathbb{R}$ ) (density :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (x :  $\mathbb{R}$ ) =>
  1 + arrivalRate *  $\int (\tau : \mathbb{R})$  in lowerEndpoint..x,  $\tau$  * density  $\tau$ 
```

GN21DriverSurgePricing.gn21EndpointWiPath

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow (\mathbb{R} \rightarrow \mathbb{R}) \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (arrivalRate lowerEndpoint :  $\mathbb{R}$ ) (density payment :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (x :  $\mathbb{R}$ ) =>
  arrivalRate *  $\int (\tau : \mathbb{R})$  in lowerEndpoint..x, payment  $\tau$  * density  $\tau$ 
```

GN21DriverSurgePricing.gn21InteriorSplitLowerPolicy

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy  $\rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow$  GN21DriverSurgePricing.TripLength  $\rightarrow$  Prop

value:
fun (sigma : GN21DriverSurgePricing.TripPolicy) (pivot cutoff :  $\mathbb{R}$ ) (a : GN21DriverSurgePricing.TripLength) =>
  (sigma n Set.lto pivot u sigma n Set.loi cutoff) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21Lemma5ComponentLowerPath

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy →
  GN21DriverSurgePricing.TripLength →
    GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripLength → Prop

value:
fun (policy : GN21DriverSurgePricing.TripPolicy) (lower upper x a : GN21DriverSurgePricing.TripLength) =>
  (policy \ Set.lou lower upper u Set.lou x upper) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21Lemma5ComponentTailLowerPath

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy →
  GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripLength → Prop

value:
fun (policy : GN21DriverSurgePricing.TripPolicy) (lower x a : GN21DriverSurgePricing.TripLength) =>
  (policy \ Set.loi lower u Set.loi x) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21Lemma5ComponentUpperPath

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy →
  GN21DriverSurgePricing.TripLength →
    GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripLength → Prop

value:
fun (policy : GN21DriverSurgePricing.TripPolicy) (lower upper x a : GN21DriverSurgePricing.TripLength) =>
  (policy \ Set.lou lower upper u Set.lou lower x) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21Lemma5FirstEndpointIndex

Declaration location: paper-local

Lean declaration body

```
type:
(extra : ℕ) → Fin (2 * (extra + 1))

value:
fun (extra : ℕ) => {0, GN21DriverSurgePricing.gn21Lemma5FirstEndpointIndex._proof_2 extra}
```

GN21DriverSurgePricing.gn21Lemma5GapLowerIndex

Declaration location: paper-local

Lean declaration body

```
type:
{extra : ℕ} → Fin extra → Fin (2 * (extra + 1))

value:
fun {extra : ℕ} (i : Fin extra) => (2 * ↑i + 2, GN21DriverSurgePricing.gn21Lemma5GapLowerIndex._proof_2 i)
```

GN21DriverSurgePricing.gn21Lemma5GapUpperIndex

Declaration location: paper-local

Lean declaration body

```
type:
{extra : ℕ} → Fin extra → Fin (2 * (extra + 1))

value:
fun {extra : ℕ} (i : Fin extra) => (2 * ↑i + 1, GN21DriverSurgePricing.gn21Lemma5GapUpperIndex._proof_2 i)
```

GN21DriverSurgePricing.gn21Lemma5LastEndpointIndex

Declaration location: paper-local

Lean declaration body

```
type:
(extra : ℕ) → Fin (2 * (extra + 1))

value:
fun (extra : ℕ) => (2 * (extra + 1) - 1, GN21DriverSurgePricing.gn21Lemma5LastEndpointIndex._proof_2 extra)
```

GN21DriverSurgePricing.gn21Lemma6Response

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
fun (q u wi Qi Qj Ti Tj Ri Rj : ℝ) => q / u * (Rj - Ri) + wi / u * (Qi / Ti + Qj / Tj) - (Qi / Ti * Rj + Qj / Tj * Ri)
```

GN21DriverSurgePricing.gn21LowerEndpointIndex

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → Fin n → Fin (2 * n)

value:
fun {n : ℕ} (i : Fin n) => (2 * ↑i, GN21DriverSurgePricing.gn21LowerEndpointIndex._proof_2 i)
```

GN21DriverSurgePricing.gn21OrderedEndpointVectors

Declaration location: paper-local

Lean declaration body

```
type:
(n : ℕ) → GN21DriverSurgePricing.GN21EndpointVector n → Prop

value:
fun (n : ℕ) (a : GN21DriverSurgePricing.GN21EndpointVector n) =>
{endpoints : GN21DriverSurgePricing.GN21EndpointVector n | ∀ {i j : Fin (2 * n)}, i ≤ j → endpoints i ≤ endpoints j} a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21EndpointVector](#)

GN21DriverSurgePricing.gn21Lemma5EndpointDomain

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.Lemma5DerivativeShape →
(extra : ℕ) → GN21DriverSurgePricing.GN21Lemma5EndpointVector extra → Prop

value:
fun (shape : GN21DriverSurgePricing.Lemma5DerivativeShape) (extra : ℕ)
(a : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra) =>
(match shape with
| GN21DriverSurgePricing.Lemma5DerivativeShape.positive =>
GN21DriverSurgePricing.gn21OrderedEndpointVectors (extra + 1) n
{endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra |
endpoints (GN21DriverSurgePricing.gn21Lemma5FirstEndpointIndex extra) = 0} n
{endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra |
endpoints (GN21DriverSurgePricing.gn21Lemma5LastEndpointIndex extra) = T}
| GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyIncreasing =>
GN21DriverSurgePricing.gn21OrderedEndpointVectors (extra + 1) n
{endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra |
endpoints (GN21DriverSurgePricing.gn21Lemma5LastEndpointIndex extra) = T}
| GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyDecreasing =>
GN21DriverSurgePricing.gn21OrderedEndpointVectors (extra + 1) n
{endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra |
endpoints (GN21DriverSurgePricing.gn21Lemma5FirstEndpointIndex extra) = 0}
| GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex =>
GN21DriverSurgePricing.gn21OrderedEndpointVectors (extra + 1) n
{endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra |
endpoints (GN21DriverSurgePricing.gn21Lemma5FirstEndpointIndex extra) = 0} n
{endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra |
endpoints (GN21DriverSurgePricing.gn21Lemma5LastEndpointIndex extra) = T}
| GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave =>
GN21DriverSurgePricing.gn21OrderedEndpointVectors (extra + 1) n
a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21Lemma5EndpointVector](#)
- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.gn21Lemma5FirstEndpointIndex](#)
- [GN21DriverSurgePricing.gn21Lemma5LastEndpointIndex](#)
- [GN21DriverSurgePricing.gn21OrderedEndpointVectors](#)

GN21DriverSurgePricing.gn21RawCycleClock

Declaration location: paper-local

Lean declaration body

```
type:
Fin 4 → ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ

value:
fun (clock : Fin 4) (n : ℕ) (a : GN21DriverSurgePricing.GN21RawCycleSeed) => a.1 clock n
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.AcceptedArrivalTime.rawClockStream

Declaration location: paper-local

Lean declaration body

```
type:
Fin 4 → GN21DriverSurgePricing.GN21RawCycleSeed → ℕ → ℝ

value:
fun (clock : Fin 4) (a : GN21DriverSurgePricing.GN21RawCycleSeed) (a_1 : ℕ) =>
GN21DriverSurgePricing.gn21RawCycleClock clock a_1 a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.gn21RawCycleClock](#)

GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGaps

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.GN21RawCycleSeed → Fin 2 → ℕ → ℝ

value:
fun (a : GN21DriverSurgePricing.GN21RawCycleSeed) (a_1 : Fin 2) (a_2 : ℕ) =>
  GN21DriverSurgePricing.gn21RawCycleClock (GN21DriverSurgePricing.RawTwoStateCTMC.switchClockIndex a_1) a_2 a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.RawTwoStateCTMC.switchClockIndex](#)
- [GN21DriverSurgePricing.gn21RawCycleClock](#)

GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGapPair

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.GN21RawCycleSeed → (ℕ → ℝ) × (ℕ → ℝ)

value:
fun (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGaps a 0, GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGaps a 1)
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGaps](#)

GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchTailAndFirstGap

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.GN21RawCycleSeed → ((ℕ → ℝ) × (ℕ → ℝ)) × ℝ

value:
fun (initial : Fin 2) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  AppliedModelingLib.Probability.PoissonProcess.twoStreamAfterActiveHead initial
  (GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGapPair a)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.twoStreamAfterActiveHead](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGapPair](#)

GN21DriverSurgePricing.RawTwoStateCTMC.stateAt

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ → Fin 2

value:
fun (initial : Fin 2) (a : GN21DriverSurgePricing.GN21RawCycleSeed) (a_1 : ℝ) =>
  AppliedModelingLib.Probability.TwoStateSwitching.stateAt initial
  (GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGaps a) a_1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.stateAt](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGaps](#)

GN21DriverSurgePricing.gn21RawCycleMark

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.TripLength

value:
fun (state : Fin 2) (n : ℕ) (a : GN21DriverSurgePricing.GN21RawCycleSeed) => a.2 state n
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.AcceptedArrivalTime.rawTripMarkStream

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.GN21RawCycleSeed → ℕ → GN21DriverSurgePricing.TripLength

value:
fun (state : Fin 2) (a : GN21DriverSurgePricing.GN21RawCycleSeed) (a_1 : ℕ) =>
  GN21DriverSurgePricing.gn21RawCycleMark state a_1 a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21RawCycleMark](#)

GN21DriverSurgePricing.AcceptedArrivalTime.gn21RawFirstAcceptedArrivalTime

Declaration location: paper-local

Lean declaration body

```
type:
Fin 4 → Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ

value:
fun (clock : Fin 4) (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy)
  (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  AppliedModelingLib.Probability.PoissonProcess.arrivalTime
  (GN21DriverSurgePricing.AcceptedTripSelection.firstAcceptedIndex sigma
   (GN21DriverSurgePricing.AcceptedArrivalTime.rawTripMarkStream state a))
  (GN21DriverSurgePricing.AcceptedArrivalTime.rawClockStream clock a)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.arrivalTime](#)
- [GN21DriverSurgePricing.AcceptedArrivalTime.rawClockStream](#)
- [GN21DriverSurgePricing.AcceptedArrivalTime.rawTripMarkStream](#)
- [GN21DriverSurgePricing.AcceptedTripSelection.firstAcceptedIndex](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.AcceptedTripSelection.gn21RawFirstAcceptedTripMark

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.TripLength
value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  GN21DriverSurgePricing.AcceptedTripSelection.firstAcceptedTripMark sigma fun (n : ℕ) =>
    GN21DriverSurgePricing.gn21RawCycleMark state n a
```

Direct retained dependencies

- [GN21DriverSurgePricing.AcceptedTripSelection.firstAcceptedTripMark](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCycleMark](#)

GN21DriverSurgePricing.gn21RawCycleSeedMeasure

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
  MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
  ℝ → ℝ → ℝ → ℝ → MeasureTheory.Measure GN21DriverSurgePricing.GN21RawCycleSeed
value:
fun (mul muj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalI arrivalJ switchIJ switchJI : ℝ) =>
  (MeasureTheory.Measure.infinitePi fun (clock : Fin 4) =>
    AppliedModelingLib.Probability.PoissonProcess.exponentialInterarrivalMeasure
      (GN21DriverSurgePricing.gn21CycleClockRate arrivalI arrivalJ switchIJ switchJI clock)).prod
  (MeasureTheory.Measure.infinitePi fun (state : Fin 2) =>
    MeasureTheory.Measure.infinitePi fun (x : ℕ) => GN21DriverSurgePricing.gn21CycleMarkLaw mul muj state)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.exponentialInterarrivalMeasure](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21CycleClockRate](#)
- [GN21DriverSurgePricing.gn21CycleMarkLaw](#)

GN21DriverSurgePricing.gn21RawCycleSeedOfStateContinuation

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.GN21SwitchFirstJointContinuation → GN21DriverSurgePricing.GN21RawCycleSeed

value:
fun (state : Fin 2) (a : GN21DriverSurgePricing.GN21SwitchFirstJointContinuation) =>
  have z : GN21DriverSurgePricing.GN21SwitchFirstJointContinuation := a;
  match state with
  | 0 =>
    (fun (x : Fin 4) =>
      match (motive := Fin 4 → ℕ → ℝ) x with
      | 0 => z.1.1
      | 1 => z.2.2.1
      | 2 => z.2.1.1
      | 3 => z.2.1.2,
      fun (x : Fin 2) =>
        match (motive := Fin 2 → ℕ → GN21DriverSurgePricing.TripLength) x with
        | 0 => z.1.2
        | 1 => z.2.2.2)
    | 1 =>
      (fun (x : Fin 4) =>
        match (motive := Fin 4 → ℕ → ℝ) x with
        | 0 => z.2.2.1
        | 1 => z.1.1
        | 2 => z.2.1.1
        | 3 => z.2.1.2,
        fun (x : Fin 2) =>
          match (motive := Fin 2 → ℕ → GN21DriverSurgePricing.TripLength) x with
          | 0 => z.2.2.2
          | 1 => z.1.2)
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21SwitchFirstJointContinuation](#)
- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceHistoryEarning

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.PricingFunction → GN21DriverSurgePricing.GN21RawPostThinningRaceHistory → ℝ

value:
fun (w : GN21DriverSurgePricing.PricingFunction) (a : GN21DriverSurgePricing.GN21RawPostThinningRaceHistory) =>
  match a with
  | Sum.inl val => 0
  | Sum.inr z => w z.2.2
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawPostThinningRaceHistory](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceHistoryTime

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.GN21RawPostThinningRaceHistory → ℝ

value:
fun (a : GN21DriverSurgePricing.GN21RawPostThinningRaceHistory) =>
  match a with
  | Sum.inl holdingTime => holdingTime
  | Sum.inr z => z.2.1 + z.2.2
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawPostThinningRaceHistory](#)
- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceSeed

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  GN21DriverSurgePricing.TripPolicy →
    GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21PostThinningRaceSeed

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (GN21DriverSurgePricing.AcceptedArrivalTime.gn21RawFirstAcceptedArrivalTime
   (GN21DriverSurgePricing.gn21ArrivalClockIndex state) state sigma a,
   GN21DriverSurgePricing.gn21RawCycleClock (GN21DriverSurgePricing.RawTwoStateCTMC.switchClockIndex state) 0 a,
   GN21DriverSurgePricing.AcceptedTripSelection.gn21RawFirstAcceptedTripMark state sigma a)
```

Direct retained dependencies

- [GN21DriverSurgePricing.AcceptedArrivalTime.gn21RawFirstAcceptedArrivalTime](#)
- [GN21DriverSurgePricing.AcceptedTripSelection.gn21RawFirstAcceptedTripMark](#)
- [GN21DriverSurgePricing.GN21PostThinningRaceSeed](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.RawTwoStateCTMC.switchClockIndex](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21ArrivalClockIndex](#)
- [GN21DriverSurgePricing.gn21RawCycleClock](#)

GN21DriverSurgePricing.gn21RawAcceptedCalendarCompletionTime

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma a).1 +
  (GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma a).2.2
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSeed](#)

GN21DriverSurgePricing.gn21RawAcceptedCompletionMarkedArrivalResidual

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  GN21DriverSurgePricing.TripPolicy →
    Fin 2 → GN21DriverSurgePricing.GN21RawCycleSeed → (ℕ → ℝ) × (ℕ → GN21DriverSurgePricing.TripLength)

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (arrivalState : Fin 2)
  (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  AppliedModelingLib.Probability.PoissonProcess.externalTimeMarkedResidualTail
  (fun (source : GN21DriverSurgePricing.GN21RawCycleSeed) =>
    GN21DriverSurgePricing.gn21RawAcceptedCalendarCompletionTime state sigma source)
  ((a,
    GN21DriverSurgePricing.AcceptedArrivalTime.rawClockStream
    (GN21DriverSurgePricing.gn21ArrivalClockIndex arrivalState a),
    GN21DriverSurgePricing.AcceptedArrivalTime.rawTripMarkStream arrivalState a)
```


Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.externalTimeMarkedResidualTail](#)
- [GN21DriverSurgePricing.AcceptedArrivalTime.rawClockStream](#)
- [GN21DriverSurgePricing.AcceptedArrivalTime.rawTripMarkStream](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21ArrivalClockIndex](#)
- [GN21DriverSurgePricing.gn21RawAcceptedCalendarCompletionTime](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionState

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → Fin 2

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  GN21DriverSurgePricing.RawTwoStateCTMC.stateAt state a
  ((GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma a).1 +
   (GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma a).2.2)
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.RawTwoStateCTMC.stateAt](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSeed](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionSwitchGaps

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → Fin 2 → ℕ → ℝ

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed)
  (a_1 : Fin 2) (a_2 : ℕ) =>
  AppliedModelingLib.Probability.TwoStateSwitching.switchGapsAfterElapsed state
  (GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGaps a)
  ((GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma a).1 +
   (GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma a).2.2)
  a_1 a_2
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.switchGapsAfterElapsed](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchGaps](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSeed](#)

GN21DriverSurgePricing.gn21RawAcceptedCompletionNextCycleSeed

Declaration location: paper-local

Lean declaration body

```
type:
  Fin 2 →
    GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21RawCycleSeed

value:
  fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
    (fun (x : Fin 4) =>
      match (motive := Fin 4 → ℕ → ℝ) x with
      | 0 => (GN21DriverSurgePricing.gn21RawAcceptedCompletionMarkedArrivalResidual state sigma 0 a).1
      | 1 => (GN21DriverSurgePricing.gn21RawAcceptedCompletionMarkedArrivalResidual state sigma 1 a).1
      | 2 => GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionSwitchGaps state sigma a 0
      | 3 => GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionSwitchGaps state sigma a 1,
      fun (x : Fin 2) =>
        match (motive := Fin 2 → ℕ → GN21DriverSurgePricing.TripLength) x with
        | 0 => (GN21DriverSurgePricing.gn21RawAcceptedCompletionMarkedArrivalResidual state sigma 0 a).2
        | 1 => (GN21DriverSurgePricing.gn21RawAcceptedCompletionMarkedArrivalResidual state sigma 1 a).2)
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawAcceptedCompletionMarkedArrivalResidual](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionSwitchGaps](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceMinimum

Declaration location: paper-local

Lean declaration body

```
type:
  Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ

value:
  fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
    AppliedModelingLib.Probability.exponentialRaceMinimum
      ((GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma a).1,
       (GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma a).2.1)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.exponentialRaceMinimum](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSeed](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst

Declaration location: paper-local

Lean declaration body

```
type:
  Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → Prop

value:
  fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
    {seed : GN21DriverSurgePricing.GN21RawCycleSeed |
      (GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma seed).2.1 <
      (GN21DriverSurgePricing.gn21RawPostThinningRaceSeed state sigma seed).1}
  a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSeed](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceAcceptedFirst

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → Prop

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst state sigma) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceExitState

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → Prop

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst state sigma u
   GN21DriverSurgePricing.gn21RawPostThinningRaceAcceptedFirst state sigma n
   {seed : GN21DriverSurgePricing.GN21RawCycleSeed |
    GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionState state sigma seed =
    AppliedModelingLib.Probability.TwoStateSwitching.otherState state})
  a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.otherState](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceAcceptedFirst](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionState](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceHistory

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  GN21DriverSurgePricing.TripPolicy →
  GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21RawPostThinningRaceHistory

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  if a ∈ GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst state sigma then
    Sum.inl (GN21DriverSurgePricing.gn21RawPostThinningRaceMinimum state sigma a)
  else
    Sum.inr
      (GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionState state sigma a,
       GN21DriverSurgePricing.gn21RawPostThinningRaceMinimum state sigma a,
       GN21DriverSurgePricing.AcceptedTripSelection.gn21RawFirstAcceptedTripMark state sigma a)
```

Direct retained dependencies

- [GN21DriverSurgePricing.AcceptedTripSelection.gn21RawFirstAcceptedTripMark](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21RawPostThinningRaceHistory](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionState](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceMinimum](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceNextState

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → Fin 2

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  if a ∈ GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst state sigma then
    AppliedModelingLib.Probability.TwoStateSwitching.otherState state
  else GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionState state sigma a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.otherState](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceCalendarCompletionState](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst](#)

GN21DriverSurgePricing.gn21RawSwitchFirstSplitInput

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21SwitchFirstSplitInput

value:
fun (state : Fin 2) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  ((GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchTailAndFirstGap state a,
    GN21DriverSurgePricing.AcceptedArrivalTime.rawClockStream (GN21DriverSurgePricing.gn21ArrivalClockIndex state) a,
    GN21DriverSurgePricing.AcceptedArrivalTime.rawTripMarkStream state a),
    GN21DriverSurgePricing.AcceptedArrivalTime.rawClockStream
    (GN21DriverSurgePricing.gn21ArrivalClockIndex (AppliedModelingLib.Probability.TwoStateSwitching.otherState state))
    a,
    GN21DriverSurgePricing.AcceptedArrivalTime.rawTripMarkStream
    (AppliedModelingLib.Probability.TwoStateSwitching.otherState state) a)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.otherState](#)
- [GN21DriverSurgePricing.AcceptedArrivalTime.rawClockStream](#)
- [GN21DriverSurgePricing.AcceptedArrivalTime.rawTripMarkStream](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21SwitchFirstSplitInput](#)
- [GN21DriverSurgePricing.RawTwoStateCTMC.rawSwitchTailAndFirstGap](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21ArrivalClockIndex](#)

GN21DriverSurgePricing.gn21RawSwitchFirstJointContinuationInput

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21SwitchFirstJointContinuationInput
value:
fun (state : Fin 2) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  have split : GN21DriverSurgePricing.GN21SwitchFirstSplitInput :=
    GN21DriverSurgePricing.gn21RawSwitchFirstSplitInput state a;
  have incoming : (ℕ → ℝ) × (ℕ → GN21DriverSurgePricing.TripLength) :=
    AppliedModelingLib.Probability.PoissonProcess.externalTimeMarkedResidualTail (fun (t : ℝ) => max 0 t)
    ((split.1.1.2, split.2.1), split.2.2);
  (((split.1.1.2, split.1.1.1, incoming), split.1.2.1), split.1.2.2)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.externalTimeMarkedResidualTail](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21SwitchFirstJointContinuationInput](#)
- [GN21DriverSurgePricing.GN21SwitchFirstSplitInput](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21RawSwitchFirstSplitInput](#)

GN21DriverSurgePricing.gn21RawSwitchFirstJointContinuation

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21SwitchFirstJointContinuation
value:
fun (state : Fin 2) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  have z : GN21DriverSurgePricing.GN21SwitchFirstJointContinuationInput :=
    GN21DriverSurgePricing.gn21RawSwitchFirstJointContinuationInput state a;
  (AppliedModelingLib.Probability.PoissonProcess.externalTimeMarkedResidualTail (fun (t : ℝ) => max 0 t)
    ((z.1.1.1, z.1.2), z.2),
    z.1.1.2)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.externalTimeMarkedResidualTail](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21SwitchFirstJointContinuation](#)
- [GN21DriverSurgePricing.GN21SwitchFirstJointContinuationInput](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21RawSwitchFirstJointContinuationInput](#)

GN21DriverSurgePricing.gn21RawSwitchFirstNextCycleSeed

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21RawCycleSeed
value:
fun (state : Fin 2) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (GN21DriverSurgePricing.gn21RawCycleSeedOfStateContinuation state ◦
    GN21DriverSurgePricing.gn21RawSwitchFirstJointContinuation state)
  a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21SwitchFirstJointContinuation](#)
- [GN21DriverSurgePricing.gn21RawCycleSeedOfStateContinuation](#)
- [GN21DriverSurgePricing.gn21RawSwitchFirstJointContinuation](#)

GN21DriverSurgePricing.gn21RawPostThinningRaceNextCycleSeed

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21RawCycleSeed

value:
fun (state : Fin 2) (sigma : GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  if a ∈ GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst state sigma then
    GN21DriverSurgePricing.gn21RawSwitchFirstNextCycleSeed state a
  else GN21DriverSurgePricing.gn21RawAcceptedCompletionNextCycleSeed state sigma a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawAcceptedCompletionNextCycleSeed](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceSwitchesFirst](#)
- [GN21DriverSurgePricing.gn21RawSwitchFirstNextCycleSeed](#)

GN21DriverSurgePricing.gn21RawCalendarAdvance

Declaration location: paper-local

Lean declaration body

```
type:
(Fin 2 → GN21DriverSurgePricing.TripPolicy) →
  Fin 2 × GN21DriverSurgePricing.GN21RawCycleSeed → Fin 2 × GN21DriverSurgePricing.GN21RawCycleSeed

value:
fun (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (a : Fin 2 × GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (GN21DriverSurgePricing.gn21RawPostThinningRaceNextState a.1 (policy a.1) a.2,
   GN21DriverSurgePricing.gn21RawPostThinningRaceNextCycleSeed a.1 (policy a.1) a.2)
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceNextCycleSeed](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceNextState](#)

GN21DriverSurgePricing.gn21RawCalendarTrace

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.TripPolicy) →
    ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → Fin 2 × GN21DriverSurgePricing.GN21RawCycleSeed

value:
fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (a : ℕ)
  (a_1 : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  Nat.brecOn (motive := fun (x : ℕ) =>
    GN21DriverSurgePricing.GN21RawCycleSeed → Fin 2 × GN21DriverSurgePricing.GN21RawCycleSeed) a
  (GN21DriverSurgePricing.gn21RawCalendarTrace._f initialState policy) a_1
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarAdvance](#)

GN21DriverSurgePricing.gn21RawCalendarTraceSeed

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.TripPolicy) →
  ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21RawCycleSeed

value:
fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (n : ℕ)
  (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (GN21DriverSurgePricing.gn21RawCalendarTrace initialState policy n a).2
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTrace](#)

GN21DriverSurgePricing.gn21RawCalendarNoExitHistoryFold

Declaration location: paper-local

Lean declaration body

```
type:
{β : Type u_1} →
Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.TripPolicy) →
  β →
  (β → GN21DriverSurgePricing.GN21RawPostThinningRaceHistory → β) →
  ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → β

value:
fun {β : Type u_1} (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (initial : β)
  (update : β → GN21DriverSurgePricing.GN21RawPostThinningRaceHistory → β) (a : ℕ)
  (a_1 : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  Nat.brecOn (motive := fun (x : ℕ) => GN21DriverSurgePricing.GN21RawCycleSeed → β) a
  (GN21DriverSurgePricing.gn21RawCalendarNoExitHistoryFold._f initialState policy initial update) a_1
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21RawPostThinningRaceHistory](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTraceSeed](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceHistory](#)

GN21DriverSurgePricing.gn21RawCalendarTraceNoExitEvent

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → Prop

value:
fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (a : ℕ)
  (a_1 : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  Nat.brecOn a (GN21DriverSurgePricing.gn21RawCalendarTraceNoExitEvent._f initialState policy) a_1
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTraceSeed](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceExitState](#)

GN21DriverSurgePricing.gn21RawCalendarTraceExitEvent

Declaration location: paper-local

Lean declaration body

```
type:
  Fin 2 → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → Prop

value:
  fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (n : ℕ)
    (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
    (GN21DriverSurgePricing.gn21RawCalendarTraceNoExitEvent initialState policy n n
      GN21DriverSurgePricing.gn21RawCalendarTraceSeed initialState policy n -1
      GN21DriverSurgePricing.gn21RawPostThinningRaceExitState initialState (policy initialState))
  a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTraceNoExitEvent](#)
- [GN21DriverSurgePricing.gn21RawCalendarTraceSeed](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceExitState](#)

GN21DriverSurgePricing.gn21RawCalendarFirstExitIndex

Declaration location: paper-local

Lean declaration body

```
type:
  Fin 2 → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → GN21DriverSurgePricing.GN21RawCycleSeed → ℕ

value:
  fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy)
    (seed : GN21DriverSurgePricing.GN21RawCycleSeed) =>
    if h : ∃ (n : ℕ), seed ∈ GN21DriverSurgePricing.gn21RawCalendarTraceExitEvent initialState policy n then Nat.find h
    else 0
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTraceExitEvent](#)

GN21DriverSurgePricing.gn21RawCalendarFirstExitHistoryFold

Declaration location: paper-local

Lean declaration body

```
type:
  {β : Type u_1} →
  Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.TripPolicy) →
  β → (β → GN21DriverSurgePricing.GN21RawPostThinningRaceHistory → β) → GN21DriverSurgePricing.GN21RawCycleSeed → β

value:
  fun {β : Type u_1} (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (initial : β)
    (update : β → GN21DriverSurgePricing.GN21RawPostThinningRaceHistory → β)
    (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
    GN21DriverSurgePricing.gn21RawCalendarFirstExitHistoryFold initialState policy initial update
    (GN21DriverSurgePricing.gn21RawCalendarFirstExitIndex initialState policy a + 1) a
```


Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21RawPostThinningRaceHistory](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarFirstExitIndex](#)
- [GN21DriverSurgePricing.gn21RawCalendarNoExitHistoryFold](#)

GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedEarning

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  GN21DriverSurgePricing.PricingFunction →
    (Fin 2 → GN21DriverSurgePricing.TripPolicy) → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ

value:
fun (initialState : Fin 2) (w : GN21DriverSurgePricing.PricingFunction)
  (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  GN21DriverSurgePricing.gn21RawCalendarFirstExitHistoryFold initialState policy 0
  (fun (earned : ℝ) (history : GN21DriverSurgePricing.GN21RawPostThinningRaceHistory) =>
    earned + GN21DriverSurgePricing.gn21RawPostThinningRaceHistoryEarning w history)
a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21RawPostThinningRaceHistory](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarFirstExitHistoryFold](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceHistoryEarning](#)

GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedTime

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ

value:
fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy)
  (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  GN21DriverSurgePricing.gn21RawCalendarFirstExitHistoryFold initialState policy 0
  (fun (elapsed : ℝ) (history : GN21DriverSurgePricing.GN21RawPostThinningRaceHistory) =>
    elapsed + GN21DriverSurgePricing.gn21RawPostThinningRaceHistoryTime history)
a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21RawPostThinningRaceHistory](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarFirstExitHistoryFold](#)
- [GN21DriverSurgePricing.gn21RawPostThinningRaceHistoryTime](#)

GN21DriverSurgePricing.gn21RawCalendarPostExitSeed

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.TripPolicy) →
    GN21DriverSurgePricing.GN21RawCycleSeed → GN21DriverSurgePricing.GN21RawCycleSeed

value:
fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy)
  (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  GN21DriverSurgePricing.gn21RawCalendarTraceSeed initialState policy
  (GN21DriverSurgePricing.gn21RawCalendarFirstExitIndex initialState policy a + 1) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarFirstExitIndex](#)
- [GN21DriverSurgePricing.gn21RawCalendarTraceSeed](#)

GN21DriverSurgePricing.gn21RawCalendarTwoStateEarningAndPostExitSeed

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.PricingFunction) →
    (Fin 2 → GN21DriverSurgePricing.TripPolicy) →
      GN21DriverSurgePricing.GN21RawCycleSeed → ( $\mathbb{R} \times \mathbb{R}$ ) × GN21DriverSurgePricing.GN21RawCycleSeed

value:
fun (initialState : Fin 2) (w : Fin 2 → GN21DriverSurgePricing.PricingFunction)
  (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  have first :  $\mathbb{R} \times$  GN21DriverSurgePricing.GN21RawCycleSeed :=
    (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedEarning initialState (w initialState) policy a,
     GN21DriverSurgePricing.gn21RawCalendarPostExitSeed initialState policy a);
  have other : Fin 2 := AppliedModelingLib.Probability.TwoStateSwitching.otherState initialState;
  have second :  $\mathbb{R} \times$  GN21DriverSurgePricing.GN21RawCycleSeed :=
    (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedEarning other (w other) policy first.2,
     GN21DriverSurgePricing.gn21RawCalendarPostExitSeed other policy first.2);
  ((first.1, second.1), second.2)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.otherState](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedEarning](#)
- [GN21DriverSurgePricing.gn21RawCalendarPostExitSeed](#)

GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateEarning

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.PricingFunction) →
    (Fin 2 → GN21DriverSurgePricing.TripPolicy) →  $\mathbb{N} \rightarrow$  GN21DriverSurgePricing.GN21RawCycleSeed →  $\mathbb{R}$ 

value:
fun (initialState : Fin 2) (w : Fin 2 → GN21DriverSurgePricing.PricingFunction)
  (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (n :  $\mathbb{N}$ ) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (AppliedModelingLib.Probability.regenerativeSummary
   (GN21DriverSurgePricing.gn21RawCalendarTwoStateEarningAndPostExitSeed initialState w policy) n a).1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.regenerativeSummary](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTwoStateEarningAndPostExitSeed](#)

GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateEarning

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.PricingFunction) →
  (Fin 2 → GN21DriverSurgePricing.TripPolicy) →  $\mathbb{N}$  → GN21DriverSurgePricing.GN21RawCycleSeed →  $\mathbb{R}$ 

value:
fun (initialState : Fin 2) (w : Fin 2 → GN21DriverSurgePricing.PricingFunction)
  (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (n :  $\mathbb{N}$ ) (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (AppliedModelingLib.Probability.regenerativeSummary
   (GN21DriverSurgePricing.gn21RawCalendarTwoStateEarningAndPostExitSeed initialState w policy) n a).2
```

Direct retained dependencies

- [AppliedModelingLib.Probability.regenerativeSummary](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTwoStateEarningAndPostExitSeed](#)

GN21DriverSurgePricing.gn21RawCalendarTwoStateTimeAndPostExitSeed

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 →
  (Fin 2 → GN21DriverSurgePricing.TripPolicy) →
  GN21DriverSurgePricing.GN21RawCycleSeed → ( $\mathbb{R} \times \mathbb{R}$ ) × GN21DriverSurgePricing.GN21RawCycleSeed

value:
fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy)
  (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  have first :  $\mathbb{R} \times$  GN21DriverSurgePricing.GN21RawCycleSeed :=
    (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedTime initialState policy a,
     GN21DriverSurgePricing.gn21RawCalendarPostExitSeed initialState policy a);
  have second :  $\mathbb{R} \times$  GN21DriverSurgePricing.GN21RawCycleSeed :=
    (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedTime
     (AppliedModelingLib.Probability.TwoStateSwitching.otherState initialState) policy first.2,
     GN21DriverSurgePricing.gn21RawCalendarPostExitSeed
     (AppliedModelingLib.Probability.TwoStateSwitching.otherState initialState) policy first.2);
  ((first.1, second.1), second.2)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.otherState](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedTime](#)
- [GN21DriverSurgePricing.gn21RawCalendarPostExitSeed](#)

GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateTime

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ

value:
fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (n : ℕ)
  (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (AppliedModelingLib.Probability.regenerativeSummary
   (GN21DriverSurgePricing.gn21RawCalendarTwoStateTimeAndPostExitSeed initialState policy) n a).1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.regenerativeSummary](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTwoStateTimeAndPostExitSeed](#)

GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateTime

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ

value:
fun (initialState : Fin 2) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (n : ℕ)
  (a : GN21DriverSurgePricing.GN21RawCycleSeed) =>
  (AppliedModelingLib.Probability.regenerativeSummary
   (GN21DriverSurgePricing.gn21RawCalendarTwoStateTimeAndPostExitSeed initialState policy) n a).2
```

Direct retained dependencies

- [AppliedModelingLib.Probability.regenerativeSummary](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21RawCalendarTwoStateTimeAndPostExitSeed](#)

GN21DriverSurgePricing.gn21StateRewardRate

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ

value:
fun (meanEarning stateCycleTime : ℝ) => meanEarning / stateCycleTime
```

GN21DriverSurgePricing.gn21SubcycleEarning

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ → ℝ

value:
fun (arrivalRate acceptProb switchRate meanEarning : ℝ) =>
  arrivalRate * acceptProb / (arrivalRate * acceptProb + switchRate) * meanEarning
```

GN21DriverSurgePricing.gn21ExpectedStateEarningInRenewalCycle

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (arrivalRate acceptProb switchRate meanEarning exitWeight :  $\mathbb{R}$ ) =>
  GN21DriverSurgePricing.gn21SubcycleEarning arrivalRate acceptProb switchRate meanEarning /
  GN21DriverSurgePricing.gn21CrossSubcycleProb arrivalRate acceptProb switchRate exitWeight
```

Direct retained dependencies

- [GN21DriverSurgePricing.gn21CrossSubcycleProb](#)
- [GN21DriverSurgePricing.gn21SubcycleEarning](#)

GN21DriverSurgePricing.gn21SubcycleLength

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (arrivalRate acceptProb switchRate stateCycleTime :  $\mathbb{R}$ ) =>
  arrivalRate * acceptProb / (arrivalRate * acceptProb + switchRate) * stateCycleTime
```

GN21DriverSurgePricing.gn21ExpectedStateTimeInRenewalCycle

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (arrivalRate acceptProb switchRate stateCycleTime exitWeight :  $\mathbb{R}$ ) =>
  GN21DriverSurgePricing.gn21SubcycleLength arrivalRate acceptProb switchRate stateCycleTime /
  GN21DriverSurgePricing.gn21CrossSubcycleProb arrivalRate acceptProb switchRate exitWeight
```

Direct retained dependencies

- [GN21DriverSurgePricing.gn21CrossSubcycleProb](#)
- [GN21DriverSurgePricing.gn21SubcycleLength](#)

GN21DriverSurgePricing.gn21SwitchProb

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (lambdalJ lambdajl s :  $\mathbb{R}$ ) => AppliedModelingLib.twoStateCtmcSwitchProb lambdalJ lambdajl s
```

Direct retained dependencies

- [AppliedModelingLib.twoStateCtmcSwitchProb](#)

GN21DriverSurgePricing.ctmcDynamicSwitchProb

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Fin } 2 \rightarrow \text{GN21DriverSurgePricing.TripLength} \rightarrow \mathbb{R}$ 

value:
fun (switch12 switch21 :  $\mathbb{R}$ ) (i : Fin 2) (a : GN21DriverSurgePricing.TripLength) =>
  (if i = 0 then GN21DriverSurgePricing.gn21SwitchProb switch12 switch21
   else GN21DriverSurgePricing.gn21SwitchProb switch21 switch12)
a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21SwitchProb](#)

GN21DriverSurgePricing.gn21ExitWeightIntegral

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → ℝ → ℝ → ℝ → GN21DriverSurgePricing.TripPolicy → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate switchIJ switchJI : ℝ)
  (σ : GN21DriverSurgePricing.TripPolicy) =>
  switchIJ +
  arrivalRate *
  ∫ (τ : GN21DriverSurgePricing.TripLength) in σ, GN21DriverSurgePricing.gn21SwitchProb switchIJ switchJI τ ∂μ
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21SwitchProb](#)

GN21DriverSurgePricing.gn21TimeFractionFormula

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
fun (arrival acceptI timeI exitWeightI arrivalJ acceptJ timeJ exitWeightJ : ℝ) =>
  arrival * acceptI * timeI * exitWeightI /
  (arrivalJ * acceptJ * timeJ * exitWeightJ + arrival * acceptI * timeI * exitWeightJ)
```

GN21DriverSurgePricing.gn21UpdateEndpoint

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → GN21DriverSurgePricing.GN21EndpointVector n → Fin (2 * n) → ℝ → Fin (2 * n) → ENNReal

value:
fun {n : ℕ} (endpoints : GN21DriverSurgePricing.GN21EndpointVector n) (coordinate : Fin (2 * n)) (value : ℝ)
  (a : Fin (2 * n)) =>
  Function.update endpoints coordinate (ENNReal.ofReal value) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21EndpointVector](#)

GN21DriverSurgePricing.gn21UpperEndpointIndex

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → Fin n → Fin (2 * n)

value:
fun {n : ℕ} (i : Fin n) => (2 * ↑i + 1, GN21DriverSurgePricing.gn21UpperEndpointIndex._proof_2 i)
```

GN21DriverSurgePricing.lemma10StructuredLowerDenominator

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (T2 Q2 Tbar1 Qbar1 switch12 :  $\mathbb{R}$ ) => Q2 * (switch12 * Tbar1 - Qbar1) + switch12 * (T2 * switch12 + Q2)
```

GN21DriverSurgePricing.lemma10StructuredLowerNumerator

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (T2 Q2 switch12 :  $\mathbb{R}$ ) => T2 * switch12 + Q2
```

GN21DriverSurgePricing.lemma10StructuredLower

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (T2 Q2 Tbar1 Qbar1 switch12 :  $\mathbb{R}$ ) =>
  -(GN21DriverSurgePricing.lemma10StructuredLowerNumerator T2 Q2 switch12 /
    GN21DriverSurgePricing.lemma10StructuredLowerDenominator T2 Q2 Tbar1 Qbar1 switch12)
```

Direct retained dependencies

- [GN21DriverSurgePricing.lemma10StructuredLowerDenominator](#)
- [GN21DriverSurgePricing.lemma10StructuredLowerNumerator](#)

GN21DriverSurgePricing.lemma10StructuredUpperDenominator

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (Qbar1 switch12 :  $\mathbb{R}$ ) => Qbar1 - switch12
```

GN21DriverSurgePricing.lemma10StructuredUpper

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (_T2 _Q2 _Tbar1 Qbar1 switch12 :  $\mathbb{R}$ ) => 1 / GN21DriverSurgePricing.lemma10StructuredUpperDenominator Qbar1 switch12
```

Direct retained dependencies

- [GN21DriverSurgePricing.lemma10StructuredUpperDenominator](#)

GN21DriverSurgePricing.lemma10StructuredBounds

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Prop}$ 

value:
fun (ratio T2 Q2 Tbar1 Qbar1 switch12 :  $\mathbb{R}$ ) =>
  GN21DriverSurgePricing.lemma10StructuredLower T2 Q2 Tbar1 Qbar1 switch12 < ratio  $\wedge$ 
  ratio < GN21DriverSurgePricing.lemma10StructuredUpper T2 Q2 Tbar1 Qbar1 switch12
```

Direct retained dependencies

- [GN21DriverSurgePricing.lemma10StructuredLower](#)
- [GN21DriverSurgePricing.lemma10StructuredUpper](#)

GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_aggregate_paths

Declaration location: paper-local

Lean declaration body

```
type:
(QiPath WiPath TiPath :  $\mathbb{R} \rightarrow \mathbb{R}$ )  $\rightarrow$ 
(q u wi Qi Qj Ti Tj Wi :  $\mathbb{R}$ )  $\rightarrow$ 
 $\mathbb{R} \rightarrow$ 
(endpointScale :  $\mathbb{R}$ )  $\rightarrow$ 
0 < endpointScale  $\rightarrow$ 
0 < Qj  $\rightarrow$ 
HasDerivAt QiPath (endpointScale * q) u  $\rightarrow$ 
HasDerivAt WiPath (endpointScale * wi) u  $\rightarrow$ 
HasDerivAt TiPath (endpointScale * u) u  $\rightarrow$ 
QiPath u = Qi  $\rightarrow$ 
WiPath u = Wi  $\rightarrow$ 
TiPath u = Ti  $\rightarrow$  Qi * Tj + Qj * Ti  $\neq$  0  $\rightarrow$  GN21DriverSurgePricing.Lemma6EndpointDerivativeData

value:
fun (QiPath WiPath TiPath :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (q u wi Qi Qj Ti Tj Wi Wj endpointScale :  $\mathbb{R}$ ) (hendpointScale_pos : 0 < endpointScale)
(hQj_pos : 0 < Qj) (hQi_deriv : HasDerivAt QiPath (endpointScale * q) u)
(hWi_deriv : HasDerivAt WiPath (endpointScale * wi) u) (hTi_deriv : HasDerivAt TiPath (endpointScale * u) u)
(hQi_val : QiPath u = Qi) (hWi_val : WiPath u = Wi) (hTi_val : TiPath u = Ti) (hden : Qi * Tj + Qj * Ti  $\neq$  0) =>
{
  rewardAlongEndpoint := fun (x :  $\mathbb{R}$ ) =>
    GN21DriverSurgePricing.gn21AggregateDynamicReward (QiPath x) Qj (TiPath x) Tj (WiPath x) Wj,
  derivativeValue :=
    endpointScale * Qj * GN21DriverSurgePricing.gn21DerivativeSignKernel q u wi Qi Qj Ti Tj Wi Wj /
    (Qi * Tj + Qj * Ti) ^ 2,
  q := q, u := u, wi := wi, Qi := Qi, Qj := Qj, Ti := Ti, Tj := Tj, Wi := Wi, Wj := Wj,
  scale := endpointScale * Qj / (Qi * Tj + Qj * Ti) ^ 2,
  scale_pos :=
    GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_aggregate_paths._proof_2 Qi Qj Ti Tj endpointScale
    hendpointScale_pos hQj_pos hden,
  has_derivative :=
    GN21DriverSurgePricing.paper_lemma6_aggregate_reward_hasDerivAt QiPath WiPath TiPath q u wi Qi Qj Ti Tj Wi Wj
    endpointScale hQi_deriv hWi_deriv hTi_deriv hQi_val hWi_val hTi_val hden,
  derivative_eq_scaled_kernel :=
    GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_aggregate_paths._proof_3 q u wi Qi Qj Ti Tj Wi Wj
    endpointScale hendpointScale_pos hQj_pos hden }
```

Direct retained dependencies

- [GN21DriverSurgePricing.Lemma6EndpointDerivativeData](#)
- [GN21DriverSurgePricing.gn21AggregateDynamicReward](#)
- [GN21DriverSurgePricing.gn21DerivativeSignKernel](#)

GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths

Declaration location: paper-local

Lean declaration body

```
type:
(arrivalRate switchRate lowerEndpoint u Qj Tj :  $\mathbb{R}$ )  $\rightarrow$ 
 $\mathbb{R} \rightarrow$ 
(density switchProb payment :  $\mathbb{R} \rightarrow \mathbb{R}$ )  $\rightarrow$ 
0 < arrivalRate  $\rightarrow$ 
0 < density u  $\rightarrow$ 
0 < Qj  $\rightarrow$ 
GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb u * Tj +
Qj * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u  $\neq$ 
0  $\rightarrow$ 
IntervalIntegrable (fun ( $\tau$  :  $\mathbb{R}$ ) => switchProb  $\tau$  * density  $\tau$ ) MeasureTheory.volume lowerEndpoint u  $\rightarrow$ 
```



```

StronglyMeasurableAtFilter (fun (τ : ℝ) => switchProb τ * density τ) (nhds u) MeasureTheory.volume →
ContinuousAt (fun (τ : ℝ) => switchProb τ * density τ) u →
IntervallIntegrable (fun (τ : ℝ) => payment τ * density τ) MeasureTheory.volume lowerEndpoint u →
StronglyMeasurableAtFilter (fun (τ : ℝ) => payment τ * density τ) (nhds u) MeasureTheory.volume →
ContinuousAt (fun (τ : ℝ) => payment τ * density τ) u →
IntervallIntegrable (fun (τ : ℝ) => τ * density τ) MeasureTheory.volume lowerEndpoint u →
StronglyMeasurableAtFilter (fun (τ : ℝ) => τ * density τ) (nhds u) MeasureTheory.volume →
ContinuousAt (fun (τ : ℝ) => τ * density τ) u →
GN21DriverSurgePricing.Lemma6EndpointDerivativeData

value:
fun (arrivalRate switchRate lowerEndpoint u Qj Tj Wj : ℝ) (density switchProb payment : ℝ → ℝ)
(harrival_pos : 0 < arrivalRate) (hdensity_pos : 0 < density u) (hQj_pos : 0 < Qj)
(hden :
  GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb u * Tj +
  Qj * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u ≠
0)
(hq_int : IntervallIntegrable (fun (τ : ℝ) => switchProb τ * density τ) MeasureTheory.volume lowerEndpoint u)
(hq_meas : StronglyMeasurableAtFilter (fun (τ : ℝ) => switchProb τ * density τ) (nhds u) MeasureTheory.volume)
(hq_cont : ContinuousAt (fun (τ : ℝ) => switchProb τ * density τ) u)
(hw_int : IntervallIntegrable (fun (τ : ℝ) => payment τ * density τ) MeasureTheory.volume lowerEndpoint u)
(hw_meas : StronglyMeasurableAtFilter (fun (τ : ℝ) => payment τ * density τ) (nhds u) MeasureTheory.volume)
(hw_cont : ContinuousAt (fun (τ : ℝ) => payment τ * density τ) u)
(ht_int : IntervallIntegrable (fun (τ : ℝ) => τ * density τ) MeasureTheory.volume lowerEndpoint u)
(ht_meas : StronglyMeasurableAtFilter (fun (τ : ℝ) => τ * density τ) (nhds u) MeasureTheory.volume)
(ht_cont : ContinuousAt (fun (τ : ℝ) => τ * density τ) u) =>
GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_aggregate_paths
(GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb)
(GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density payment)
(GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density) (switchProb u) u (payment u)
(GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb u) Qj
(GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u) Tj
(GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density payment u) Wj (arrivalRate * density u)
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths._proof_1 arrivalRate u density
harrival_pos hdensity_pos)
hQj_pos
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths._proof_2 arrivalRate switchRate
lowerEndpoint u density switchProb hq_int hq_meas hq_cont)
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths._proof_3 arrivalRate lowerEndpoint u
density payment hw_int hw_meas hw_cont)
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths._proof_4 arrivalRate lowerEndpoint u
density ht_int ht_meas ht_cont)
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths._proof_5 arrivalRate switchRate
lowerEndpoint u density switchProb)
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths._proof_6 arrivalRate lowerEndpoint u
density payment)
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths._proof_7 arrivalRate lowerEndpoint u
density)
hden

```

Direct retained dependencies

- [GN21DriverSurgePricing.Lemma6EndpointDerivativeData](#)
- [GN21DriverSurgePricing.gn21EndpointQiPath](#)
- [GN21DriverSurgePricing.gn21EndpointTiPath](#)
- [GN21DriverSurgePricing.gn21EndpointWiPath](#)
- [GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_aggregate_paths](#)

GN21DriverSurgePricing.lemma9StructuredLowerDenominator

Declaration location: paper-local

Lean declaration body

```

type:
ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
fun (T1 Q1 Tbar2 Qbar2 switch21 : ℝ) => Q1 * (switch21 * Tbar2 - Qbar2) + switch21 * (Q1 + T1 * switch21)

```

GN21DriverSurgePricing.lemma9StructuredLowerNumerator

Declaration location: paper-local

Lean declaration body

```

type:
ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
fun (T1 Q1 Tbar2 Qbar2 switch21 : ℝ) => T1 * (switch21 * Tbar2 - Qbar2) - (Q1 + T1 * switch21)

```

GN21DriverSurgePricing.lemma9StructuredLower

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (T1 Q1 Tbar2 Qbar2 switch21 :  $\mathbb{R}$ ) =>
  GN21DriverSurgePricing.lemma9StructuredLowerNumerator T1 Q1 Tbar2 Qbar2 switch21 /
  GN21DriverSurgePricing.lemma9StructuredLowerDenominator T1 Q1 Tbar2 Qbar2 switch21
```

Direct retained dependencies

- [GN21DriverSurgePricing.lemma9StructuredLowerDenominator](#)
- [GN21DriverSurgePricing.lemma9StructuredLowerNumerator](#)

GN21DriverSurgePricing.lemma9StructuredUpperDenominator

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (Q1 Qbar2 switch21 :  $\mathbb{R}$ ) => Q1 * (Qbar2 - switch21)
```

GN21DriverSurgePricing.lemma9StructuredUpperNumerator

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (T1 Q1 Qbar2 :  $\mathbb{R}$ ) => Qbar2 * T1 + Q1
```

GN21DriverSurgePricing.lemma9StructuredUpper

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (T1 Q1 Tbar2 Qbar2 switch21 :  $\mathbb{R}$ ) =>
  GN21DriverSurgePricing.lemma9StructuredUpperNumerator T1 Q1 Qbar2 /
  GN21DriverSurgePricing.lemma9StructuredUpperDenominator Q1 Qbar2 switch21
```

Direct retained dependencies

- [GN21DriverSurgePricing.lemma9StructuredUpperDenominator](#)
- [GN21DriverSurgePricing.lemma9StructuredUpperNumerator](#)

GN21DriverSurgePricing.lemma9StructuredBounds

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Prop}$ 

value:
fun (ratio T1 Q1 Tbar2 Qbar2 switch21 :  $\mathbb{R}$ ) =>
  GN21DriverSurgePricing.lemma9StructuredLower T1 Q1 Tbar2 Qbar2 switch21 < ratio  $\wedge$ 
  ratio < GN21DriverSurgePricing.lemma9StructuredUpper T1 Q1 Tbar2 Qbar2 switch21
```

Direct retained dependencies

- [GN21DriverSurgePricing.lemma9StructuredLower](#)
- [GN21DriverSurgePricing.lemma9StructuredUpper](#)

GN21DriverSurgePricing.multiplicativePricing

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \text{GN21DriverSurgePricing.TripLength} \rightarrow \mathbb{R}$ 

value:
fun (m :  $\mathbb{R}$ ) (a : GN21DriverSurgePricing.TripLength) => m * a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.policyAlmostEverywhereEq

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
  GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (σ τ : GN21DriverSurgePricing.TripPolicy) =>
  μ (symmDiff σ τ) = 0
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.ProofBridge.policyEqualitiesModuloNullSets

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.SingleStateReward → Prop

value:
fun (mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (Rhat : GN21DriverSurgePricing.SingleStateReward) =>
  ∀ {left right : GN21DriverSurgePricing.TripPolicy},
    GN21DriverSurgePricing.policyAlmostEverywhereEq mu left right → Rhat left = Rhat right
```

Direct retained dependencies

- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.policyAlmostEverywhereEq](#)

GN21DriverSurgePricing.positiveTripLengths

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripLength → Prop

value:
fun (a : GN21DriverSurgePricing.TripLength) => AppliedModelingLib.positiveRealAcceptAll a
```

Direct retained dependencies

- [AppliedModelingLib.positiveRealAcceptAll](#)
- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.acceptAllPolicy

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripLength → Prop

value:
fun (a : GN21DriverSurgePricing.TripLength) => GN21DriverSurgePricing.positiveTripLengths a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.positiveTripLengths](#)

GN21DriverSurgePricing.acceptAllAlmostEverywhere

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (σ : GN21DriverSurgePricing.TripPolicy) =>
  μ (GN21DriverSurgePricing.acceptAllPolicy \ σ) = 0
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)

GN21DriverSurgePricing.acceptAllDynamicPolicy

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripLength → Prop

value:
fun (a : Fin 2) (a_1 : GN21DriverSurgePricing.TripLength) => GN21DriverSurgePricing.acceptAllPolicy a_1
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)

GN21DriverSurgePricing.dynamicAcceptAllAlmostEverywhere

Declaration location: paper-local

Lean declaration body

```
type:
(Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → Prop

value:
fun (μ : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength)
  (σ : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
  ∀ (i : Fin 2), GN21DriverSurgePricing.acceptAllAlmostEverywhere (μ i) (σ i)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllAlmostEverywhere](#)

GN21DriverSurgePricing.dynamicFeasibleOpenPolicy

Declaration location: paper-local

Lean declaration body

```
type:
  (Fin 2 → GN21DriverSurgePricing.TripPolicy) → Prop

value:
  fun (σ : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
    ∀ (i : Fin 2), σ i ⊆ GN21DriverSurgePricing.acceptAllPolicy ∧ IsOpen (σ i)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)

GN21DriverSurgePricing.dynamicIncentiveCompatible

Declaration location: paper-local

Lean declaration body

```
type:
  GN21DriverSurgePricing.DynamicReward → Prop

value:
  fun (R : GN21DriverSurgePricing.DynamicReward) =>
    GN21DriverSurgePricing.dynamicOptimal R GN21DriverSurgePricing.acceptAllDynamicPolicy
```

Direct retained dependencies

- [GN21DriverSurgePricing.DynamicReward](#)
- [GN21DriverSurgePricing.acceptAllDynamicPolicy](#)
- [GN21DriverSurgePricing.dynamicOptimal](#)

GN21DriverSurgePricing.dynamicOpenOptimal

Declaration location: paper-local

Lean declaration body

```
type:
  GN21DriverSurgePricing.DynamicReward → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → Prop

value:
  fun (R : GN21DriverSurgePricing.DynamicReward) (σ : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
    GN21DriverSurgePricing.dynamicFeasibleOpenPolicy σ ∧
    ∀ (p : Fin 2 → GN21DriverSurgePricing.TripPolicy), GN21DriverSurgePricing.dynamicFeasibleOpenPolicy p → R p ≤ R σ
```

Direct retained dependencies

- [GN21DriverSurgePricing.DynamicReward](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.dynamicFeasibleOpenPolicy](#)

GN21DriverSurgePricing.dynamicProfitableDeviation

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.DynamicReward → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → Prop

value:
fun (R : GN21DriverSurgePricing.DynamicReward) (p : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
  R GN21DriverSurgePricing.acceptAllDynamicPolicy < R p
```

Direct retained dependencies

- [GN21DriverSurgePricing.DynamicReward](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllDynamicPolicy](#)

GN21DriverSurgePricing.gn21AcceptAllExitWeightIntegral

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → ℝ → ℝ → ℝ → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate switchIj switchJl : ℝ) =>
  GN21DriverSurgePricing.gn21ExitWeightIntegral μ arrivalRate switchIj switchJl GN21DriverSurgePricing.acceptAllPolicy
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21ExitWeightIntegral](#)

GN21DriverSurgePricing.gn21SourceUpperEndpointDerivativePositiveAt

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.DynamicReward → Fin 2 → Prop

value:
fun (R : GN21DriverSurgePricing.DynamicReward) (state : Fin 2) =>
  ∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
    GN21DriverSurgePricing.dynamicFeasibleOpenPolicy rho →
    ∀ (upper : GN21DriverSurgePricing.TripLength)
      (endpoint : GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint (rho state) upper),
      ∃ (derivativeValue : ℝ),
        HasDerivAt
          (fun (x : GN21DriverSurgePricing.TripLength) =>
            R
              (Function.update rho state
                (GN21DriverSurgePricing.gn21ComponentUpperPath (rho state) endpoint.lower upper x)))
            derivativeValue upper Λ
          0 < derivativeValue
```

Direct retained dependencies

- [GN21DriverSurgePricing.DynamicReward](#)
- [GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.dynamicFeasibleOpenPolicy](#)
- [GN21DriverSurgePricing.gn21ComponentUpperPath](#)

GN21DriverSurgePricing.gn21Theorem4NonsurgeLiteralEndpointPriceCase

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.DynamicReward →
(Fin 2 → GN21DriverSurgePricing.PricingFunction) → (Fin 2 → GN21DriverSurgePricing.Lemma5DerivativeShape) → Prop

value:
fun (R : GN21DriverSurgePricing.DynamicReward) (w : Fin 2 → GN21DriverSurgePricing.PricingFunction)
(shape : Fin 2 → GN21DriverSurgePricing.Lemma5DerivativeShape) =>
(∃ (m : ℝ) (a : ℝ),
  0 ≤ a ∧
  w 0 = GN21DriverSurgePricing.affinePricing m a ∧
  shape 0 = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyDecreasing) v
(∃ (m : ℝ) (a : ℝ),
  0 < a ∧
  w 0 = GN21DriverSurgePricing.affinePricing m (-a) ∧
  shape 0 = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave) v
GN21DriverSurgePricing.gn21SourceUpperEndpointDerivativePositiveAt R 0 ∧
shape 0 = GN21DriverSurgePricing.Lemma5DerivativeShape.positive
```

Direct retained dependencies

- [GN21DriverSurgePricing.DynamicReward](#)
- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.affinePricing](#)
- [GN21DriverSurgePricing.gn21SourceUpperEndpointDerivativePositiveAt](#)

GN21DriverSurgePricing.gn21Theorem4SurgeLiteralEndpointPriceCase

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.DynamicReward →
(Fin 2 → GN21DriverSurgePricing.PricingFunction) → (Fin 2 → GN21DriverSurgePricing.Lemma5DerivativeShape) → Prop

value:
fun (R : GN21DriverSurgePricing.DynamicReward) (w : Fin 2 → GN21DriverSurgePricing.PricingFunction)
(shape : Fin 2 → GN21DriverSurgePricing.Lemma5DerivativeShape) =>
(∃ (m : ℝ) (a : ℝ),
  0 ≤ a ∧
  w 1 = GN21DriverSurgePricing.affinePricing m (-a) ∧
  shape 1 = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyIncreasing) v
(∃ (m : ℝ) (a : ℝ),
  0 < a ∧
  w 1 = GN21DriverSurgePricing.affinePricing m a ∧
  shape 1 = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex) v
GN21DriverSurgePricing.gn21SourceUpperEndpointDerivativePositiveAt R 1 ∧
shape 1 = GN21DriverSurgePricing.Lemma5DerivativeShape.positive
```

Direct retained dependencies

- [GN21DriverSurgePricing.DynamicReward](#)
- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.affinePricing](#)
- [GN21DriverSurgePricing.gn21SourceUpperEndpointDerivativePositiveAt](#)

GN21DriverSurgePricing.rejectLongTripsPolicy

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → GN21DriverSurgePricing.TripLength → Prop

value:
fun (t : ℝ) (a : GN21DriverSurgePricing.TripLength) => (Set.loi 0 n Set.lio t) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21LeftExtendedCutoffPolicy

Declaration location: paper-local

Lean declaration body

```
type:
ENNReal → GN21DriverSurgePricing.TripLength → Prop

value:
fun (t : ENNReal) (a : GN21DriverSurgePricing.TripLength) =>
  ENNReal.recTopCoe GN21DriverSurgePricing.acceptAllPolicy
    (fun (u : NNReal) => GN21DriverSurgePricing.rejectLongTripsPolicy ↑u) t a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.rejectLongTripsPolicy](#)

GN21DriverSurgePricing.rejectShortTripsPolicy

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → GN21DriverSurgePricing.TripLength → Prop

value:
fun (t : ℝ) (a : GN21DriverSurgePricing.TripLength) => (Set.loi 0 n Set.loi t) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21RightExtendedCutoffPolicy

Declaration location: paper-local

Lean declaration body

```
type:
ENNReal → GN21DriverSurgePricing.TripLength → Prop

value:
fun (t : ENNReal) (a : GN21DriverSurgePricing.TripLength) =>
  ENNReal.recTopCoe ∅ (fun (u : NNReal) => GN21DriverSurgePricing.rejectShortTripsPolicy ↑u) t a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.rejectShortTripsPolicy](#)

GN21DriverSurgePricing.gn21ExtendedMiddlePolicy

Declaration location: paper-local

Lean declaration body

```
type:
ENNReal → ENNReal → GN21DriverSurgePricing.TripLength → Prop

value:
fun (lower upper : ENNReal) (a : GN21DriverSurgePricing.TripLength) =>
  (GN21DriverSurgePricing.gn21RightExtendedCutoffPolicy lower n
   GN21DriverSurgePricing.gn21LeftExtendedCutoffPolicy upper)
a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21LeftExtendedCutoffPolicy](#)
- [GN21DriverSurgePricing.gn21RightExtendedCutoffPolicy](#)

GN21DriverSurgePricing.gn21EndpointVectorPolicy

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → GN21DriverSurgePricing.GN21EndpointVector n → GN21DriverSurgePricing.TripLength → Prop

value:
fun {n : ℕ} (endpoints : GN21DriverSurgePricing.GN21EndpointVector n) (a : GN21DriverSurgePricing.TripLength) =>
  (U (i : Fin n),
   GN21DriverSurgePricing.gn21ExtendedMiddlePolicy (endpoints (GN21DriverSurgePricing.gn21LowerEndpointIndex i))
   (endpoints (GN21DriverSurgePricing.gn21UpperEndpointIndex i)))
a
```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21EndpointVector](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21ExtendedMiddlePolicy](#)
- [GN21DriverSurgePricing.gn21LowerEndpointIndex](#)
- [GN21DriverSurgePricing.gn21UpperEndpointIndex](#)

GN21DriverSurgePricing.gn21ExtendedTwoTailPolicy

Declaration location: paper-local

Lean declaration body

```
type:
ENNReal → ENNReal → GN21DriverSurgePricing.TripLength → Prop

value:
fun (lower upper : ENNReal) (a : GN21DriverSurgePricing.TripLength) =>
  (GN21DriverSurgePricing.gn21LeftExtendedCutoffPolicy lower u
   GN21DriverSurgePricing.gn21RightExtendedCutoffPolicy upper)
a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21LeftExtendedCutoffPolicy](#)
- [GN21DriverSurgePricing.gn21RightExtendedCutoffPolicy](#)

GN21DriverSurgePricing.lemma5SourcePolicyForm

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.Lemma5DerivativeShape → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (shape : GN21DriverSurgePricing.Lemma5DerivativeShape) (policy : GN21DriverSurgePricing.TripPolicy) =>
  match shape with
  | GN21DriverSurgePricing.Lemma5DerivativeShape.positive => policy = GN21DriverSurgePricing.acceptAllPolicy
  | GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyIncreasing =>
    ∃ (cutoff : ENNReal), policy = GN21DriverSurgePricing.gn21RightExtendedCutoffPolicy ↑ cutoff
  | GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyDecreasing =>
    ∃ (cutoff : ENNReal), policy = GN21DriverSurgePricing.gn21LeftExtendedCutoffPolicy cutoff
  | GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex =>
    ∃ (lower : ENNReal) (upper : ENNReal), policy = GN21DriverSurgePricing.gn21ExtendedTwoTailPolicy lower upper
  | GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave =>
    ∃ (lower : ENNReal) (upper : ENNReal), policy = GN21DriverSurgePricing.gn21ExtendedMiddlePolicy lower upper
```

Direct retained dependencies

- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21ExtendedMiddlePolicy](#)
- [GN21DriverSurgePricing.gn21ExtendedTwoTailPolicy](#)
- [GN21DriverSurgePricing.gn21LeftExtendedCutoffPolicy](#)
- [GN21DriverSurgePricing.gn21RightExtendedCutoffPolicy](#)

GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
GN21DriverSurgePricing.Lemma5DerivativeShape → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength)
  (shape : GN21DriverSurgePricing.Lemma5DerivativeShape) (policy : GN21DriverSurgePricing.TripPolicy) =>
  ∃ (representative : GN21DriverSurgePricing.TripPolicy),
    GN21DriverSurgePricing.lemma5SourcePolicyForm shape representative ∧
    GN21DriverSurgePricing.policyAlmostEverywhereEq mu policy representative
```

Direct retained dependencies

- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.lemma5SourcePolicyForm](#)
- [GN21DriverSurgePricing.policyAlmostEverywhereEq](#)

GN21DriverSurgePricing.rejectsLongTrips

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (t : ℝ) (σ : GN21DriverSurgePricing.TripPolicy) =>
  ∀ {τ : GN21DriverSurgePricing.TripLength}, 0 < τ → (τ ∈ σ ↔ τ < t)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoff

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (σ : GN21DriverSurgePricing.TripPolicy) =>
  (∃ (t : ℝ), GN21DriverSurgePricing.rejectsLongTrips t σ) ∨ GN21DriverSurgePricing.acceptsAllTrips σ
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptsAllTrips](#)
- [GN21DriverSurgePricing.rejectsLongTrips](#)

GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoffAlmostEverywhere

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (σ : GN21DriverSurgePricing.TripPolicy) =>
  ∃ (ostar : GN21DriverSurgePricing.TripPolicy),
    GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoff ostar ∧
    GN21DriverSurgePricing.policyAlmostEverywhereEq μ σ ostar
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.policyAlmostEverywhereEq](#)
- [GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoff](#)

GN21DriverSurgePricing.rejectsShortTrips

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (t : ℝ) (σ : GN21DriverSurgePricing.TripPolicy) =>
  ∀ {τ : GN21DriverSurgePricing.TripLength}, 0 < τ → (τ ∈ σ ↔ t < τ)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.rejectsShortTripsFiniteOrInfiniteCutoffAlmostEverywhere

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripPolicy → Prop

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (σ : GN21DriverSurgePricing.TripPolicy) =>
  ∃ (ostar : GN21DriverSurgePricing.TripPolicy),
    ((∃ (t : ℝ), GN21DriverSurgePricing.rejectsShortTrips t ostar) ∨ ostar = ∅) ∧
    GN21DriverSurgePricing.policyAlmostEverywhereEq μ σ ostar
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.policyAlmostEverywhereEq](#)
- [GN21DriverSurgePricing.rejectsShortTrips](#)

GN21DriverSurgePricing.sameStrictSign

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → Prop

value:
fun (x y : ℝ) => (0 < x ↔ 0 < y) ∧ (x < 0 ↔ y < 0)
```

GN21DriverSurgePricing.GN21Lemma5ComponentLowerDerivativeCondition

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.SingleStateReward →
(GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ) → Prop

value:
fun (Rhat : GN21DriverSurgePricing.SingleStateReward)
  (response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ) =>
  ∀ {policy : GN21DriverSurgePricing.TripPolicy} {point lower upper : GN21DriverSurgePricing.TripLength},
  isOpen policy →
  policy ⊆ GN21DriverSurgePricing.acceptAllPolicy →
  point ∈ policy →
  connectedComponentIn policy point = Set.lcc lower upper →
  0 < lower →
  lower < upper →
  ∃ (derivativeValue : ℝ),
  HasDerivAt
    (fun (x : GN21DriverSurgePricing.TripLength) =>
      Rhat (GN21DriverSurgePricing.gn21Lemma5ComponentLowerPath policy lower upper x))
    derivativeValue lower ∧
  GN21DriverSurgePricing.sameStrictSign derivativeValue (-response policy lower)
```

Direct retained dependencies

- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21Lemma5ComponentLowerPath](#)
- [GN21DriverSurgePricing.sameStrictSign](#)

GN21DriverSurgePricing.GN21Lemma5ComponentLowerRightDerivativeCondition

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.SingleStateReward →
(GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ) → Prop

value:
fun (Rhat : GN21DriverSurgePricing.SingleStateReward)
  (response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ) =>
  ∀ {policy : GN21DriverSurgePricing.TripPolicy} {point upper : GN21DriverSurgePricing.TripLength},
  isOpen policy →
  policy ⊆ GN21DriverSurgePricing.acceptAllPolicy →
  point ∈ policy →
  connectedComponentIn policy point = Set.lcc 0 upper →
```

```

0 < upper →
  ∃ (derivativeValue : ℝ),
    HasDerivWithinAt
      (fun (x : GN21DriverSurgePricing.TripLength) =>
        Rhat (GN21DriverSurgePricing.gn21Lemma5ComponentLowerPath policy 0 upper x))
      derivativeValue (Set.Ici 0) 0 ∧
    GN21DriverSurgePricing.sameStrictSign derivativeValue (-response policy 0)

```

Direct retained dependencies

- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21Lemma5ComponentLowerPath](#)
- [GN21DriverSurgePricing.sameStrictSign](#)

GN21DriverSurgePricing.GN21Lemma5ComponentTailLowerDerivativeCondition

Declaration location: paper-local

Lean declaration body

```

type:
GN21DriverSurgePricing.SingleStateReward →
  (GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ) → Prop

value:
fun (Rhat : GN21DriverSurgePricing.SingleStateReward)
  (response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ) =>
  ∀ {policy : GN21DriverSurgePricing.TripPolicy} {point lower : GN21DriverSurgePricing.TripLength},
    IsOpen policy →
      policy ⊆ GN21DriverSurgePricing.acceptAllPolicy →
        point ∈ policy →
          connectedComponentIn policy point = Set.Ioi lower →
            0 < lower →
              ∃ (derivativeValue : ℝ),
                HasDerivAt
                  (fun (x : GN21DriverSurgePricing.TripLength) =>
                    Rhat (GN21DriverSurgePricing.gn21Lemma5ComponentTailLowerPath policy lower x))
                  derivativeValue lower ∧
                GN21DriverSurgePricing.sameStrictSign derivativeValue (-response policy lower)

```

Direct retained dependencies

- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21Lemma5ComponentTailLowerPath](#)
- [GN21DriverSurgePricing.sameStrictSign](#)

GN21DriverSurgePricing.GN21Lemma5ComponentUpperDerivativeCondition

Declaration location: paper-local

Lean declaration body

```

type:
GN21DriverSurgePricing.SingleStateReward →
  (GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ) → Prop

value:
fun (Rhat : GN21DriverSurgePricing.SingleStateReward)
  (response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ) =>
  ∀ {policy : GN21DriverSurgePricing.TripPolicy} {point lower upper : GN21DriverSurgePricing.TripLength},
    IsOpen policy →
      policy ⊆ GN21DriverSurgePricing.acceptAllPolicy →
        point ∈ policy →
          connectedComponentIn policy point = Set.Ioo lower upper →
            0 ≤ lower →
              lower < upper →
                ∃ (derivativeValue : ℝ),

```

```

HasDerivAt
  (fun (x : GN21DriverSurgePricing.TripLength) =>
    Rhat (GN21DriverSurgePricing.gn21Lemma5ComponentUpperPath policy lower upper x))
  derivativeValue upper  $\wedge$ 
  GN21DriverSurgePricing.sameStrictSign derivativeValue (response policy upper)

```

Direct retained dependencies

- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21Lemma5ComponentUpperPath](#)
- [GN21DriverSurgePricing.sameStrictSign](#)

GN21DriverSurgePricing.singleStateMeasurableIncentiveCompatible

Declaration location: paper-local

Lean declaration body

```

type:
  GN21DriverSurgePricing.SingleStateReward → Prop

value:
  fun (R : GN21DriverSurgePricing.SingleStateReward) =>
     $\forall \sigma \subseteq \text{GN21DriverSurgePricing.acceptAllPolicy}, \text{MeasurableSet } \sigma \rightarrow R \sigma \leq R \text{ GN21DriverSurgePricing.acceptAllPolicy}$ 

```

Direct retained dependencies

- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)

GN21DriverSurgePricing.singleStateMeasurableOptimal

Declaration location: paper-local

Lean declaration body

```

type:
  GN21DriverSurgePricing.SingleStateReward → GN21DriverSurgePricing.TripPolicy → Prop

value:
  fun (R : GN21DriverSurgePricing.SingleStateReward) ( $\sigma$  : GN21DriverSurgePricing.TripPolicy) =>
     $\sigma \subseteq \text{GN21DriverSurgePricing.acceptAllPolicy} \wedge$ 
     $\text{MeasurableSet } \sigma \wedge \forall \rho \subseteq \text{GN21DriverSurgePricing.acceptAllPolicy}, \text{MeasurableSet } \rho \rightarrow R \rho \leq R \sigma$ 

```

Direct retained dependencies

- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)

GN21DriverSurgePricing.singleStateTripMass

Declaration location: paper-local

Lean declaration body

```

type:
  MeasureTheory.Measure GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripPolicy →  $\mathbb{R}$ 

value:
  fun ( $\mu$  : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) ( $\sigma$  : GN21DriverSurgePricing.TripPolicy) =>
    ( $\mu \sigma$ ).toReal

```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.singleStateTripPayment

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
  GN21DriverSurgePricing.PricingFunction → GN21DriverSurgePricing.TripPolicy → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (w : GN21DriverSurgePricing.PricingFunction)
  (σ : GN21DriverSurgePricing.TripPolicy) =>
  ∫ (τ : GN21DriverSurgePricing.TripLength) in σ, w τ ∂μ
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.gn21ScaledStateEarning

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
  ℝ → GN21DriverSurgePricing.PricingFunction → GN21DriverSurgePricing.TripPolicy → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ)
  (w : GN21DriverSurgePricing.PricingFunction) (σ : GN21DriverSurgePricing.TripPolicy) =>
  arrivalRate * GN21DriverSurgePricing.singleStateTripPayment μ w σ
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.singleStateTripPayment](#)

GN21DriverSurgePricing.gn21StateMeanEarning

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
  GN21DriverSurgePricing.PricingFunction → GN21DriverSurgePricing.TripPolicy → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (w : GN21DriverSurgePricing.PricingFunction)
  (σ : GN21DriverSurgePricing.TripPolicy) =>
  GN21DriverSurgePricing.singleStateTripPayment μ w σ / GN21DriverSurgePricing.singleStateTripMass μ σ
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)
- [GN21DriverSurgePricing.singleStateTripPayment](#)

GN21DriverSurgePricing.singleStateTripTime

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → GN21DriverSurgePricing.TripPolicy → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (σ : GN21DriverSurgePricing.TripPolicy) =>
  ∫ (τ : GN21DriverSurgePricing.TripLength) in σ, τ ∂μ
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.affineSingleStateRenewalReward

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → ℝ → ℝ → ℝ → GN21DriverSurgePricing.TripPolicy → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate m a : ℝ)
  (σ : GN21DriverSurgePricing.TripPolicy) =>
  arrivalRate *
    (m * GN21DriverSurgePricing.singleStateTripTime μ σ + a * GN21DriverSurgePricing.singleStateTripMass μ σ) /
    (1 + arrivalRate * GN21DriverSurgePricing.singleStateTripTime μ σ)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)
- [GN21DriverSurgePricing.singleStateTripTime](#)

GN21DriverSurgePricing.gn21ScaledStateTime

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → ℝ → GN21DriverSurgePricing.TripPolicy → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ)
  (σ : GN21DriverSurgePricing.TripPolicy) =>
  1 + arrivalRate * GN21DriverSurgePricing.singleStateTripTime μ σ
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.singleStateTripTime](#)

GN21DriverSurgePricing.gn21AcceptAllScaledStateTime

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength → ℝ → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ) =>
  GN21DriverSurgePricing.gn21ScaledStateTime μ arrivalRate GN21DriverSurgePricing.acceptAllPolicy
```


Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21ScaledStateTime](#)

GN21DriverSurgePricing.gn21MeasuredAggregateRewardPrimitives

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
 $\mathbb{R} \rightarrow$ 
 $\mathbb{R} \rightarrow$ 
 $\mathbb{R} \rightarrow$ 
 $\mathbb{R} \rightarrow$ 
GN21DriverSurgePricing.PricingFunction →
GN21DriverSurgePricing.PricingFunction →
GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripPolicy →  $\mathbb{R}$ 

value:
fun (μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival arrivalJ switchIJ switchJI :  $\mathbb{R}$ )
(wl wj : GN21DriverSurgePricing.PricingFunction) (σl σj : GN21DriverSurgePricing.TripPolicy) =>
GN21DriverSurgePricing.gn21AggregateDynamicReward
(GN21DriverSurgePricing.gn21ExitWeightIntegral μl arrivalI switchIJ switchJI σl)
(GN21DriverSurgePricing.gn21ExitWeightIntegral μj arrivalJ switchJI switchIJ σj)
(GN21DriverSurgePricing.gn21ScaledStateTime μl arrivalI σl)
(GN21DriverSurgePricing.gn21ScaledStateTime μj arrivalJ σj)
(GN21DriverSurgePricing.gn21ScaledStateEarning μl arrivalI wl σl)
(GN21DriverSurgePricing.gn21ScaledStateEarning μj arrivalJ wj σj)
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21AggregateDynamicReward](#)
- [GN21DriverSurgePricing.gn21ExitWeightIntegral](#)
- [GN21DriverSurgePricing.gn21ScaledStateEarning](#)
- [GN21DriverSurgePricing.gn21ScaledStateTime](#)

GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional

Declaration location: paper-local

Lean declaration body

```
type:
(Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) →
(Fin 2 →  $\mathbb{R}$ ) →
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow$  (Fin 2 → GN21DriverSurgePricing.PricingFunction) → (Fin 2 → GN21DriverSurgePricing.TripPolicy) →  $\mathbb{R}$ 

value:
fun (μ : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 →  $\mathbb{R}$ ) (switch12 switch21 :  $\mathbb{R}$ )
(w : Fin 2 → GN21DriverSurgePricing.PricingFunction) (a : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
GN21DriverSurgePricing.gn21MeasuredAggregateRewardPrimitives (μ 0) (μ 1) (arrival 0) (arrival 1) switch12 switch21
(w 0) (w 1) (a 0) (a 1)
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21MeasuredAggregateRewardPrimitives](#)

GN21DriverSurgePricing.gn21AggregateMultiplicativeDynamicReward

Declaration location: paper-local

Lean declaration body

```
type:
  (Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) →
  (Fin 2 → ℝ) → ℝ → ℝ → (Fin 2 → ℝ) → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → ℝ

value:
  fun (μ : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ) (switch12 switch21 : ℝ)
    (m : Fin 2 → ℝ) (a : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
    GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional μ arrival switch12 switch21
    (fun (i : Fin 2) => GN21DriverSurgePricing.multiplicativePricing (m i)) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional](#)
- [GN21DriverSurgePricing.multiplicativePricing](#)

GN21DriverSurgePricing.gn21StateCycleTime

Declaration location: paper-local

Lean declaration body

```
type:
  MeasureTheory.Measure GN21DriverSurgePricing.TripLength → ℝ → GN21DriverSurgePricing.TripPolicy → ℝ

value:
  fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ)
    (σ : GN21DriverSurgePricing.TripPolicy) =>
    1 / (arrivalRate * GN21DriverSurgePricing.singleStateTripMass μ σ) +
    GN21DriverSurgePricing.singleStateTripTime μ σ / GN21DriverSurgePricing.singleStateTripMass μ σ
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)
- [GN21DriverSurgePricing.singleStateTripTime](#)

GN21DriverSurgePricing.gn21MeasuredStateRewardRate

Declaration location: paper-local

Lean declaration body

```
type:
  MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
  ℝ → GN21DriverSurgePricing.PricingFunction → GN21DriverSurgePricing.TripPolicy → ℝ

value:
  fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ)
    (w : GN21DriverSurgePricing.PricingFunction) (σ : GN21DriverSurgePricing.TripPolicy) =>
    GN21DriverSurgePricing.gn21StateRewardRate (GN21DriverSurgePricing.gn21StateMeanEarning μ w σ)
    (GN21DriverSurgePricing.gn21StateCycleTime μ arrivalRate σ)
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21StateCycleTime](#)
- [GN21DriverSurgePricing.gn21StateMeanEarning](#)
- [GN21DriverSurgePricing.gn21StateRewardRate](#)

GN21DriverSurgePricing.gn21MeasuredTimeFraction

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{GN21DriverSurgePricing.TripPolicy} \rightarrow \text{GN21DriverSurgePricing.TripPolicy} \rightarrow \mathbb{R}$ 

value:
fun (μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivall arrivall switchl switchl :  $\mathbb{R}$ )
(σl σj : GN21DriverSurgePricing.TripPolicy) =>
GN21DriverSurgePricing.gn21TimeFractionFormula arrivall (GN21DriverSurgePricing.singleStateTripMass μl σl)
(GN21DriverSurgePricing.gn21StateCycleTime μl arrivall σl)
(GN21DriverSurgePricing.gn21ExitWeightIntegral μj arrivall switchl switchl σj) arrivall
(GN21DriverSurgePricing.singleStateTripMass μj σj) (GN21DriverSurgePricing.gn21StateCycleTime μj arrivall σj)
(GN21DriverSurgePricing.gn21ExitWeightIntegral μl arrivall switchl switchl σl)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21ExitWeightIntegral](#)
- [GN21DriverSurgePricing.gn21StateCycleTime](#)
- [GN21DriverSurgePricing.gn21TimeFractionFormula](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)

GN21DriverSurgePricing.gn21MeasuredDynamicReward

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
 $\mathbb{R} \rightarrow$ 
 $\mathbb{R} \rightarrow$ 
 $\mathbb{R} \rightarrow$ 
 $\mathbb{R} \rightarrow$ 
GN21DriverSurgePricing.PricingFunction →
GN21DriverSurgePricing.PricingFunction →
GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripPolicy →  $\mathbb{R}$ 

value:
fun (μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivall arrivall switchl switchl :  $\mathbb{R}$ )
(wl wj : GN21DriverSurgePricing.PricingFunction) (σl σj : GN21DriverSurgePricing.TripPolicy) =>
GN21DriverSurgePricing.gn21DynamicRewardFormula
(GN21DriverSurgePricing.gn21MeasuredTimeFraction μl μj arrivall arrivall switchl switchl σl σj)
(GN21DriverSurgePricing.gn21MeasuredStateRewardRate μl arrivall wl σl)
(GN21DriverSurgePricing.gn21MeasuredTimeFraction μj μl arrivall switchl switchl σj σl)
(GN21DriverSurgePricing.gn21MeasuredStateRewardRate μj arrivall wj σj)
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21DynamicRewardFormula](#)
- [GN21DriverSurgePricing.gn21MeasuredStateRewardRate](#)
- [GN21DriverSurgePricing.gn21MeasuredTimeFraction](#)

GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional

Declaration location: paper-local

Lean declaration body

```
type:
(Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) →
(Fin 2 → ℝ) →
ℝ → ℝ → (Fin 2 → GN21DriverSurgePricing.PricingFunction) → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → ℝ

value:
fun (μ : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ) (switch12 switch21 : ℝ)
(w : Fin 2 → GN21DriverSurgePricing.PricingFunction) (a : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
GN21DriverSurgePricing.gn21MeasuredDynamicReward (μ 0) (μ 1) (arrival 0) (arrival 1) switch12 switch21 (w 0) (w 1)
(a 0) (a 1)
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21MeasuredDynamicReward](#)

GN21DriverSurgePricing.gn21SourceSurgeStateDominance

Declaration location: paper-local

Lean declaration body

```
type:
(Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) →
(Fin 2 → ℝ) → (Fin 2 → GN21DriverSurgePricing.PricingFunction) → Prop

value:
fun (μ : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ)
(w : Fin 2 → GN21DriverSurgePricing.PricingFunction) =>
∃ σ2 ⊆ GN21DriverSurgePricing.acceptAllPolicy,
  IsOpen σ2 ∧
  ∀ σ1 ⊆ GN21DriverSurgePricing.acceptAllPolicy,
    IsOpen σ1 →
      GN21DriverSurgePricing.gn21MeasuredStateRewardRate (μ 0) (arrival 0) (w 0) σ1 <
        GN21DriverSurgePricing.gn21MeasuredStateRewardRate (μ 1) (arrival 1) (w 1) σ2
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21MeasuredStateRewardRate](#)

GN21DriverSurgePricing.singleStateRenewalReward

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength →
ℝ → GN21DriverSurgePricing.PricingFunction → GN21DriverSurgePricing.TripPolicy → ℝ

value:
fun (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ)
(w : GN21DriverSurgePricing.PricingFunction) (σ : GN21DriverSurgePricing.TripPolicy) =>
arrivalRate * GN21DriverSurgePricing.singleStateTripPayment μ w σ /
(1 + arrivalRate * GN21DriverSurgePricing.singleStateTripTime μ σ)
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.singleStateTripPayment](#)
- [GN21DriverSurgePricing.singleStateTripTime](#)

GN21DriverSurgePricing.strictQuasiConcaveOnPositive

Declaration location: paper-local

Lean declaration body

```
type:
  (GN21DriverSurgePricing.TripLength → ℝ) → Prop
value:
  fun (f : GN21DriverSurgePricing.TripLength → ℝ) =>
    ∀ (x y θ : ℝ), 0 < x → 0 < y → x ≠ y → 0 < θ → θ < 1 → min (f x) (f y) < f (θ * x + (1 - θ) * y)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.strictQuasiConvexOnPositive

Declaration location: paper-local

Lean declaration body

```
type:
  (GN21DriverSurgePricing.TripLength → ℝ) → Prop
value:
  fun (f : GN21DriverSurgePricing.TripLength → ℝ) =>
    ∀ (x y θ : ℝ), 0 < x → 0 < y → x ≠ y → 0 < θ → θ < 1 → f (θ * x + (1 - θ) * y) < max (f x) (f y)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.strictThresholdPolicy

Declaration location: paper-local

Lean declaration body

```
type:
  GN21DriverSurgePricing.PricingFunction → ℝ → GN21DriverSurgePricing.TripLength → Prop
value:
  fun (w : GN21DriverSurgePricing.PricingFunction) (c : ℝ) (a : GN21DriverSurgePricing.TripLength) =>
    (Set.lci 0 n {τ : GN21DriverSurgePricing.TripLength | c < w τ / τ}) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

GN21DriverSurgePricing.structuredSurgePrice

Declaration location: paper-local

Lean declaration body

```
type:
  ℝ → ℝ → (GN21DriverSurgePricing.TripLength → ℝ) → GN21DriverSurgePricing.TripLength → ℝ
value:
  fun (m z : ℝ) (q : GN21DriverSurgePricing.TripLength → ℝ) (a : GN21DriverSurgePricing.TripLength) => m * a + z * q a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice

Declaration location: paper-local

Lean declaration body

```
type:
(Fin 2 → ℝ) → (Fin 2 → ℝ) → ℝ → ℝ → Fin 2 → GN21DriverSurgePricing.TripLength → ℝ

value:
fun (m z : Fin 2 → ℝ) (switch12 switch21 : ℝ) (a : Fin 2) (a_1 : GN21DriverSurgePricing.TripLength) =>
  GN21DriverSurgePricing.structuredSurgePrice (m a) (z a)
  (GN21DriverSurgePricing.ctmcDynamicSwitchProb switch12 switch21 a) a_1
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.ctmcDynamicSwitchProb](#)
- [GN21DriverSurgePricing.structuredSurgePrice](#)

GN21DriverSurgePricing.ctmcStructuredSurgePrice

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ → GN21DriverSurgePricing.TripLength → ℝ

value:
fun (m z lambdaJl lambdaJl : ℝ) (a : GN21DriverSurgePricing.TripLength) =>
  GN21DriverSurgePricing.structuredSurgePrice m z (GN21DriverSurgePricing.gn21SwitchProb lambdaJl lambdaJl) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21SwitchProb](#)
- [GN21DriverSurgePricing.structuredSurgePrice](#)

GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward

Declaration location: paper-local

Lean declaration body

```
type:
(Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) →
(Fin 2 → ℝ) → ℝ → ℝ → (Fin 2 → ℝ) → (Fin 2 → ℝ) → (Fin 2 → GN21DriverSurgePricing.TripPolicy) → ℝ

value:
fun (μ : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ) (switch12 switch21 : ℝ)
  (m z : Fin 2 → ℝ) (a : Fin 2 → GN21DriverSurgePricing.TripPolicy) =>
  GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional μ arrival switch12 switch21
  (GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21) a
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice](#)
- [GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional](#)

GN21DriverSurgePricing.theorem2BothStatesContinuousArrival

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → ℝ

value:
fun (a : Fin 2) =>
  match a with
  | 0 => 1
  | 1 => 2
```

GN21DriverSurgePricing.theorem2BothStatesContinuousLongUniform

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength

value:
50 • MeasureTheory.volume.restrict (Set.Icc (599 / 100) (601 / 100))
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.theorem2BothStatesContinuousM

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → ℝ

value:
fun (a : Fin 2) =>
  match a with
  | 0 => 1
  | 1 => 10
```

GN21DriverSurgePricing.theorem2BothStatesContinuousNonsurgeDeviation

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripLength → Prop

value:
fun (a : Fin 2) (a_1 : GN21DriverSurgePricing.TripLength) =>
  (match a with
  | 0 => GN21DriverSurgePricing.rejectLongTripsPolicy 2
  | 1 => GN21DriverSurgePricing.acceptAllPolicy)
a_1
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.rejectLongTripsPolicy](#)

GN21DriverSurgePricing.theorem2BothStatesContinuousShortUniform

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength

value:
50 • MeasureTheory.volume.restrict (Set.Icc (99 / 100) (101 / 100))
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)

GN21DriverSurgePricing.theorem2BothStatesContinuousMix

Declaration location: paper-local

Lean declaration body

```
type:
MeasureTheory.Measure GN21DriverSurgePricing.TripLength

value:
(1 / 2) • GN21DriverSurgePricing.theorem2BothStatesContinuousShortUniform +
(1 / 2) • GN21DriverSurgePricing.theorem2BothStatesContinuousLongUniform
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.theorem2BothStatesContinuousLongUniform](#)
- [GN21DriverSurgePricing.theorem2BothStatesContinuousShortUniform](#)

GN21DriverSurgePricing.theorem2BothStatesContinuousMu

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength

value:
fun (a : Fin 2) =>
  match a with
  | x => GN21DriverSurgePricing.theorem2BothStatesContinuousMix
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.theorem2BothStatesContinuousMix](#)

GN21DriverSurgePricing.theorem2BothStatesContinuousSurgeDeviation

Declaration location: paper-local

Lean declaration body

```
type:
Fin 2 → GN21DriverSurgePricing.TripLength → Prop

value:
fun (a : Fin 2) (a_1 : GN21DriverSurgePricing.TripLength) =>
  (match a with
   | 0 => GN21DriverSurgePricing.acceptAllPolicy
   | 1 => GN21DriverSurgePricing.rejectShortTripsPolicy 2)
  a_1
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.rejectShortTripsPolicy](#)

GN21DriverSurgePricing.theorem3FeasibilityDenominator

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
fun (T1 T2 Q1 Q2 switch12 : ℝ) => Q2 * (switch12 * T1 - Q1) + switch12 * (T2 * switch12 + Q2)
```


GN21DriverSurgePricing.theorem3FeasibilityNumerator

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (T1 T2 Q1 Q2 switch12 :  $\mathbb{R}$ ) => Q2 * (switch12 * T1 - Q1) + Q1 * (T2 * switch12 + Q2)
```

GN21DriverSurgePricing.theorem3FeasibilityThresholdC

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (T1 T2 Q1 Q2 switch12 :  $\mathbb{R}$ ) =>
  1 -
  GN21DriverSurgePricing.theorem3FeasibilityNumerator T1 T2 Q1 Q2 switch12 /
  (T1 * GN21DriverSurgePricing.theorem3FeasibilityDenominator T1 T2 Q1 Q2 switch12)
```

Direct retained dependencies

- [GN21DriverSurgePricing.theorem3FeasibilityDenominator](#)
- [GN21DriverSurgePricing.theorem3FeasibilityNumerator](#)

GN21DriverSurgePricing.thresholdRatePolicy

Declaration location: paper-local

Lean declaration body

```
type:
GN21DriverSurgePricing.PricingFunction  $\rightarrow \mathbb{R} \rightarrow$  GN21DriverSurgePricing.TripPolicy  $\rightarrow$  Prop

value:
fun (w : GN21DriverSurgePricing.PricingFunction) (c :  $\mathbb{R}$ ) ( $\sigma$  : GN21DriverSurgePricing.TripPolicy) =>
   $\forall \{\tau : \text{GN21DriverSurgePricing.TripLength}\}, 0 < \tau \rightarrow (\tau \in \sigma \leftrightarrow c \leq w \ \tau / \tau)$ 
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

Paper-specific semantic prerequisites

GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel

Paper-local declaration:

GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel

Source locator: cited publication, lines 217-252

Verbatim paper-source connection

We start with a model where there is a single world state, i.e., all model components are constant over time. Time is continuous and indexed by t . At each time t , the driver is either open, or busy. While open, the driver receives job (trip) requests from riders according to a Poisson process at rate λ , i.e., the time between requests is exponential with mean λ^{-1} . Job lengths, denoted by τ , are drawn independently and identically from a continuous distribution F . If the driver accepts a job request of length τ at time t (as discussed below), they receive a payout of $w(\tau)$ at time $t + \tau$, at which time they become open again. Otherwise, the driver remains open. Except where specified, the only assumptions on w are that it is continuous and

asymptotically (sub)-linear: $\exists c : \liminf_{\tau \rightarrow \infty} w(\tau) \leq c$, which ensures that the driver reward is also bounded.³

2.1.2
Dynamic model with surge pricing

A model with fixed pricing and arrival rates of jobs is not a realistic representation of ride-hailing platforms. In particular, rider demand (both in intensity and in distribution) may vary substantially over time, even within a day (cf. Appendix Figure 9c). To study how this dynamic nature affects driver decisions, we consider a model with two states, $i \in \{1, 2\}$, where $i = 2$ denotes the surge state. (At a high level, the surge state provides a higher earnings rate to the driver. The precise definition is in Section 2.2.2, after we formulate the driver's earnings rate in each state). The world evolves stochastically between the two states, as a Continuous Time Markov Chain (CTMC). When the world is in state i , the state changes to j according to a fixed exponential clock that ticks at rate $\lambda_i \rightarrow j$, independently of other randomness. When open in state i , the driver receives job requests at rate λ_i with lengths $\tau \sim F_i$, and collects payout according to payment function w_i , which is presumed to have the same properties as w in the single-state model. The state of the world may change while a driver is on trip. Crucially, the driver receives payments according to the state of the world i when the trip begins. We will use $w = \{w_1, w_2\}$ to denote the overall pricing mechanism.

2.2

Driver strategies and earnings

In our model, the driver can decide whether to accept the trip request, with no penalty.⁴

[Next verbatim source excerpt]

[form-feed in source extraction]

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is

$R(w, \sigma), \liminf_{t \rightarrow \infty}$

$R(w, \sigma, t)$

t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

[form-feed in source extraction]

Lemma 1. In the dynamic model, the earnings rate can be decomposed into each state i earnings rate $R_i(w, \sigma_i)$ and fraction of time $\mu_i(\sigma)$ spent in state i :

$R(w, \sigma) = \mu_1(\sigma)R_1(w_1, \sigma_1) + \mu_2(\sigma)R_2(w_2, \sigma_2)$

with probability 1.

$i(\sigma_i)$

As in the single-state model, $R_i(w_i, \sigma_i) = W_i(\sigma_i)$, where

Z

1

$W_i(\sigma_i) =$

$w_i(\tau)dF_i(\tau),$

$F_i(\sigma_i) \tau \in \sigma_i$

$$\frac{1}{T_i(\sigma_i)} = \frac{1}{\lambda_i F_i(\sigma_i) F_i(\sigma_i)}$$

$$\sum_{\tau \in \sigma_i} dF_i(\tau)$$

We prove the result by defining a new renewal process, in which a single reward renewal cycle is: the time between the driver is open in state 1 to the next time the driver is open in state 1 after being open in state 2 at least once. In other words, each renewal cycle is composed of some number (potentially zero) of sub-cycles in which the driver is open in state 1 and then is open in state 1 again after a completed trip; one sub-cycle starting with the driver open in state 1 and ending with being open in state 2 (either after a completed trip or a state transition while open); some number (potentially zero) of sub-cycles in which the driver is open in state 2 and then is open in state 2 again after a completed trip; and finally one sub-cycle starting in state 2 and ending with the driver open in state 1.

Given the number of such renewal reward cycles completed up to time t , the total earnings on trips starting in each state (earnings in each sub-cycle) are independent of each other, and then we use Wald's identity (Wald, 1973) to separate $\mu_i(\sigma)$ and $R_i(\sigma_i)$.

Note that $T_i(\sigma_i)$ is not exactly the expected length of time in a single sub-cycle in a state given σ_i , but rather is proportional to it; the multiplicative constant $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$ cancels out with the same constant in the expected earnings in a single sub-cycle in a state given σ_i . This constant emerges from the primitives: when the driver is open in state i , there are two competing exponential clocks (with rates $\lambda_i F_i(\sigma_i)$ and $\lambda_{i \rightarrow j}$, respectively) that determine whether the driver will accept a request before the world state changes.

[Next verbatim source excerpt]

Lemma 3. Let $T_i(\sigma_i)$ be as defined in Lemma 1. The fraction of time a driver following strategy $\sigma = \{\sigma_1, \sigma_2\}$ spends either open in state i or on a trip started in state i is

$$\frac{\lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)}{\lambda_j F_j(\sigma_j) T_j(\sigma_j) Q_i(\sigma_i) + \lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)}$$

$$\sum_{\tau \in \sigma_i} dF_i(\tau)$$

$$\mu_i(\sigma) =$$

where

$$\tau \in \sigma_i$$

We prove this lemma by finding the expected number of sub-cycles in each state i , i.e., within a larger renewal reward cycle as defined, the expected number of sub-cycles that start with the driver

[form-feed in source extraction]

being open in state i . This expectation is the mean of a geometric random variable parameterized by the probability that the driver will next be open in state j , given the driver is currently open in state i . $Q_i(\sigma_i)$ is proportional to this probability. (As with $T_i(\sigma_i)$, there is a normalizing constant

$\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$); the larger it is, the fewer sub-cycles spent in state i . It has two components: the first is the probability that the state changes before the driver accepts a trip request; the second is the probability that the world state is j when the driver completes a trip. Thus, the numerator in $\mu_i(\sigma)$ is proportional to the length of a sub-cycle in state i , times the fraction of sub-cycles that are started in state i . The larger $Q_j(\sigma_j)$ or $T_i(\sigma_i)$, the more time the driver spends in state i .

Lean-elaborated paper signature

field stateTimeI:

```
{Ω : Type u_1} →
[inst : MeasurableSpace Ω] →
{PQ : MeasureTheory.Measure Ω} →
{μI μJ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} →
{arrivall arrivall switchJI switchJI : ℝ} →
{σI σJ : GN21DriverSurgePricing.TripPolicy} →
GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μI μJ arrivall arrivall switchJI switchJI σI σJ →
ℕ → Ω → ℝ
```

field stateTimeJ:

```
{Ω : Type u_1} →
[inst : MeasurableSpace Ω] →
{PQ : MeasureTheory.Measure Ω} →
{μI μJ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} →
{arrivall arrivall switchJI switchJI : ℝ} →
{σI σJ : GN21DriverSurgePricing.TripPolicy} →
GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μI μJ arrivall arrivall switchJI switchJI σI σJ →
ℕ → Ω → ℝ
```

field arrivall_pos:

```
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μI μJ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrivall arrivall switchJI switchJI : ℝ}
{σI σJ : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μI μJ arrivall arrivall switchJI switchJI σI σJ),
0 < arrivall
```

field arrivall_pos:

```
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μI μJ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrivall arrivall switchJI switchJI : ℝ}
{σI σJ : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μI μJ arrivall arrivall switchJI switchJI σI σJ),
```

```

0 < arrivalJ

field switchI_pos:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
0 < switchI

field switchJI_pos:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
0 < switchJI

field σ_measurable:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
MeasurableSet σ

field σj_measurable:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
MeasurableSet σj

field σ_subset:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
σ ⊆ GN21DriverSurgePricing.acceptAllPolicy

field σj_subset:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
σj ⊆ GN21DriverSurgePricing.acceptAllPolicy

field massI_pos:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
0 < GN21DriverSurgePricing.singleStateTripMass μ σ

field massJ_pos:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
0 < GN21DriverSurgePricing.singleStateTripMass μj σj

field stateTimeI_integrable:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
MeasureTheory.Integrable (self.stateTimeI 0) PQ

field stateTimeJ_integrable:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
MeasureTheory.Integrable (self.stateTimeJ 0) PQ

field stateTimeI_independent:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
Pairwise (Function.onFun (fun (x1 x2 : Ω → ℝ) => ProbabilityTheory.IndepFun x1 x2 PQ) self.stateTimeI)

field stateTimeJ_independent:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj),
Pairwise (Function.onFun (fun (x1 x2 : Ω → ℝ) => ProbabilityTheory.IndepFun x1 x2 PQ) self.stateTimeJ)

field stateTimeI_identDistrib:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μ μj arrival arrivalJ switchI switchJI σ σj)
(n : ℕ), ProbabilityTheory.IdentDistrib (self.stateTimeI n) (self.stateTimeI 0) PQ PQ

field stateTimeJ_identDistrib:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μ μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchI switchJI : ℝ}
{σ σj : GN21DriverSurgePricing.TripPolicy}

```

```
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PΩ μl μj arrivall arrivalJ switchlJ switchJl σl σj)
(n : ℕ), ProbabilityTheory.IdentDistrib (self.stateTimeJ n) (self.stateTimeJ 0) PΩ PΩ
```

field stateTimeI_mean_eq:

```
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PΩ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrivall arrivalJ switchlJ switchJl : ℝ}
{σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PΩ μl μj arrivall arrivalJ switchlJ switchJl σl σj),
f (ω : Ω), self.stateTimeI 0 ω ∂PΩ =
  GN21DriverSurgePricing.gn21ExpectedStateTimeInRenewalCycle arrivall
  (GN21DriverSurgePricing.singleStateTripMass μl σl) switchlJ
  (GN21DriverSurgePricing.gn21StateCycleTime μl arrivall σl)
  (GN21DriverSurgePricing.gn21ExitWeightIntegral μl arrivall switchlJ switchJl σl)
```

field stateTimeJ_mean_eq:

```
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PΩ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrivall arrivalJ switchlJ switchJl : ℝ}
{σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PΩ μl μj arrivall arrivalJ switchlJ switchJl σl σj),
f (ω : Ω), self.stateTimeJ 0 ω ∂PΩ =
  GN21DriverSurgePricing.gn21ExpectedStateTimeInRenewalCycle arrivall
  (GN21DriverSurgePricing.singleStateTripMass μj σj) switchJl
  (GN21DriverSurgePricing.gn21StateCycleTime μj arrivall σj)
  (GN21DriverSurgePricing.gn21ExitWeightIntegral μj arrivall switchJl switchlJ σj)
```

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21ExitWeightIntegral](#)
- [GN21DriverSurgePricing.gn21ExpectedStateTimeInRenewalCycle](#)
- [GN21DriverSurgePricing.gn21StateCycleTime](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Lemma 3's renewal argument requires feasible positive state policies, IID integrable state-time observations, and the exact mean identities. The structure records those facts and its raw-calendar constructor derives them from the source process.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GN21DriverSurgePricing.GN21DynamicIIDCycleModel

Paper-local declaration:

GN21DriverSurgePricing.GN21DynamicIIDCycleModel

Source locator: cited publication, lines 217-252

Verbatim paper-source connection

We start with a model where there is a single world state, i.e., all model components are constant over time. Time is continuous and indexed by t . At each time t , the driver is either open, or busy. While open, the driver receives job (trip) requests from riders according to a Poisson process at rate λ , i.e., the time between requests is exponential with mean λ^{-1} . Job lengths, denoted by τ , are drawn independently and identically from a continuous distribution F . If the driver accepts a job request of length τ at time t (as discussed below), they receive a payout of $w(\tau)$ at time $t + \tau$, at which time they become open again. Otherwise, the driver remains open. Except where specified, the only assumptions on w are that it is continuous and asymptotically (sub)-linear: $\exists c : \liminf_{\tau \rightarrow \infty} w(\tau) \leq c$, which ensures that the driver reward is also bounded.³

Dynamic model with surge pricing

A model with fixed pricing and arrival rates of jobs is not a realistic representation of ride-hailing platforms. In particular, rider demand (both in intensity and in distribution) may vary substantially over time, even within a day (cf. Appendix Figure 9c). To study how this dynamic nature affects driver decisions, we consider a model with two states, $i \in \{1, 2\}$, where $i = 2$ denotes the surge state. (At a high level, the surge state provides a higher earnings rate to the driver. The precise definition is in Section 2.2.2, after we formulate the driver's earnings rate in each state). The world evolves stochastically between the two states, as a Continuous Time Markov Chain (CTMC). When the world is in state i , the state changes to j according to a fixed exponential clock that ticks at rate $\lambda_i \rightarrow j$, independently of other randomness. When open in state i , the driver receives job requests at rate λ_i with lengths $\tau \sim F_i$, and collects payout according to payment function w_i , which is presumed to have the same properties as w in the single-state model. The state of the world may change while a driver is on trip. Crucially, the driver receives payments according to the state of the world i when the trip begins. We will use $w = \{w_1, w_2\}$ to denote the overall pricing mechanism.

2.2

Driver strategies and earnings

In our model, the driver can decide whether to accept the trip request, with no penalty.⁴

[Next verbatim source excerpt]

[form-feed in source extraction]

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is $R(w, \sigma), \liminf_{t \rightarrow \infty}$

$R(w, \sigma, t)$
:
 t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

[form-feed in source extraction]

Lemma 1. In the dynamic model, the earnings rate can be decomposed into each state i earnings rate $R_i(w_i, \sigma_i)$ and fraction of time $\mu_i(\sigma)$ spent in state i :
 $R(w, \sigma) = \mu_1(\sigma)R_1(w_1, \sigma_1) + \mu_2(\sigma)R_2(w_2, \sigma_2)$

with probability 1.

$i(\sigma_i)$
As in the single-state model, $R_i(w_i, \sigma_i) = W$
 $T_i(\sigma_i)$, where

Z
 1
 $W_i(\sigma_i) =$
 $w_i(\tau)dF_i(\tau),$
 $F_i(\sigma_i) \tau \in \sigma_i$

1
 1

$$T_i(\sigma_i) = \sum_{\lambda_i} F_i(\sigma_i) F_i(\sigma_i)$$

$$\sum_{\tau} dF_i(\tau)$$

$$\tau \in \sigma_i$$

We prove the result by defining a new renewal process, in which a single reward renewal cycle is: the time between the driver is open in state 1 to the next time the driver is open in state 1 after being open in state 2 at least once. In other words, each renewal cycle is composed of some number (potentially zero) of sub-cycles in which the driver is open in state 1 and then is open in state 1 again after a completed trip; one sub-cycle starting with the driver open in state 1 and ending with being open in state 2 (either after a completed trip or a state transition while open); some number (potentially zero) of sub-cycles in which the driver is open in state 2 and then is open in state 2 again after a completed trip; and finally one sub-cycle starting in state 2 and ending with the driver open in state 1.

Given the number of such renewal reward cycles completed up to time t , the total earnings on trips starting in each state (earnings in each sub-cycle) are independent of each other, and then we use Wald's identity (Wald, 1973) to separate $\mu_i(\sigma)$ and $R_i(\sigma_i)$.

Note that $T_i(\sigma_i)$ is not exactly the expected length of time in a single sub-cycle in a state given σ_i , but rather is proportional to it; the multiplicative constant $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$ cancels out with the same constant in the expected earnings in a single sub-cycle in a state given σ_i . This constant emerges from the primitives: when the driver is open in state i , there are two competing exponential clocks (with rates $\lambda_i F_i(\sigma_i)$ and $\lambda_{i \rightarrow j}$, respectively) that determine whether the driver will accept a request before the world state changes.

[Next verbatim source excerpt]

Lemma 3. Let $T_i(\sigma_i)$ be as defined in Lemma 1. The fraction of time a driver following strategy $\sigma = \{\sigma_1, \sigma_2\}$ spends either open in state i or on a trip started in state i is

$$\lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)$$

$$\lambda_j F_j(\sigma_j) T_j(\sigma_j) Q_i(\sigma_i) + \lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)$$

$$\sum_{\tau} Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$$

$$q_{i \rightarrow j}(\tau) dF_i(\tau)$$

$$\mu_i(\sigma) =$$

where

$$\tau \in \sigma_i$$

We prove this lemma by finding the expected number of sub-cycles in each state i , i.e., within a larger renewal reward cycle as defined, the expected number of sub-cycles that start with the driver

[form-feed in source extraction]

being open in state i . This expectation is the mean of a geometric random variable parameterized by the probability that the driver will next be open in state j , given the driver is currently open in state i . $Q_i(\sigma_i)$ is proportional to this probability. (As with $T_i(\sigma_i)$, there is a normalizing constant

$$\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$$

); the larger it is, the fewer sub-cycles spent in state i . It has two components: the first is the probability that the state changes before the driver accepts a trip request; the second is the probability that the world state is j when the driver completes a trip. Thus, the numerator in $\mu_i(\sigma)$ is proportional to the length of a sub-cycle in state i , times the fraction of sub-cycles that are started in state i . The larger $Q_j(\sigma_j)$ or $T_i(\sigma_i)$, the more time the driver spends in state i .

Lean-elaborated paper signature

field toGN21TimeFractionIIDCycleModel:

```
{Ω : Type u_1} →
[inst : MeasurableSpace Ω] →
{PQ : MeasureTheory.Measure Ω} →
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} →
{arrivall arrivall switchl switchl : ℝ} →
{wl wj : GN21DriverSurgePricing.PricingFunction} →
{σl σj : GN21DriverSurgePricing.TripPolicy} →
GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrivall arrivall switchl switchl wl wj σl σj →
GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel PQ μl μj arrivall arrivall switchl switchl σl σj
```

field stateEarningl:

```
{Ω : Type u_1} →
[inst : MeasurableSpace Ω] →
{PQ : MeasureTheory.Measure Ω} →
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} →
{arrivall arrivall switchl switchl : ℝ} →
{wl wj : GN21DriverSurgePricing.PricingFunction} →
{σl σj : GN21DriverSurgePricing.TripPolicy} →
GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrivall arrivall switchl switchl wl wj σl σj →
ℕ → Ω → ℝ
```

field stateEarningj:

```
{Ω : Type u_1} →
[inst : MeasurableSpace Ω] →
{PQ : MeasureTheory.Measure Ω} →
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} →
{arrivall arrivall switchl switchl : ℝ} →
{wl wj : GN21DriverSurgePricing.PricingFunction} →
{σl σj : GN21DriverSurgePricing.TripPolicy} →
GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrivall arrivall switchl switchl wl wj σl σj →
ℕ → Ω → ℝ
```

field stateEarningl_integrable:

```
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
```

```

{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrival switch switchJl : ℝ}
{wl wj : GN21DriverSurgePricing.PricingFunction} {σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrival arrival switch switchJl wl wj σl σj),
MeasureTheory.Integrable (self.stateEarningl 0) PQ

field stateEarningJ_integrable:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrival switch switchJl : ℝ}
{wl wj : GN21DriverSurgePricing.PricingFunction} {σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrival arrival switch switchJl wl wj σl σj),
MeasureTheory.Integrable (self.stateEarningJ 0) PQ

field stateEarningl_independent:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrival switch switchJl : ℝ}
{wl wj : GN21DriverSurgePricing.PricingFunction} {σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrival arrival switch switchJl wl wj σl σj),
Pairwise (Function.onFun (fun (x1 x2 : Ω → ℝ) => ProbabilityTheory.IndepFun x1 x2 PQ) self.stateEarningl)

field stateEarningJ_independent:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrival switch switchJl : ℝ}
{wl wj : GN21DriverSurgePricing.PricingFunction} {σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrival arrival switch switchJl wl wj σl σj),
Pairwise (Function.onFun (fun (x1 x2 : Ω → ℝ) => ProbabilityTheory.IndepFun x1 x2 PQ) self.stateEarningJ)

field stateEarningl_identDistrib:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrival switch switchJl : ℝ}
{wl wj : GN21DriverSurgePricing.PricingFunction} {σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrival arrival switch switchJl wl wj σl σj)
(n : ℕ), ProbabilityTheory.IdentDistrib (self.stateEarningl n) (self.stateEarningl 0) PQ PQ

field stateEarningJ_identDistrib:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrival switch switchJl : ℝ}
{wl wj : GN21DriverSurgePricing.PricingFunction} {σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrival arrival switch switchJl wl wj σl σj)
(n : ℕ), ProbabilityTheory.IdentDistrib (self.stateEarningJ n) (self.stateEarningJ 0) PQ PQ

field stateEarningl_mean_eq:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrival switch switchJl : ℝ}
{wl wj : GN21DriverSurgePricing.PricingFunction} {σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrival arrival switch switchJl wl wj σl σj),
f (ω : Ω), self.stateEarningl 0 ω ∂PQ =
  GN21DriverSurgePricing.gn21ExpectedStateEarningInRenewalCycle arrival
  (GN21DriverSurgePricing.singleStateTripMass μl σl) switchJl (GN21DriverSurgePricing.gn21StateMeanEarning μl wl σl)
  (GN21DriverSurgePricing.gn21ExitWeightIntegral μl arrival switch switchJl σl)

field stateEarningJ_mean_eq:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] {PQ : MeasureTheory.Measure Ω}
{μl μj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrival switch switchJl : ℝ}
{wl wj : GN21DriverSurgePricing.PricingFunction} {σl σj : GN21DriverSurgePricing.TripPolicy}
(self : GN21DriverSurgePricing.GN21DynamicIIDCycleModel PQ μl μj arrival arrival switch switchJl wl wj σl σj),
f (ω : Ω), self.stateEarningJ 0 ω ∂PQ =
  GN21DriverSurgePricing.gn21ExpectedStateEarningInRenewalCycle arrival
  (GN21DriverSurgePricing.singleStateTripMass μj σj) switchJl (GN21DriverSurgePricing.gn21StateMeanEarning μj wj σj)
  (GN21DriverSurgePricing.gn21ExitWeightIntegral μj arrival switch switchJl σj)

```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21ExitWeightIntegral](#)
- [GN21DriverSurgePricing.gn21ExpectedStateEarningInRenewalCycle](#)
- [GN21DriverSurgePricing.gn21StateMeanEarning](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Lemma 1's renewal cycles require state times, earnings, IID laws, integrability, and exact means. This structure records those ingredients, and the current raw-calendar constructor derives them from the source seed rather than assuming them.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint

Paper-local declaration:

GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint

Source locator: cited publication, lines 3652-3680

Verbatim paper-source connection

These lemmas primarily involve properties of derivatives of the reward function $R(w, \sigma)$ and its components in the dynamic model, as a function of the pricing.

55

[form-feed in source extraction]

D.3.1

Notation and assumptions

Recall in the dynamic model that we constrain σ_i to be measurable, open, subsets of the R^+ . Then, σ_i can be written as a countable union of disjoint subsets of R^+ , i.e. $\sigma_i = \bigcup_{k=0}^{\infty} (u_k, u_{k+1}]$. We further

assume that $u_{k+1} = u_k + m$, for any k, m ; we can do so without loss of generality by making a measure 0 change to σ_i , by adding $u_k = m$ to σ_i .

Suppose u is an upper-endpoint of σ_i , i.e. $\exists k$ such that $u = u_k$. Then, we use $\partial_u H(\sigma_i)$ to denote

the derivative of the set function H with respect to u at σ_i . Similarly, $\partial^- H(\sigma_i)$ is the derivative of H at σ_i with respect to a lower-endpoint of σ_i .

We derive $\partial_u R(w, \{\sigma_1, \sigma_2\})$, $\partial^- R(w, \{\sigma_1, \sigma_2\})$. We will make it clear in each instance whether ∂_u

refers to the derivative at an endpoint of σ_1 or σ_2 . For all the derivatives in this subsection ∂_u

with respect to an upper endpoint in σ_i , and ∂^- refers to a derivative at a lower endpoint of σ_i .

[Next verbatim source excerpt]

Lemma 6. Let $R(w, \sigma)$ be as defined in Lemma 1. Then, ∂_u

[U+0013 control character]

[U+0012 control character]

[U+0013 control character] [U+0012 control character]

$q_{i \rightarrow j}(u)$

Q_j

Q_i

$w_i(u, Q_i, Q_j)$

$r(u, i, w, \sigma)$,

$+$

$R_j +$

R_i

$\Delta_j +$

u

u

T_i

T_j

T_i

T_j

Proof. Proof.

$\mu_i(\{\sigma_j, \sigma_2\}) =$

$Q_j T_i$

$Q_j T_i + Q_i T_j$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

[U+0015 control character]

[U+0014 control character]

1

$[Q_2 W_1 + Q_1 W_2]$

$=$

$Q_2 T_1 + Q_1 T_2$

W_i

$R_i(\sigma_i) =$

T_i

[U+0014 control character]

[U+0015 control character]

Z

∂

∂

$Q_i =$

$\lambda_{i \rightarrow j} + \lambda_i$

$q_{i \rightarrow j}(\tau) dF_i(\tau) = \lambda_i q_{i \rightarrow j}(u) f_i(u)$

∂_u

∂_u

$\tau \in \sigma_i$

Lean-elaborated paper signature

field point:
 $\{\sigma : \text{GN21DriverSurgePricing.TripPolicy}\} \rightarrow$
 $\{\text{upper} : \text{GN21DriverSurgePricing.TripLength}\} \rightarrow$
 $\text{GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint } \sigma \text{ upper} \rightarrow \text{GN21DriverSurgePricing.TripLength}$

field point_mem:
 $\forall \{\sigma : \text{GN21DriverSurgePricing.TripPolicy}\} \{\text{upper} : \text{GN21DriverSurgePricing.TripLength}\}$
 $\{\text{self} : \text{GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint } \sigma \text{ upper}\}, \text{self.point} \in \sigma$

field lower:
 $\{\sigma : \text{GN21DriverSurgePricing.TripPolicy}\} \rightarrow$
 $\{\text{upper} : \text{GN21DriverSurgePricing.TripLength}\} \rightarrow$
 $\text{GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint } \sigma \text{ upper} \rightarrow \text{GN21DriverSurgePricing.TripLength}$

field component:
 $\forall \{\sigma : \text{GN21DriverSurgePricing.TripPolicy}\} \{\text{upper} : \text{GN21DriverSurgePricing.TripLength}\}$
 $\{\text{self} : \text{GN21DriverSurgePricing.GN21SourceFiniteUpperEndpoint } \sigma \text{ upper}\},$
 $\text{connectedComponentIn } \sigma \text{ self.point} = \text{Set.lou self.lower upper}$

Direct retained dependencies

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Theorem 4 varies a finite upper endpoint of an interval component of an open policy. The structure records precisely that endpoint and component data.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GN21DriverSurgePricing.Lemma5DerivativeShape

Paper-local declaration:

GN21DriverSurgePricing.Lemma5DerivativeShape

Source locator: cited publication, lines 3325-3342

Verbatim paper-source connection

Lemma 5 shows how the structure of an optimizer $\sigma^* = \text{uk}(\cdot^k, \text{uk})$ of a set function $\hat{R}(\sigma)$ depends on the derivative of the set function with respect to the endpoints of the sets that make up the policy, $\partial \hat{R}(\sigma)$. The main idea is that as long as the derivative can be shown to be positive for some u that is an endpoint of σ_i , that policy can be locally modified to accept more trips and increase the overall reward function. We work with a function $r(u, \sigma)$ that has the same sign as the derivative $\partial \hat{R}(\sigma)$. Given $r(u, \sigma)$, we analyze the sign of $\partial \hat{R}(\sigma)$ as it depends on the structure of σ , and in turn can characterize the structure of the optimal σ^* . In particular, we will show how properties of $r(u, \sigma)$ – whether for a fixed σ it is always positive, strictly increasing, strictly decreasing, strictly quasi-convex, or strictly quasi-concave in u – lead to different optimal σ^* . The rest of the appendix section applies Lemma 5 to our context, by showing how different pricing functions induce different properties of $r(u, \sigma)$ and thus different optimal policies for each state σ_i^* .

[Next verbatim source excerpt]

These lemmas primarily involve properties of derivatives of the reward function $R(w, \sigma)$ and its components in the dynamic model, as a function of the pricing.
55

[form-feed in source extraction]
D.3.1

Notation and assumptions

Recall in the dynamic model that we constrain σ_i to be measurable, open, subsets of the R^+ . Then, σ_i can be written as a countable union of disjoint subsets of R^+ , i.e. $\sigma_i = \bigcup_{k=0}^{\infty} (k, \text{uk})$. We further assume that $\text{uk} \leq m$, for any k, m ; we can do so without loss of generality by making a measure 0 change to σ_i , by adding $\text{uk} = m$ to σ_i . Suppose u is an upper-endpoint of σ_i , i.e. $\exists k$ such that $u = \text{uk}$. Then, we use $\partial \hat{R}(\sigma_i)$ to denote the derivative of the set function H with respect to u at σ_i . Similarly, $\partial^- \hat{R}(\sigma_i)$ is the derivative of H at σ_i with respect to a lower-endpoint of σ_i . We derive $\partial \hat{R}(w, \{\sigma_1, \sigma_2\})$, $\partial^- \hat{R}(w, \{\sigma_1, \sigma_2\})$. We will make it clear in each instance whether ∂ refers to the derivative or ∂^- is an endpoint of σ_1 or σ_2 . For all the derivatives in this subsection ∂ with respect to an upper endpoint in σ_i , and ∂^- refers to a derivative at a lower endpoint of σ_i . Note that $\partial \hat{R}(w, \{\sigma_1, \sigma_2\}) = -\partial^- \hat{R}(w, \{\sigma_1, \sigma_2\})$. Furthermore:

- We use σ in the function argument when the function depends on policies in both states, and σ_i when it only depends on the policy in state i .
- We use $\propto$ to denote that “two functions of u have the same sign except where $f(u) = 0$ ”, rather than proportional to.
- All policy equalities are up to measure 0.

Lean-elaborated paper signature

type:
Type

constructor GN21DriverSurgePricing.Lemma5DerivativeShape.positive:
GN21DriverSurgePricing.Lemma5DerivativeShape

constructor GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyIncreasing:
GN21DriverSurgePricing.Lemma5DerivativeShape

constructor GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyDecreasing:
GN21DriverSurgePricing.Lemma5DerivativeShape

constructor GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex:
GN21DriverSurgePricing.Lemma5DerivativeShape

constructor GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave:
GN21DriverSurgePricing.Lemma5DerivativeShape

Recorded source-to-declaration screening Verdict: matches

Reason: Lemma 5 has exactly the five response-shape cases positive, strictly increasing, strictly decreasing, strictly quasi-convex, and strictly quasi-concave; the inductive is exactly that list.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GN21DriverSurgePricing.Lemma6EndpointDerivativeData

Paper-local declaration:

GN21DriverSurgePricing.Lemma6EndpointDerivativeData

Source locator: cited publication, lines 3652-3680

Verbatim paper-source connection

These lemmas primarily involve properties of derivatives of the reward function $R(w, \sigma)$ and its components in the dynamic model, as a function of the pricing.

55

[form-feed in source extraction]

D.3.1

Notation and assumptions

Recall in the dynamic model that we constrain σ_i to be measurable, open, subsets of the R^+ . Then, σ_i can be written as a countable union of disjoint subsets of R^+ , i.e. $\sigma_i = \bigcup_{k=0}^{\infty} (u_k, u_{k+1}]$. We further

assume that $u_{k+1} = u_k + \delta$, for any k, m ; we can do so without loss of generality by making a measure 0 change to σ_i , by adding $u_k = \delta$ to σ_i .

Suppose u is an upper-endpoint of σ_i , i.e. $\exists k$ such that $u = u_k$. Then, we use $\partial_u H(\sigma_i)$ to denote

the derivative of the set function H with respect to u at σ_i . Similarly, $\partial^- H(\sigma_i)$ is the derivative of H at σ_i with respect to a lower-endpoint of σ_i .

We derive $\partial_u R(w, \{\sigma_1, \sigma_2\})$, $\partial^- R(w, \{\sigma_1, \sigma_2\})$. We will make it clear in each instance whether ∂_u

refers to the derivative at an endpoint of σ_1 or σ_2 . For all the derivatives in this subsection ∂_u

with respect to an upper endpoint in σ_i , and ∂^- refers to a derivative at a lower endpoint of σ_i .

[Next verbatim source excerpt]

Lemma 6. Let $R(w, \sigma)$ be as defined in Lemma 1. Then, ∂_u

[U+0013 control character]

[U+0012 control character]

[U+0013 control character] [U+0012 control character]

$q_i \rightarrow j(u)$

Q_j

Q_i

$w_i(u, Q_i, Q_j)$

$r(u, i, w, \sigma)$,

$+$

$R_j +$

R_i

$\Delta_j +$

u

u

T_i

T_j

T_i

T_j

Proof. Proof.

$\mu_i(\{\sigma_j, \sigma_2\}) =$

$Q_j T_i$

$Q_j T_i + Q_i T_j$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

[U+0015 control character]

[U+0014 control character]

1

$[Q_2 W_1 + Q_1 W_2]$

$=$

$Q_2 T_1 + Q_1 T_2$

W_i

$R_i(\sigma_i) =$

T_i

[U+0014 control character]

[U+0015 control character]

Z

∂

∂

$Q_i =$

$\lambda_i \rightarrow j + \lambda_i$

$q_i \rightarrow j(\tau) dF_i(\tau) = \lambda_i q_i \rightarrow j(u) f_i(u)$

∂_u

∂_u

$\tau \in \sigma_i$

Lean-elaborated paper signature

field rewardAlongEndpoint:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R} \rightarrow \mathbb{R}$

field derivativeValue:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field q:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field u:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field wi:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field Qi:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field Qj:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field Ti:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field Tj:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field Wi:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field Wj:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field scale:
GN21DriverSurgePricing.Lemma6EndpointDerivativeData $\rightarrow \mathbb{R}$

field scale_pos:
 $\forall$ (self : GN21DriverSurgePricing.Lemma6EndpointDerivativeData), $0 < \text{self.scale}$

field has_derivative:
 $\forall$ (self : GN21DriverSurgePricing.Lemma6EndpointDerivativeData),
HasDerivAt self.rewardAlongEndpoint self.derivativeValue self.u

field derivative_eq_scaled_kernel:
 $\forall$ (self : GN21DriverSurgePricing.Lemma6EndpointDerivativeData),
self.derivativeValue =
self.scale *
GN21DriverSurgePricing.gn21DerivativeSignKernel self.q self.u self.wi self.Qi self.Qj self.Ti self.Tj self.Wi
self.Wj

Direct retained dependencies

- [GN21DriverSurgePricing.gn21DerivativeSignKernel](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Lemma 6 supplies an endpoint derivative and identifies its strict sign with the displayed response. The structure's derivative witness and positive-multiple kernel equality are exactly that sign-preserving source input.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GN21DriverSurgePricing.PaperInterface.review_definition_incentive_compatibleSpec

Paper-local declaration:

GN21DriverSurgePricing.PaperInterface.review_definition_incentive_compatibleSpec

Source locator: cited publication, lines 264-287

Verbatim paper-source connection

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is $R(w, \sigma), \liminf_{t \rightarrow \infty}$

$R(w, \sigma, t)$
.
 t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w . In other words, payment function w is incentive compatible if an earnings-maximizing driver (who knows all the primitives, w , and the trip length τ at request time) accepts every trip request.

Lean-expanded paper semantic target

type:

$(\text{GN21DriverSurgePricing.SingleStateReward} \rightarrow \text{Prop}) \times (\text{GN21DriverSurgePricing.DynamicReward} \rightarrow \text{Prop})$

value:

```
(fun (R : GN21DriverSurgePricing.SingleStateReward) =>
  ∀ σ ⊆ GN21DriverSurgePricing.acceptAllPolicy, MeasurableSet σ → R σ ≤ R GN21DriverSurgePricing.acceptAllPolicy,
  fun (R : GN21DriverSurgePricing.DynamicReward) =>
    GN21DriverSurgePricing.dynamicFeasibleOpenPolicy GN21DriverSurgePricing.acceptAllDynamicPolicy ∧
    ∀ (σ : Fin 2 → GN21DriverSurgePricing.TripPolicy),
      GN21DriverSurgePricing.dynamicFeasibleOpenPolicy σ → R σ ≤ R GN21DriverSurgePricing.acceptAllDynamicPolicy)
```

Direct retained dependencies

- [GN21DriverSurgePricing.DynamicReward](#)
- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllDynamicPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.dynamicFeasibleOpenPolicy](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The source defines incentive compatibility by accept-all maximizing reward over measurable one-state and open measurable two-state policies. The paired predicate target reproduces exactly those two comparisons.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GN21DriverSurgePricing.PaperInterface.review_definition_surge_stateSpec

Paper-local declaration:

GN21DriverSurgePricing.PaperInterface.review_definition_surge_stateSpec

Source locator: cited publication, lines 377-381

Verbatim paper-source connection

Finally, we can now precisely define what it means for $i = 2$ to be the surge state: it has a higher potential earning rate than state 1. There exists some policy σ_2 such that $R_2(w_2, \sigma_2) > R_1(w_1, \sigma_1)$, for all $\sigma_1 \in R^+$. In other words, suppose that instead we were in the single-state setting, where the primitives were set as either (λ_1, F_1, w_1) or (λ_2, F_2, w_2) . Then the latter set of primitives would yield a higher maximum earnings.⁵

Lean-expanded paper semantic target

type:

```
(Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) →  
(Fin 2 → ℝ) → (Fin 2 → GN21DriverSurgePricing.PricingFunction) → Prop
```

value:

```
fun (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ)  
  (w : Fin 2 → GN21DriverSurgePricing.PricingFunction) =>  
  ∃ σ2 ⊆ GN21DriverSurgePricing.acceptAllPolicy,  
    IsOpen σ2 ∧  
    ∀ σ1 ⊆ GN21DriverSurgePricing.acceptAllPolicy,  
      IsOpen σ1 →  
        GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 0) (arrival 0) (w 0) σ1 <  
        GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 1) (arrival 1) (w 1) σ2
```

Direct retained dependencies

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21MeasuredStateRewardRate](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The source defines state 2 as surge when a feasible state-2 policy strictly dominates every feasible state-1 policy; the target is exactly that existential-universal comparison.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization

Paper-local declaration:

GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization

Source locator: cited publication, lines 3325-3342

Verbatim paper-source connection

Lemma 5 shows how the structure of an optimizer $\sigma^* = \text{uk}(\cdot^k, \text{uk})$ of a set function $\hat{R}(\sigma)$ depends on the derivative of the set function with respect to the endpoints of the sets that make up the policy, $\partial_u \hat{R}(\sigma)$. The main idea is that as long as the derivative can be shown to be positive for some u that is an endpoint of σ_i , that policy can be locally modified to accept more trips and increase the overall reward function. We work with a function $r(u, \sigma)$ that has the same sign as the derivative $\partial_u \hat{R}(\sigma)$. Given $r(u, \sigma)$, we analyze the sign of $\partial_u \hat{R}(\sigma)$ as it depends on the structure of σ , and in turn can characterize the structure of the optimal σ^* . In particular, we will show how properties of $r(u, \sigma)$ – whether for a fixed σ it is always positive, strictly increasing, strictly decreasing, strictly quasi-convex, or strictly quasi-concave in u – lead to different optimal σ^* . The rest of the appendix section applies Lemma 5 to our context, by showing how different pricing functions induce different properties of $r(u, \sigma)$ and thus different optimal policies for each state σ^* .

[Next verbatim source excerpt]

These lemmas primarily involve properties of derivatives of the reward function $R(w, \sigma)$ and its components in the dynamic model, as a function of the pricing.
55

[form-feed in source extraction]

D.3.1

Notation and assumptions

Recall in the dynamic model that we constrain σ_i to be measurable, open, subsets of the $\mathbb{R}^+$. Then, σ_i can be written as a countable union of disjoint subsets of $\mathbb{R}^+$, i.e. $\sigma_i = \bigcup_{k=0}^{\infty} (\cdot^k, \text{uk})$. We further assume that $\text{uk} \leq \cdot^m$, for any k, m ; we can do so without loss of generality by making a measure 0 change to σ_i , by adding $\text{uk} = \cdot^m$ to σ_i . Suppose u is an upper-endpoint of σ_i , i.e. $\exists k$ such that $u = \text{uk}$. Then, we use $\partial_u H(\sigma_i)$ to denote the derivative of the set function H with respect to u at σ_i . Similarly, $\partial^- H(\sigma_i)$ is the derivative of H at σ_i with respect to a lower-endpoint of σ_i . We derive $\partial_u R(w, \{\sigma_1, \sigma_2\})$, $\partial^- R(w, \{\sigma_1, \sigma_2\})$. We will make it clear in each instance whether u refers to the derivative or ∂^- is an endpoint of σ_1 or σ_2 . For all the derivatives in this subsection ∂_u with respect to an upper endpoint in σ_i , and ∂^- refers to a derivative at a lower endpoint of σ_i . Note that $R(w, \{\sigma_1, \sigma_2\}) = -R(w, \{\sigma_1, \sigma_2\})$. Furthermore:

- We use σ in the function argument when the function depends on policies in both states, and σ_i when it only depends on the policy in state i .
- We use $\propto$ to denote that “two functions of u have the same sign except where $f(u) = 0$ ”, rather than proportional to.
- All policy equalities are up to measure 0.

Lean-elaborated paper signature

field upper_derivative:

```
∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
  {response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
  {shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
  GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
  ∀ (extra : ℕ) (endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra) (i : Fin extra) {value : ℝ},
  endpoints (GN21DriverSurgePricing.gn21Lemma5GapUpperIndex i) = ENNReal.ofReal value →
  ∃ (derivativeValue : ℝ),
    HasDerivAt
      (fun (x : ℝ) =>
        Rhat
          (GN21DriverSurgePricing.gn21EndpointVectorPolicy
            (GN21DriverSurgePricing.gn21UpdateEndpoint endpoints
              (GN21DriverSurgePricing.gn21Lemma5GapUpperIndex i) x)))
        derivativeValue value ∧
    GN21DriverSurgePricing.sameStrictSign derivativeValue
```

```

(response (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints) value)

field lower_derivative:
  ∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
  {response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
  {shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
  GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
  ∀ (extra : ℕ) (endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra) (slot : Fin (extra + 1))
  {value : ℝ},
  endpoints (GN21DriverSurgePricing.gn21LowerEndpointIndex slot) = ENNReal.ofReal value →
  ∃ (derivativeValue : ℝ),
  HasDerivAt
    (fun (x : ℝ) =>
      Rhat
        (GN21DriverSurgePricing.gn21EndpointVectorPolicy
          (GN21DriverSurgePricing.gn21UpdateEndpoint endpoints
            (GN21DriverSurgePricing.gn21LowerEndpointIndex slot) x)))
    derivativeValue value ∧
  GN21DriverSurgePricing.sameStrictSign derivativeValue
    (-response (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints) value)

field upper_right_derivative:
  ∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
  {response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
  {shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
  GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
  ∀ (extra : ℕ) (endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra) (i : Fin extra),
  endpoints (GN21DriverSurgePricing.gn21Lemma5GapUpperIndex i) = 0 →
  ∃ (derivativeValue : ℝ),
  HasDerivWithinAt
    (fun (x : ℝ) =>
      Rhat
        (GN21DriverSurgePricing.gn21EndpointVectorPolicy
          (GN21DriverSurgePricing.gn21UpdateEndpoint endpoints
            (GN21DriverSurgePricing.gn21Lemma5GapUpperIndex i) x)))
    derivativeValue (Set.lci 0) 0 ∧
  GN21DriverSurgePricing.sameStrictSign derivativeValue
    (response (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints) 0)

field lower_right_derivative:
  ∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
  {response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
  {shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
  GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
  ∀ (extra : ℕ) (endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra) (slot : Fin (extra + 1)),
  endpoints (GN21DriverSurgePricing.gn21LowerEndpointIndex slot) = 0 →
  ∃ (derivativeValue : ℝ),
  HasDerivWithinAt
    (fun (x : ℝ) =>
      Rhat
        (GN21DriverSurgePricing.gn21EndpointVectorPolicy
          (GN21DriverSurgePricing.gn21UpdateEndpoint endpoints
            (GN21DriverSurgePricing.gn21LowerEndpointIndex slot) x)))
    derivativeValue (Set.lci 0) 0 ∧
  GN21DriverSurgePricing.sameStrictSign derivativeValue
    (-response (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints) 0)

field positive_path_continuous:
  ∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
  {response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
  {shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
  GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
  ∀ (extra : ℕ),
  ∃
    endpoints ∈
      GN21DriverSurgePricing.gn21Lemma5EndpointDomain GN21DriverSurgePricing.Lemma5DerivativeShape.positive extra,
  ∀ (i : Fin extra) (upper : ℝ),
  0 < upper →
  ContinuousOn
    (fun (x : ℝ) =>
      Rhat
        (GN21DriverSurgePricing.gn21EndpointVectorPolicy
          (GN21DriverSurgePricing.gn21UpdateEndpoint endpoints
            (GN21DriverSurgePricing.gn21Lemma5GapUpperIndex i) x)))
    (Set.lcc 0 upper)

field positive_path_derivative:
  ∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
  {response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
  {shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
  GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
  ∀ (extra : ℕ) (endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra) (i : Fin extra) (upper x : ℝ),
  x ∈ Set.lcc 0 upper →
  ∃ (derivativeValue : ℝ),
  HasDerivAt
    (fun (y : ℝ) =>
      Rhat
        (GN21DriverSurgePricing.gn21EndpointVectorPolicy
          (GN21DriverSurgePricing.gn21UpdateEndpoint endpoints
            (GN21DriverSurgePricing.gn21Lemma5GapUpperIndex i) y)))
    derivativeValue x ∧
  GN21DriverSurgePricing.sameStrictSign derivativeValue
    (response
      (GN21DriverSurgePricing.gn21EndpointVectorPolicy
        (GN21DriverSurgePricing.gn21UpdateEndpoint endpoints
          (GN21DriverSurgePricing.gn21Lemma5GapUpperIndex i) x))
      x)

```

field right_top_witness:

```

∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex →
  ∀ (extra : ℕ) (endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra),
    Rhat sigma - margin ≤ Rhat (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints) →
      ∀ (j : Fin extra),
        endpoints (GN21DriverSurgePricing.gn21Lemma5GapLowerIndex j) = T →
          ∃ (rightValue : ℝ),
            endpoints (GN21DriverSurgePricing.gn21Lemma5GapUpperIndex j) < ENNReal.ofReal rightValue ∧
              response (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints) rightValue ≤ 0

```

field component_upper_derivative:

```

∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
GN21DriverSurgePricing.GN21Lemma5ComponentUpperDerivativeCondition Rhat response

```

field component_lower_derivative:

```

∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
GN21DriverSurgePricing.GN21Lemma5ComponentLowerDerivativeCondition Rhat response

```

field component_lower_right_derivative:

```

∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
GN21DriverSurgePricing.GN21Lemma5ComponentLowerRightDerivativeCondition Rhat response

```

field component_tail_lower_derivative:

```

∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
GN21DriverSurgePricing.GN21Lemma5ComponentTailLowerDerivativeCondition Rhat response

```

field open_split_lower_derivative:

```

∀ {Rhat : GN21DriverSurgePricing.SingleStateReward}
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ},
GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin →
  ∀ (policy : GN21DriverSurgePricing.TripPolicy) (pivot : ℝ),
    IsOpen policy →
      policy ⊆ GN21DriverSurgePricing.acceptAllPolicy →
        pivot ∈ policy →
          ∃ (derivativeValue : ℝ),
            HasDerivAt
              (fun (cutoff : ℝ) => Rhat (GN21DriverSurgePricing.gn21InteriorSplitLowerPolicy policy pivot cutoff))
              derivativeValue pivot ∧
                GN21DriverSurgePricing.sameStrictSign derivativeValue (-response policy pivot)

```

Direct retained dependencies

- [GN21DriverSurgePricing.GN21Lemma5ComponentLowerDerivativeCondition](#)
- [GN21DriverSurgePricing.GN21Lemma5ComponentLowerRightDerivativeCondition](#)
- [GN21DriverSurgePricing.GN21Lemma5ComponentTailLowerDerivativeCondition](#)
- [GN21DriverSurgePricing.GN21Lemma5ComponentUpperDerivativeCondition](#)
- [GN21DriverSurgePricing.GN21Lemma5EndpointVector](#)
- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21EndpointVectorPolicy](#)
- [GN21DriverSurgePricing.gn21InteriorSplitLowerPolicy](#)
- [GN21DriverSurgePricing.gn21Lemma5EndpointDomain](#)
- [GN21DriverSurgePricing.gn21Lemma5GapLowerIndex](#)
- [GN21DriverSurgePricing.gn21Lemma5GapUpperIndex](#)
- [GN21DriverSurgePricing.gn21LowerEndpointIndex](#)
- [GN21DriverSurgePricing.gn21UpdateEndpoint](#)
- [GN21DriverSurgePricing.sameStrictSign](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Appendix D's endpoint argument concerns actual separated components while their complements stay fixed. The record's component-local derivative/sign fields encode that source domain; touching components are handled by the proved coalescing reduction.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation

Source locator: cited publication, lines 3660-3692

Verbatim paper-source connection

Recall in the dynamic model that we constrain σ_i to be measurable, open, subsets of the R^+ . Then, σ_i can be written as a countable union of disjoint subsets of R^+ , i.e. $\sigma_i = \bigcup_{k=0}^{\infty} (u_k, u_{k+1}]$. We further

assume that $u_k \leq u_{k+1}$, for any k, m ; we can do so without loss of generality by making a measure 0 change to σ_i , by adding $u_k = u_{k+1}$ to σ_i .

Suppose u is an upper-endpoint of σ_i , i.e. $\exists k$ such that $u = u_k$. Then, we use $\partial_u H(\sigma_i)$ to denote

the derivative of the set function H with respect to u at σ_i . Similarly, $\partial^- H(\sigma_i)$ is the derivative of H at σ_i with respect to a lower-endpoint of σ_i .

We derive ∂_u

$R(w, \{\sigma_1, \sigma_2\})$, ∂^-

$R(w, \{\sigma_1, \sigma_2\})$. We will make it clear in each instance whether ∂_u

refers to the derivative or ∂^- is an endpoint of σ_1 or σ_2 . For all the derivatives in this subsection ∂_u

with respect to an upper endpoint in σ_i , and ∂^- refers to a derivative at a lower endpoint of σ_i . Note that

$R(w, \{\sigma_1, \sigma_2\}) = -R(w, \{\sigma_1, \sigma_2\})$.

Furthermore:

- We use σ in the function argument when the function depends on policies in both states, and σ_i when it only depends on the policy in state i .
- We use $\propto$ to denote that "two functions of u have the same sign except where $f(u) = 0$ ", rather than proportional to.
- All policy equalities are up to measure 0.

[Next verbatim source excerpt]

Lemma 6. Let $R(w, \sigma)$ be as defined in Lemma 1. Then, ∂_u

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

Proof. Proof.

$\mu_i(\{\sigma_j, \sigma_2\}) =$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

$R(w, \sigma) = \mu_1(\sigma) R_1(w_1, \sigma_1) + \mu_2(\sigma) R_2(w_2, \sigma_2)$

Lean-elaborated paper signature

field densityNN:

```
{mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} →  
{Rhat : GN21DriverSurgePricing.SingleStateReward} →  
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ} →  
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} →  
{sigma : GN21DriverSurgePricing.TripPolicy} →  
{margin : ℝ} →  
  GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation mu Rhat response shape sigma margin →  
  GN21DriverSurgePricing.TripLength → NNReal
```

field measure_eq_withDensity:

```
∀ {mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {Rhat : GN21DriverSurgePricing.SingleStateReward}  
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}  
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ}  
(self : GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation mu Rhat response shape sigma margin),  
mu = MeasureTheory.volume.withDensity fun (tau : GN21DriverSurgePricing.TripLength) => ↑(self.densityNN tau)
```

field density_measurable:

```
∀ {mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {Rhat : GN21DriverSurgePricing.SingleStateReward}  
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}  
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ}  
(self : GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation mu Rhat response shape sigma margin),  
Measurable self.densityNN
```

field policy_equalities_modulo_null_sets:

```
∀ {mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {Rhat : GN21DriverSurgePricing.SingleStateReward}  
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}  
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ}  
(self : GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation mu Rhat response shape sigma margin),  
GN21DriverSurgePricing.ProofBridge.policyEqualitiesModuloNullSets mu Rhat
```

field endpoint_sign_realization:

```
∀ {mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {Rhat : GN21DriverSurgePricing.SingleStateReward}  
{response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ}  
{shape : GN21DriverSurgePricing.Lemma5DerivativeShape} {sigma : GN21DriverSurgePricing.TripPolicy} {margin : ℝ}  
(self : GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation mu Rhat response shape sigma margin),  
GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization Rhat response shape sigma margin
```

Direct retained dependencies

- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.ProofBridge.GN21Lemma5EndpointSignRealization](#)
- [GN21DriverSurgePricing.ProofBridge.policyEqualitiesModuloNullSets](#)
- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The source's Lemma 5 argument works modulo null policy changes and through legal endpoint movements. This record packages the source a.e.-equality convention with the now component-local endpoint variation data, not the rejected arbitrary-context condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar

Source locator: cited publication, lines 217-252**Verbatim paper-source connection**

We start with a model where there is a single world state, i.e., all model components are constant over time. Time is continuous and indexed by t . At each time t , the driver is either open, or busy. While open, the driver receives job (trip) requests from riders according to a Poisson process at rate λ , i.e., the time between requests is exponential with mean λ^{-1} . Job lengths, denoted by τ , are drawn independently and identically from a continuous distribution F . If the driver accepts a job request of length τ at time t (as discussed below), they receive a payout of $w(\tau)$ at time $t + \tau$, at which time they become open again. Otherwise, the driver remains open. Except where specified, the only assumptions on w are that it is continuous and) asymptotically (sub)-linear: $\exists c : \liminf_{\tau \rightarrow \infty} w(\tau) \leq c$, which ensures that the driver reward is also bounded.³

Dynamic model with surge pricing

A model with fixed pricing and arrival rates of jobs is not a realistic representation of ride-hailing platforms. In particular, rider demand (both in intensity and in distribution) may vary substantially over time, even within a day (cf. Appendix Figure 9c). To study how this dynamic nature affects driver decisions, we consider a model with two states, $i \in \{1, 2\}$, where $i = 2$ denotes the surge state. (At a high level, the surge state provides a higher earnings rate to the driver. The precise definition is in Section 2.2.2, after we formulate the driver's earnings rate in each state). The world evolves stochastically between the two states, as a Continuous Time Markov Chain (CTMC). When the world is in state i , the state changes to j according to a fixed exponential clock that ticks at rate $\lambda_i \rightarrow \lambda_j$, independently of other randomness. When open in state i , the driver receives job requests at rate λ_i with lengths $\tau \sim F_i$, and collects payout according to payment function w_i , which is presumed to have the same properties as w in the single-state model. The state of the world may change while a driver is on trip. Crucially, the driver receives payments according to the state of the world i when the trip begins. We will use $w = \{w_1, w_2\}$ to denote the overall pricing mechanism.

2.2

Driver strategies and earnings

In our model, the driver can decide whether to accept the trip request, with no penalty.⁴

[Next verbatim source excerpt]

[form-feed in source extraction]

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is

$$R(w, \sigma), \liminf_{t \rightarrow \infty}$$

$$R(w, \sigma, t)$$

t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

[form-feed in source extraction]

Lemma 1. In the dynamic model, the earnings rate can be decomposed into each state i earnings rate $R_i(w_i, \sigma_i)$ and fraction of time $\mu_i(\sigma)$ spent in state i :

$$R(w, \sigma) = \mu_1(\sigma)R_1(w_1, \sigma_1) + \mu_2(\sigma)R_2(w_2, \sigma_2)$$

with probability 1.

$$i(\sigma_i)$$

As in the single-state model, $R_i(w_i, \sigma_i) = W$ $T_i(\sigma_i)$, where

$$Z$$

$$1$$

$$W_i(\sigma_i) =$$

$$w_i(\tau) dF_i(\tau),$$

$$F_i(\sigma_i) \tau \in \sigma_i$$

$$1$$

$$1$$

$$T_i(\sigma_i) = \sum_{\tau \in \sigma_i} dF_i(\tau)$$

We prove the result by defining a new renewal process, in which a single reward renewal cycle is: the time between the driver is open in state 1 to the next time the driver is open in state 1 after being open in state 2 at least once. In other words, each renewal cycle is composed of some number (potentially zero) of sub-cycles in which the driver is open in state 1 and then is open in state 1 again after a completed trip; one sub-cycle starting with the driver open in state 1 and ending with being open in state 2 (either after a completed trip or a state transition while open); some number (potentially zero) of sub-cycles in which the driver is open in state 2 and then is open in state 2 again after a completed trip; and finally one sub-cycle starting in state 2 and ending with the driver open in state 1.

Given the number of such renewal reward cycles completed up to time t , the total earnings on trips starting in each state (earnings in each sub-cycle) are independent of each other, and then we use Wald's identity (Wald, 1973) to separate $\mu_i(\sigma)$ and $R_i(\sigma_i)$.

Note that $T_i(\sigma_i)$ is not exactly the expected length of time in a single sub-cycle in a state given σ_i , but rather is proportional to it; the multiplicative constant $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$ cancels out with the same constant in the expected earnings in a single sub-cycle in a state given σ_i . This constant emerges from the primitives: when the driver is open in state i , there are two competing exponential clocks (with rates $\lambda_i F_i(\sigma_i)$ and $\lambda_{i \rightarrow j}$, respectively) that determine whether the driver will accept a request before the world state changes.

[Next verbatim source excerpt]

Lemma 3. Let $T_i(\sigma_i)$ be as defined in Lemma 1. The fraction of time a driver following strategy $\sigma = \{\sigma_1, \sigma_2\}$ spends either open in state i or on a trip started in state i is

$$\frac{\lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)}{\lambda_j F_j(\sigma_j) T_j(\sigma_j) Q_i(\sigma_i) + \lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)}$$

$$Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$$

$$q_{i \rightarrow j}(\tau) dF_i(\tau)$$

$$\mu_i(\sigma) =$$

where

$$\tau \in \sigma_i$$

We prove this lemma by finding the expected number of sub-cycles in each state i , i.e., within a larger renewal reward cycle as defined, the expected number of sub-cycles that start with the driver

[form-feed in source extraction]

being open in state i . This expectation is the mean of a geometric random variable parameterized by the probability that the driver will next be open in state j , given the driver is currently open in state i . $Q_i(\sigma_i)$ is proportional to this probability. (As with $T_i(\sigma_i)$, there is a normalizing constant

$$\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$$

); the larger it is, the fewer sub-cycles spent in state i . It has two components: the first is the probability that the state changes before the driver accepts a trip request; the second is the probability that the world state is j when the driver completes a trip. Thus, the numerator in $\mu_i(\sigma)$ is proportional to the length of a sub-cycle in state i , times the fraction of sub-cycles that are started in state i . The larger $Q_j(\sigma_j)$ or $T_i(\sigma_i)$, the more time the driver spends in state i .

Lean-expanded paper semantic target

```
type:
(mul mulj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) →
(arrivall arrivall switchll switchll : ℝ) →
[MeasureTheory.IsProbabilityMeasure mul] →
[MeasureTheory.IsProbabilityMeasure mulj] →
0 < arrivall →
0 < arrivall →
0 < switchll →
0 < switchll →
(policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) →
(∀ (state : Fin 2), MeasurableSet (policy state)) →
(∀ (state : Fin 2), policy state ⊆ GN21DriverSurgePricing.acceptAllPolicy) →
(∀ (state : Fin 2),
0 <
GN21DriverSurgePricing.singleStateTripMass
(GN21DriverSurgePricing.gn21CycleMarkLaw mul mulj state) (policy state)) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 0)
mul →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 1)
mulj →
GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel
(GN21DriverSurgePricing.gn21RawCycleSeedMeasure mul mulj arrivall arrivall switchll
switchll)
mul mulj arrivall arrivall switchll switchll (policy 0) (policy 1)

value:
fun (mul mulj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivall arrivall switchll switchll : ℝ)
[MeasureTheory.IsProbabilityMeasure mul] [MeasureTheory.IsProbabilityMeasure mulj] (harrivall : 0 < arrivall)
(harrivall : 0 < arrivall) (hswitchll : 0 < switchll) (hswitchll : 0 < switchll)
(policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) (hpolicy : ∀ (state : Fin 2), MeasurableSet (policy state))
(hpolicy_subset : ∀ (state : Fin 2), policy state ⊆ GN21DriverSurgePricing.acceptAllPolicy)
(hpolicy_mass :
∀ (state : Fin 2),
0 <
GN21DriverSurgePricing.singleStateTripMass (GN21DriverSurgePricing.gn21CycleMarkLaw mul mulj state)
```



```

(policy state))
(htime1 : MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 0) mul)
(htime1 : MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 1) mul) =>
let P : MeasureTheory.Measure GN21DriverSurgePricing.GN21RawCycleSeed :=
  GN21DriverSurgePricing.gn21RawCycleSeedMeasure mul muj arrivall arrivall switchl switchl;
let initial : Fin 2 := 0;
let other : Fin 2 := AppliedModelingLib.Probability.TwoStateSwitching.otherState initial;
let time1 : ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ :=
  GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateTime initial policy;
let timej : ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ :=
  GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateTime initial policy;
have htime1_source :
  MeasureTheory.Integrable (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedTime initial policy) P :=
  GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_1 mul muj arrivall arrivall switchl
  switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass htime1;
have htimej_source :
  MeasureTheory.Integrable (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedTime other policy) P :=
  GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_2 mul muj arrivall arrivall switchl
  switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass htimej;
have htimej_zero :
  ProbabilityTheory.IdentDistrib (timej 0)
  (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedTime other policy) P P :=
  GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_3 mul muj arrivall arrivall switchl
  switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass;
{ stateTime1 := time1, stateTimej := timej, arrivall_pos := arrivall, arrivall_pos := arrivall,
  switchl_pos := hswitchl, switchl_pos := hswitchl,
  ol_measurable := GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_4 policy hpolicy,
  oj_measurable := GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_5 policy hpolicy,
  ol_subset := GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_6 policy hpolicy_subset,
  oj_subset := GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_7 policy hpolicy_subset,
  massl_pos :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_8 mul muj policy hpolicy_mass,
  massj_pos :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_9 mul muj policy hpolicy_mass,
  stateTime1_integrable :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_10 mul muj arrivall arrivall
    switchl switchl policy htime1_source,
  stateTimej_integrable :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_11 mul muj arrivall arrivall
    switchl switchl policy htimej_source htimej_zero,
  stateTime1_independent :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_12 mul muj arrivall arrivall
    switchl switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass,
  stateTimej_independent :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_13 mul muj arrivall arrivall
    switchl switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass,
  stateTime1_identDistrib :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_14 mul muj arrivall arrivall
    switchl switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass,
  stateTimej_identDistrib :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_15 mul muj arrivall arrivall
    switchl switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass,
  stateTime1_mean_eq :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_16 mul muj arrivall arrivall
    switchl switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass htime1,
  stateTimej_mean_eq :=
    GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar._proof_17 mul muj arrivall arrivall
    switchl switchl arrivall arrivall hswitchl hswitchl policy hpolicy hpolicy_subset hpolicy_mass htimej
  htimej_zero }

```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.otherState](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21CycleMarkLaw](#)
- [GN21DriverSurgePricing.gn21ExitWeightIntegral](#)
- [GN21DriverSurgePricing.gn21ExpectedStateTimeInRenewalCycle](#)
- [GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateTime](#)
- [GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateTime](#)
- [GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedTime](#)
- [GN21DriverSurgePricing.gn21RawCycleSeedMeasure](#)
- [GN21DriverSurgePricing.gn21StateCycleTime](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)

Recorded source-to-declaration screening Verdict: matches

Reason: This constructor derives the Lemma 3 time-cycle record from the literal raw calendar, including pairwise independence, identical distributions, integrability, and exact means for deterministic-tail state-time observations.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar

Paper-local declaration:

GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar

Source locator: cited publication, lines 217-252

Verbatim paper-source connection

We start with a model where there is a single world state, i.e., all model components are constant over time. Time is continuous and indexed by t . At each time t , the driver is either open, or busy. While open, the driver receives job (trip) requests from riders according to a Poisson process at rate λ , i.e., the time between requests is exponential with mean λ^{-1} . Job lengths, denoted by τ , are drawn independently and identically from a continuous distribution F . If the driver accepts a job request of length τ at time t (as discussed below), they receive a payout of $w(\tau)$ at time $t + \tau$, at which time they become open again. Otherwise, the driver remains open. Except where specified, the only assumptions on w are that it is continuous and asymptotically (sub)-linear: $\exists c : \liminf_{\tau \rightarrow \infty} w(\tau) \leq c$, which ensures that the driver reward is also bounded.³

2.1.2

Dynamic model with surge pricing

A model with fixed pricing and arrival rates of jobs is not a realistic representation of ride-hailing platforms. In particular, rider demand (both in intensity and in distribution) may vary substantially over time, even within a day (cf. Appendix Figure 9c). To study how this dynamic nature affects driver decisions, we consider a model with two states, $i \in \{1, 2\}$, where $i = 2$ denotes the surge state. (At a high level, the surge state provides a higher earnings rate to the driver. The precise definition is in Section 2.2.2, after we formulate the driver's earnings rate in each state). The world evolves stochastically between the two states, as a Continuous Time Markov Chain (CTMC). When the world is in state i , the state changes to j according to a fixed exponential clock that ticks at rate $\lambda_i \rightarrow j$, independently of other randomness. When open in state i , the driver receives job requests at rate λ_i with lengths $\tau \sim F_i$, and collects payout according to payment function w_i , which is presumed to have the same properties as w in the single-state model. The state of the world may change while a driver is on trip. Crucially, the driver receives payments according to the state of the world i when the trip begins. We will use $w = \{w_1, w_2\}$ to denote the overall pricing mechanism.

2.2

Driver strategies and earnings

In our model, the driver can decide whether to accept the trip request, with no penalty.⁴

[Next verbatim source excerpt]

[form-feed in source extraction]

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is

$$R(w, \sigma), \liminf_{t \rightarrow \infty}$$

$$R(w, \sigma, t)$$

:

t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

[form-feed in source extraction]

Lemma 1. In the dynamic model, the earnings rate can be decomposed into each state i earnings rate $R_i(w_i, \sigma_i)$ and fraction of time $\mu_i(\sigma)$ spent in state i :

$$R(w, \sigma) = \mu_1(\sigma)R_1(w_1, \sigma_1) + \mu_2(\sigma)R_2(w_2, \sigma_2)$$

with probability 1.

$$i(\sigma_i)$$

As in the single-state model, $R_i(w_i, \sigma_i) = W$

$T_i(\sigma_i)$, where

$$Z$$

$$1$$

$$W_i(\sigma_i) =$$

$$w_i(\tau) dF_i(\tau),$$

$$F_i(\sigma_i) \tau \in \sigma_i$$

$$1$$

$$1$$

$$T_i(\sigma_i) = \sum_{\tau \in \sigma_i} \lambda_i F_i(\sigma_i) F_i(\tau)$$

We prove the result by defining a new renewal process, in which a single reward renewal cycle is: the time between the driver is open in state 1 to the next time the driver is open in state 1 after being open in state 2 at least once. In other words, each renewal cycle is composed of some number (potentially zero) of sub-cycles in which the driver is open in state 1 and then is open in state 1 again after a completed trip; one sub-cycle starting with the driver open in state 1 and ending with being open in state 2 (either after a completed trip or a state transition while open); some number (potentially zero) of sub-cycles in which the driver is open in state 2 and then is open in state 2 again after a completed trip; and finally one sub-cycle starting in state 2 and ending with the driver open in state 1.

Given the number of such renewal reward cycles completed up to time t , the total earnings on trips starting in each state (earnings in each sub-cycle) are independent of each other, and then we use Wald's identity (Wald, 1973) to separate $\mu_i(\sigma)$ and $R_i(\sigma_i)$.

Note that $T_i(\sigma_i)$ is not exactly the expected length of time in a single sub-cycle in a state given σ_i , but rather is proportional to it; the multiplicative constant $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$ cancels out with the same constant in the expected earnings in a single sub-cycle in a state given σ_i . This constant emerges from the primitives: when the driver is open in state i , there are two competing exponential clocks (with rates $\lambda_i F_i(\sigma_i)$ and $\lambda_{i \rightarrow j}$, respectively) that determine whether the driver will accept a request before the world state changes.

[Next verbatim source excerpt]

Lemma 3. Let $T_i(\sigma_i)$ be as defined in Lemma 1. The fraction of time a driver following strategy $\sigma = \{\sigma_1, \sigma_2\}$ spends either open in state i or on a trip started in state i is

$$\lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j) + \lambda_j F_j(\sigma_j) T_j(\sigma_j) Q_i(\sigma_i) + \lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)$$

$$Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$$

$$q_{i \rightarrow j}(\tau) dF_i(\tau)$$

$$\mu_i(\sigma) =$$

where

$$\tau \in \sigma_i$$

We prove this lemma by finding the expected number of sub-cycles in each state i , i.e., within a larger renewal reward cycle as defined, the expected number of sub-cycles that start with the driver

[form-feed in source extraction]

being open in state i . This expectation is the mean of a geometric random variable parameterized by the probability that the driver will next be open in state j , given the driver is currently open in state i . $Q_i(\sigma_i)$ is proportional to this probability. (As with $T_i(\sigma_i)$, there is a normalizing constant

$$\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$$

); the larger it is, the fewer sub-cycles spent in state i . It has two components: the first is the probability that the state changes before the driver accepts a trip request; the second is the probability that the world state is j when the driver completes a trip. Thus, the numerator in $\mu_i(\sigma)$ is proportional to the length of a sub-cycle in state i , times the fraction of sub-cycles that are started in state i . The larger $Q_j(\sigma_j)$ or $T_i(\sigma_i)$, the more time the driver spends in state i .

Lean-expanded paper semantic target

type:

```
(mul muj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) →
(arrivall arrivall switchll switchll : ℝ) →
(wl wj : GN21DriverSurgePricing.PricingFunction) →
[MeasureTheory.IsProbabilityMeasure mul] →
[MeasureTheory.IsProbabilityMeasure muj] →
0 < arrivall →
0 < arrivall →
0 < switchll →
0 < switchll →
(policy : Fin 2 → GN21DriverSurgePricing.TripPolicy) →
(∀ (state : Fin 2), MeasurableSet (policy state)) →
(∀ (state : Fin 2), policy state ⊆ GN21DriverSurgePricing.acceptAllPolicy) →
(∀ (state : Fin 2),
0 <
GN21DriverSurgePricing.singleStateTripMass
(GN21DriverSurgePricing.gn21CycleMarkLaw mul muj state) (policy state)) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 0)
mul →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 1)
muj →
MeasureTheory.IntegrableOn wl (policy 0) mul →
MeasureTheory.IntegrableOn wj (policy 1) muj →
Measurable wl →
Measurable wj →
GN21DriverSurgePricing.GN21DynamicIIDCycleModel
(GN21DriverSurgePricing.gn21RawCycleSeedMeasure mul muj arrivall arrivall
switchll switchll)
mul muj arrivall arrivall switchll switchll wl wj (policy 0) (policy 1)
```

value:

```
fun (mul muj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivall arrivall switchll switchll : ℝ)
(wl wj : GN21DriverSurgePricing.PricingFunction) [MeasureTheory.IsProbabilityMeasure mul]
[MeasureTheory.IsProbabilityMeasure muj] (harrivall : 0 < arrivall) (harrivall : 0 < arrivall)
(hswitchll : 0 < switchll) (hswitchll : 0 < switchll) (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy)
```

```

(hpolicy : ∀ (state : Fin 2), MeasurableSet (policy state))
(hpolicy_subset : ∀ (state : Fin 2), policy state ⊆ GN21DriverSurgePricing.acceptAllPolicy)
(hpolicy_mass :
  ∀ (state : Fin 2),
    0 <
      GN21DriverSurgePricing.singleStateTripMass (GN21DriverSurgePricing.gn21CycleMarkLaw mul muj state)
        (policy state))
(htimeI : MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 0) mul)
(htimeJ : MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 1) muj)
(hpaymentI : MeasureTheory.IntegrableOn wI (policy 0) mul)
(hpaymentJ : MeasureTheory.IntegrableOn wJ (policy 1) muj) (hwI : Measurable wI) (hwJ : Measurable wJ) =>
let P : MeasureTheory.Measure GN21DriverSurgePricing.GN21RawCycleSeed :=
  GN21DriverSurgePricing.gn21RawCycleSeedMeasure mul muj arrivalI arrivalJ switchIJ switchJI;
let initial : Fin 2 := 0;
let other : Fin 2 := AppliedModelingLib.Probability.TwoStateSwitching.otherState initial;
let prices : Fin 2 → GN21DriverSurgePricing.PricingFunction := fun (state : Fin 2) => if state = 0 then wI else wJ;
have timeModel :
  GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel P mul muj arrivalI arrivalJ switchIJ switchJI (policy 0)
    (policy 1) :=
  GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar mul muj arrivalI arrivalJ switchIJ switchJI
    harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass htimeI htimeJ;
let earningI : ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ :=
  GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateEarning initial prices policy;
let earningJ : ℕ → GN21DriverSurgePricing.GN21RawCycleSeed → ℝ :=
  GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateEarning initial prices policy;
have hprices : ∀ (state : Fin 2), Measurable (prices state) :=
  GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_1 wI wJ hwI hwJ;
have hpaymentI_source :
  MeasureTheory.Integrable
    (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedEarning initial (prices initial) policy) P :=
  GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_2 mul muj arrivalI arrivalJ switchIJ
    switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hpaymentI;
have hpaymentJ_source :
  MeasureTheory.Integrable
    (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedEarning other (prices other) policy) P :=
  GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_3 mul muj arrivalI arrivalJ switchIJ
    switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hpaymentJ;
have hpaymentJ_zero :
  ProbabilityTheory.IdentDistrib (earningJ 0)
    (GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedEarning other (prices other) policy) P P :=
  GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_4 mul muj arrivalI arrivalJ switchIJ
    switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hprices;
{ toGN21TimeFractionIIDCycleModel := timeModel, stateEarningI := earningI, stateEarningJ := earningJ,
  stateEarningI_integrable :=
    GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_5 mul muj arrivalI arrivalJ switchIJ
      switchJI wI wJ policy hpaymentI_source,
  stateEarningJ_integrable :=
    GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_6 mul muj arrivalI arrivalJ switchIJ
      switchJI wI wJ policy hpaymentJ_source hpaymentJ_zero,
  stateEarningI_independent :=
    GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_7 mul muj arrivalI arrivalJ switchIJ
      switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hprices,
  stateEarningJ_independent :=
    GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_8 mul muj arrivalI arrivalJ switchIJ
      switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hprices,
  stateEarningI_identDistrib :=
    GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_9 mul muj arrivalI arrivalJ switchIJ
      switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hprices,
  stateEarningJ_identDistrib :=
    GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_10 mul muj arrivalI arrivalJ switchIJ
      switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hprices,
  stateEarningI_mean_eq :=
    GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_11 mul muj arrivalI arrivalJ switchIJ
      switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hpaymentI,
  stateEarningJ_mean_eq :=
    GN21DriverSurgePricing.gn21DynamicIIDCycleModel_of_actualCalendar._proof_12 mul muj arrivalI arrivalJ switchIJ
      switchJI wI wJ harrivalI harrivalJ hswitchIJ hswitchJI policy hpolicy hpolicy_subset hpolicy_mass hpaymentJ
      hpaymentJ_zero }

```

Direct retained dependencies

- [AppliedModelingLib.Probability.TwoStateSwitching.otherState](#)
- [GN21DriverSurgePricing.GN21DynamicIIDCycleModel](#)
- [GN21DriverSurgePricing.GN21RawCycleSeed](#)
- [GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21CycleMarkLaw](#)
- [GN21DriverSurgePricing.gn21ExitWeightIntegral](#)
- [GN21DriverSurgePricing.gn21ExpectedStateEarningInRenewalCycle](#)

- [GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateEarning](#)
- [GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateEarning](#)
- [GN21DriverSurgePricing.gn21RawCalendarFirstExitAccumulatedEarning](#)
- [GN21DriverSurgePricing.gn21RawCycleSeedMeasure](#)
- [GN21DriverSurgePricing.gn21StateMeanEarning](#)
- [GN21DriverSurgePricing.gn21TimeFractionIIDCycleModel_of_actualCalendar](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)

Recorded source-to-declaration screening Verdict: matches

Reason: This constructor derives the Lemma 1 renewal-cycle record from the paper's independent arrival/mark and CTMC-clock primitives by reading successive earning and time observations from deterministic post-exit tails of one raw calendar path.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GN21DriverSurgePricing.paper_lemma1_stochastic_dynamic_reward_decomposition_of_actual_calendar

Source locator: cited publication, lines 217-252**Verbatim paper-source connection**

We start with a model where there is a single world state, i.e., all model components are constant over time. Time is continuous and indexed by t . At each time t , the driver is either open, or busy. While open, the driver receives job (trip) requests from riders according to a Poisson process at rate λ , i.e., the time between requests is exponential with mean λ^{-1} . Job lengths, denoted by τ , are drawn independently and identically from a continuous distribution F . If the driver accepts a job request of length τ at time t (as discussed below), they receive a payout of $w(\tau)$ at time $t + \tau$, at which time they become open again. Otherwise, the driver remains open. Except where specified, the only assumptions on w are that it is continuous and asymptotically (sub)-linear: $\exists c : \liminf_{\tau \rightarrow \infty} w(\tau) \leq c$, which ensures that the driver reward is also bounded.³

Dynamic model with surge pricing

A model with fixed pricing and arrival rates of jobs is not a realistic representation of ride-hailing platforms. In particular, rider demand (both in intensity and in distribution) may vary substantially over time, even within a day (cf. Appendix Figure 9c). To study how this dynamic nature affects driver decisions, we consider a model with two states, $i \in \{1, 2\}$, where $i = 2$ denotes the surge state. (At a high level, the surge state provides a higher earnings rate to the driver. The precise definition is in Section 2.2.2, after we formulate the driver's earnings rate in each state). The world evolves stochastically between the two states, as a Continuous Time Markov Chain (CTMC). When the world is in state i , the state changes to j according to a fixed exponential clock that ticks at rate $\lambda_i \rightarrow j$, independently of other randomness. When open in state i , the driver receives job requests at rate λ_i with lengths $\tau \sim F_i$, and collects payout according to payment function w_i , which is presumed to have the same properties as w in the single-state model. The state of the world may change while a driver is on trip. Crucially, the driver receives payments according to the state of the world i when the trip begins. We will use $w = \{w_1, w_2\}$ to denote the overall pricing mechanism.

2.2

Driver strategies and earnings

In our model, the driver can decide whether to accept the trip request, with no penalty.⁴

[Next verbatim source excerpt]

[form-feed in source extraction]

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is

$$R(w, \sigma), \liminf_{t \rightarrow \infty}$$

$$R(w, \sigma, t)$$

t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

[form-feed in source extraction]

Lemma 1. In the dynamic model, the earnings rate can be decomposed into each state i earnings rate $R_i(w_i, \sigma_i)$ and fraction of time $\mu_i(\sigma)$ spent in state i :

$$R(w, \sigma) = \mu_1(\sigma)R_1(w_1, \sigma_1) + \mu_2(\sigma)R_2(w_2, \sigma_2)$$

with probability 1.

$$i(\sigma_i)$$

As in the single-state model, $R_i(w_i, \sigma_i) = W$ $T_i(\sigma_i)$, where

$$Z$$

$$1$$

$$W_i(\sigma_i) =$$

$$w_i(\tau) dF_i(\tau),$$

$$F_i(\sigma_i) \tau \in \sigma_i$$

$$1$$

$$1$$

$$T_i(\sigma_i) = \sum_{\tau \in \sigma_i} dF_i(\tau)$$

We prove the result by defining a new renewal process, in which a single reward renewal cycle is: the time between the driver is open in state 1 to the next time the driver is open in state 1 after being open in state 2 at least once. In other words, each renewal cycle is composed of some number (potentially zero) of sub-cycles in which the driver is open in state 1 and then is open in state 1 again after a completed trip; one sub-cycle starting with the driver open in state 1 and ending with being open in state 2 (either after a completed trip or a state transition while open); some number (potentially zero) of sub-cycles in which the driver is open in state 2 and then is open in state 2 again after a completed trip; and finally one sub-cycle starting in state 2 and ending with the driver open in state 1.

Given the number of such renewal reward cycles completed up to time t , the total earnings on trips starting in each state (earnings in each sub-cycle) are independent of each other, and then we use Wald's identity (Wald, 1973) to separate $\mu_i(\sigma)$ and $R_i(\sigma_i)$.

Note that $T_i(\sigma_i)$ is not exactly the expected length of time in a single sub-cycle in a state given σ_i , but rather is proportional to it; the multiplicative constant $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$ cancels out with the same constant in the expected earnings in a single sub-cycle in a state given σ_i . This constant emerges from the primitives: when the driver is open in state i , there are two competing exponential clocks (with rates $\lambda_i F_i(\sigma_i)$ and $\lambda_{i \rightarrow j}$, respectively) that determine whether the driver will accept a request before the world state changes.

[Next verbatim source excerpt]

Lemma 3. Let $T_i(\sigma_i)$ be as defined in Lemma 1. The fraction of time a driver following strategy $\sigma = \{\sigma_1, \sigma_2\}$ spends either open in state i or on a trip started in state i is

$$\frac{\lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)}{\lambda_j F_j(\sigma_j) T_j(\sigma_j) Q_i(\sigma_i) + \lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)}$$

$$Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$$

$$q_{i \rightarrow j}(\tau) dF_i(\tau)$$

$$\mu_i(\sigma) =$$

where

$$\tau \in \sigma_i$$

We prove this lemma by finding the expected number of sub-cycles in each state i , i.e., within a larger renewal reward cycle as defined, the expected number of sub-cycles that start with the driver

[form-feed in source extraction]

being open in state i . This expectation is the mean of a geometric random variable parameterized by the probability that the driver will next be open in state j , given the driver is currently open in state i . $Q_i(\sigma_i)$ is proportional to this probability. (As with $T_i(\sigma_i)$, there is a normalizing constant

$$\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$$

); the larger it is, the fewer sub-cycles spent in state i . It has two components: the first is the probability that the state changes before the driver accepts a trip request; the second is the probability that the world state is j when the driver completes a trip. Thus, the numerator in $\mu_i(\sigma)$ is proportional to the length of a sub-cycle in state i , times the fraction of sub-cycles that are started in state i . The larger $Q_j(\sigma_j)$ or $T_i(\sigma_i)$, the more time the driver spends in state i .

Lean-elaborated paper signature

```

∀ (mul muj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivall arrivalJ switchlJ switchJl : ℝ)
(wl wj : GN21DriverSurgePricing.PricingFunction) [MeasureTheory.IsProbabilityMeasure mul]
[MeasureTheory.IsProbabilityMeasure muj],
0 < arrivall →
0 < arrivalJ →
0 < switchlJ →
0 < switchJl →
∀ (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy),
(∀ (state : Fin 2), MeasurableSet (policy state)) →
(∀ (state : Fin 2), policy state ⊆ GN21DriverSurgePricing.acceptAllPolicy) →
(∀ (state : Fin 2),
0 <
GN21DriverSurgePricing.singleStateTripMass (GN21DriverSurgePricing.gn21CycleMarkLaw mul muj state)
(policy state)) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 0) mul →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 1) muj →
MeasureTheory.IntegrableOn wl (policy 0) mul →
MeasureTheory.IntegrableOn wj (policy 1) muj →
Measurable wl →
Measurable wj →
∀m (seed :
GN21DriverSurgePricing.GN21RawCycleSeed) ∂GN21DriverSurgePricing.gn21RawCycleSeedMeasure
mul muj arrivall arrivalJ switchlJ switchJl,
Filter.Tendsto
(fun (n : ℕ) =>
(∑ k ∈ Finset.range n,
GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateEarning 0
(fun (state : Fin 2) => if state = 0 then wl else wj) policy k seed +
∑ k ∈ Finset.range n,
GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateEarning 0
(fun (state : Fin 2) => if state = 0 then wl else wj) policy k seed) /
(∑ k ∈ Finset.range n,
GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateTime 0
policy k seed +
∑ k ∈ Finset.range n,

```



```

GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateTime 0 policy
  k seed))
Filter.atTop
(nhds
  (GN21DriverSurgePricing.gn21MeasuredDynamicReward mul muJ arrivalJ switchJ)
  switchJ) wI wJ (policy 0) (policy 1)))

```

Direct retained dependencies

- GN21DriverSurgePricing.GN21RawCycleSeed
- GN21DriverSurgePricing.PricingFunction
- GN21DriverSurgePricing.TripLength
- GN21DriverSurgePricing.TripPolicy
- GN21DriverSurgePricing.acceptAllPolicy
- GN21DriverSurgePricing.gn21CycleMarkLaw
- GN21DriverSurgePricing.gn21MeasuredDynamicReward
- GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateEarning
- GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateTime
- GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateEarning
- GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateTime
- GN21DriverSurgePricing.gn21RawCycleSeedMeasure
- GN21DriverSurgePricing.singleStateTripMass

Recorded source-to-declaration screening Verdict: matches

Reason: The direct theorem is the source Lemma 1 convergence statement on the actual raw calendar: cumulative earnings and time along its successive deterministic tails converge almost surely to the stated dynamic reward formula.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GN21DriverSurgePricing.paper_lemma3_stochastic_time_fraction_formula_of_actual_calendar

Source locator: cited publication, lines 217-252**Verbatim paper-source connection**

We start with a model where there is a single world state, i.e., all model components are constant over time. Time is continuous and indexed by t . At each time t , the driver is either open, or busy. While open, the driver receives job (trip) requests from riders according to a Poisson process at rate λ , i.e., the time between requests is exponential with mean $1/\lambda$. Job lengths, denoted by τ , are drawn independently and identically from a continuous distribution F . If the driver accepts a job request of length τ at time t (as discussed below), they receive a payout of $w(\tau)$ at time $t + \tau$, at which time they become open again. Otherwise, the driver remains open. Except where specified, the only assumptions on w are that it is continuous and asymptotically (sub)-linear: $\exists c : \liminf_{\tau \rightarrow \infty} w(\tau) \leq c$, which ensures that the driver reward is also bounded.³

Dynamic model with surge pricing

A model with fixed pricing and arrival rates of jobs is not a realistic representation of ride-hailing platforms. In particular, rider demand (both in intensity and in distribution) may vary substantially over time, even within a day (cf. Appendix Figure 9c). To study how this dynamic nature affects driver decisions, we consider a model with two states, $i \in \{1, 2\}$, where $i = 2$ denotes the surge state. (At a high level, the surge state provides a higher earnings rate to the driver. The precise definition is in Section 2.2.2, after we formulate the driver's earnings rate in each state). The world evolves stochastically between the two states, as a Continuous Time Markov Chain (CTMC). When the world is in state i , the state changes to j according to a fixed exponential clock that ticks at rate $\lambda_i \rightarrow \lambda_j$, independently of other randomness. When open in state i , the driver receives job requests at rate λ_i with lengths $\tau \sim F_i$, and collects payout according to payment function w_i , which is presumed to have the same properties as w in the single-state model. The state of the world may change while a driver is on trip. Crucially, the driver receives payments according to the state of the world i when the trip begins. We will use $w = \{w_1, w_2\}$ to denote the overall pricing mechanism.

2.2

Driver strategies and earnings

In our model, the driver can decide whether to accept the trip request, with no penalty.⁴

[Next verbatim source excerpt]

[form-feed in source extraction]

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq \mathbb{R}^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of $\mathbb{R}^+$. When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is

$$R(w, \sigma), \liminf_{t \rightarrow \infty}$$

$$R(w, \sigma, t)$$

t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

[form-feed in source extraction]

Lemma 1. In the dynamic model, the earnings rate can be decomposed into each state i earnings rate $R_i(w_i, \sigma_i)$ and fraction of time $\mu_i(\sigma)$ spent in state i :

$$R(w, \sigma) = \mu_1(\sigma)R_1(w_1, \sigma_1) + \mu_2(\sigma)R_2(w_2, \sigma_2)$$

with probability 1.

$$i(\sigma_i)$$

As in the single-state model, $R_i(w_i, \sigma_i) = W$ $T_i(\sigma_i)$, where

$$Z$$

$$1$$

$$W_i(\sigma_i) =$$

$$w_i(\tau) dF_i(\tau),$$

$$F_i(\sigma_i) \tau \in \sigma_i$$

$$1$$

$$1$$

$$T_i(\sigma_i) = \sum_{\tau \in \sigma_i} dF_i(\tau)$$

We prove the result by defining a new renewal process, in which a single reward renewal cycle is: the time between the driver is open in state 1 to the next time the driver is open in state 1 after being open in state 2 at least once. In other words, each renewal cycle is composed of some number (potentially zero) of sub-cycles in which the driver is open in state 1 and then is open in state 1 again after a completed trip; one sub-cycle starting with the driver open in state 1 and ending with being open in state 2 (either after a completed trip or a state transition while open); some number (potentially zero) of sub-cycles in which the driver is open in state 2 and then is open in state 2 again after a completed trip; and finally one sub-cycle starting in state 2 and ending with the driver open in state 1.

Given the number of such renewal reward cycles completed up to time t , the total earnings on trips starting in each state (earnings in each sub-cycle) are independent of each other, and then we use Wald's identity (Wald, 1973) to separate $\mu_i(\sigma)$ and $R_i(\sigma_i)$.

Note that $T_i(\sigma_i)$ is not exactly the expected length of time in a single sub-cycle in a state given σ_i , but rather is proportional to it; the multiplicative constant $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$ cancels out with the same constant in the expected earnings in a single sub-cycle in a state given σ_i . This constant emerges from the primitives: when the driver is open in state i , there are two competing exponential clocks (with rates $\lambda_i F_i(\sigma_i)$ and $\lambda_{i \rightarrow j}$, respectively) that determine whether the driver will accept a request before the world state changes.

[Next verbatim source excerpt]

Lemma 3. Let $T_i(\sigma_i)$ be as defined in Lemma 1. The fraction of time a driver following strategy $\sigma = \{\sigma_1, \sigma_2\}$ spends either open in state i or on a trip started in state i is

$$\frac{\lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)}{\lambda_j F_j(\sigma_j) T_j(\sigma_j) Q_i(\sigma_i) + \lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)}$$

$$Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$$

$$q_{i \rightarrow j}(\tau) dF_i(\tau)$$

$$\mu_i(\sigma) =$$

where

$$\tau \in \sigma_i$$

We prove this lemma by finding the expected number of sub-cycles in each state i , i.e., within a larger renewal reward cycle as defined, the expected number of sub-cycles that start with the driver

[form-feed in source extraction]

being open in state i . This expectation is the mean of a geometric random variable parameterized by the probability that the driver will next be open in state j , given the driver is currently open in state i . $Q_i(\sigma_i)$ is proportional to this probability. (As with $T_i(\sigma_i)$, there is a normalizing constant

$$\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$$

); the larger it is, the fewer sub-cycles spent in state i . It has two components: the first is the probability that the state changes before the driver accepts a trip request; the second is the probability that the world state is j when the driver completes a trip. Thus, the numerator in $\mu_i(\sigma)$ is proportional to the length of a sub-cycle in state i , times the fraction of sub-cycles that are started in state i . The larger $Q_j(\sigma_j)$ or $T_i(\sigma_i)$, the more time the driver spends in state i .

Lean-elaborated paper signature

```

∀ (mul muj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivall arrivalJ switchIJ switchJI : ℝ)
  [MeasureTheory.IsProbabilityMeasure mul] [MeasureTheory.IsProbabilityMeasure muj],
0 < arrivall →
0 < arrivalJ →
0 < switchIJ →
0 < switchJI →
  ∀ (policy : Fin 2 → GN21DriverSurgePricing.TripPolicy),
  (∀ (state : Fin 2), MeasurableSet (policy state)) →
  (∀ (state : Fin 2), policy state ⊆ GN21DriverSurgePricing.acceptAllPolicy) →
  (∀ (state : Fin 2),
    0 <
      GN21DriverSurgePricing.singleStateTripMass (GN21DriverSurgePricing.gn21CycleMarkLaw mul muj state)
        (policy state)) →
  MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 0) mul →
  MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) (policy 1) muj →
  ∀ᵐ (seed :
    GN21DriverSurgePricing.GN21RawCycleSeed) aGN21DriverSurgePricing.gn21RawCycleSeedMeasure mul muj
    arrivall arrivalJ switchIJ switchJI,
  Filter.Tendsto
    (fun (n : ℕ) =>
      (∑ k ∈ Finset.range n,
        GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateTime 0 policy k
          seed) /
      (∑ k ∈ Finset.range n,
        GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateTime 0 policy k
          seed +
        ∑ k ∈ Finset.range n,
        GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateTime 0 policy k seed))
    Filter.atTop
    (nhds
      (GN21DriverSurgePricing.gn21MeasuredTimeFraction mul muj arrivall arrivalJ switchIJ switchJI
        (policy 0) (policy 1)))

```

Direct retained dependencies

- GN21DriverSurgePricing.GN21RawCycleSeed
- GN21DriverSurgePricing.TripLength
- GN21DriverSurgePricing.TripPolicy
- GN21DriverSurgePricing.acceptAllPolicy
- GN21DriverSurgePricing.gn21CycleMarkLaw
- GN21DriverSurgePricing.gn21MeasuredTimeFraction
- GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleInitialStateTime
- GN21DriverSurgePricing.gn21RawCalendarAlternatingCycleOtherStateTime
- GN21DriverSurgePricing.gn21RawCycleSeedMeasure
- GN21DriverSurgePricing.singleStateTripMass

Recorded source-to-declaration screening Verdict: matches

Reason: The direct theorem is the source Lemma 3 convergence statement on the actual raw calendar: its cumulative initial-state time fraction converges almost surely to the stated measured time-fraction formula.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix source claims

Review row 1

Source locator: cited publication, lines 3828-3829

Verbatim source input

Lemma 10. Fix arbitrary σ_2 , and thus Q_2, T_2, R_2 . Let $\bar{Q}_1, \bar{T}_1$ be the respective values of Q_1, T_1 at $\sigma_1 = (0, \infty)$. Let $w_1(\tau) = m\tau + zq_{1 \rightarrow 2}(\tau)$, where $m = R_2$.

If
—

$$\begin{aligned} & \frac{z}{1} \\ & (T_2 \lambda_{1 \rightarrow 2} + Q_2) \\ & < \\ & R_2 \\ & Q_2 (\lambda_{1 \rightarrow 2} \bar{T}_1 - \bar{Q}_1) + \lambda_{1 \rightarrow 2} (T_2 \lambda_{1 \rightarrow 2} + Q_2) \\ & (\bar{Q}_1 - \lambda_{1 \rightarrow 2}) \end{aligned}$$

∂
Then ∂u
 $R(w, \sigma) > 0$, for all u, σ_1 . Furthermore, the constraint set is feasible regardless of the primitives.

D.4

[Next verbatim source excerpt]

We can now do the same thing for the first state, assuming that $w_1(\tau)$ is of the form $w_1(\tau) = m\tau + zq_{1 \rightarrow 2}(\tau)$, where now $z \leq 0$ and $m = R_2$. In the next lemma, we consider u an upper endpoint

[Next verbatim source excerpt]

[form-feed in source extraction]

Proof. Proof. Similar to previous proof. Suppose we have $w_1(u) = mu + zq_{1 \rightarrow 2}(u)$, for some $m = R_2, z \leq 0$.

Expanded PaperInterface specification

```
∀ (arrivalRate lowerEndpoint u T2 Q2 T1 Q1 Tbar1 Qbar1 switch12 switch21 R2 z ratio : ℝ) (density : ℝ → ℝ)
(harrival_pos : 0 < arrivalRate) (hdensity_pos : 0 < density u) (hQ2_pos : 0 < Q2)
(hden :
  GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch12 lowerEndpoint density
    (GN21DriverSurgePricing.gn21SwitchProb switch12 switch21) u *
    T2 +
    Q2 * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u ≠
    0)
(hq_int :
  IntervalIntegrable (fun (τ : ℝ) => GN21DriverSurgePricing.gn21SwitchProb switch12 switch21 τ * density τ)
    MeasureTheory.volume lowerEndpoint u)
(hq_meas :
  StronglyMeasurableAtFilter (fun (τ : ℝ) => GN21DriverSurgePricing.gn21SwitchProb switch12 switch21 τ * density τ)
    (nhds u) MeasureTheory.volume)
(hq_cont : ContinuousAt (fun (τ : ℝ) => GN21DriverSurgePricing.gn21SwitchProb switch12 switch21 τ * density τ) u)
(hw_int :
  IntervalIntegrable
    (fun (τ : ℝ) => GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 τ * density τ)
    MeasureTheory.volume lowerEndpoint u)
(hw_meas :
  StronglyMeasurableAtFilter
    (fun (τ : GN21DriverSurgePricing.TripLength) =>
      GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 τ * density τ)
    (nhds u) MeasureTheory.volume)
(hw_cont :
  ContinuousAt
    (fun (τ : GN21DriverSurgePricing.TripLength) =>
      GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 τ * density τ)
    u)
(ht_int : IntervalIntegrable (fun (τ : ℝ) => τ * density τ) MeasureTheory.volume lowerEndpoint u)
(ht_meas : StronglyMeasurableAtFilter (fun (τ : ℝ) => τ * density τ) (nhds u) MeasureTheory.volume)
(ht_cont : ContinuousAt (fun (τ : ℝ) => τ * density τ) u),
GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch12 lowerEndpoint density
  (GN21DriverSurgePricing.gn21SwitchProb switch12 switch21) u =
  Q1 →
GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u = T1 →
GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density
  (GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21) u =
  R2 * (T1 - 1) + z * (Q1 - switch12) →
GN21DriverSurgePricing.lemma10StructuredBounds ratio T2 Q2 Tbar1 Qbar1 switch12 →
z ≤ 0 →
z = ratio * R2 →
0 < R2 →
0 < switch12 →
0 < switch12 + switch21 →
0 < u →
0 < T2 * switch12 + Q2 →
0 ≤ switch12 * T1 - Q1 →
```

```

switch12 * T1 - Q1 ≤ switch12 * Tbar1 - Qbar1 →
switch12 < Q1 →
Q1 ≤ Qbar1 →
0 <
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths
arrivalRate switch12 lowerEndpoint u Q2 T2 (R2 * T2) density
(GN21DriverSurgePricing.gn21SwitchProb switch12 switch21)
(GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21)
harrival_pos hdensity_pos hQ2_pos hden hq_int hq_meas hq_cont hw_int hw_meas
hw_cont ht_int ht_meas ht_cont).derivativeValue

```

Retained governing declarations

- [GN21DriverSurgePricing.Lemma6EndpointDerivativeData](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.ctmcStructuredSurgePrice](#)
- [GN21DriverSurgePricing.gn21EndpointQiPath](#)
- [GN21DriverSurgePricing.gn21EndpointTiPath](#)
- [GN21DriverSurgePricing.gn21EndpointWiPath](#)
- [GN21DriverSurgePricing.gn21SwitchProb](#)
- [GN21DriverSurgePricing.lemma10StructuredBounds](#)
- [GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. parameter: $\mathbb{R}$
5. parameter: $\mathbb{R}$
6. parameter: $\mathbb{R}$
7. parameter: $\mathbb{R}$
8. parameter: $\mathbb{R}$
9. parameter: $\mathbb{R}$
10. parameter: $\mathbb{R}$
11. parameter: $\mathbb{R}$
12. parameter: $\mathbb{R}$
13. parameter: $\mathbb{R}$
14. parameter: $\mathbb{R}$
15. parameter: $\mathbb{R} \rightarrow \mathbb{R}$
16. assumption: $0 < \text{arrivalRate}$
17. assumption: $0 < \text{density } u$
18. assumption: $0 < Q2$
19. assumption: $\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch12 lowerEndpoint density (GN21DriverSurgePricing.gn21SwitchProb switch12 switch21) } u * T2 + Q2 * \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u \neq 0$
20. assumption: $\text{IntervalIntegrable (fun } (\tau : \mathbb{R}) \Rightarrow \text{GN21DriverSurgePricing.gn21SwitchProb switch12 switch21 } \tau * \text{density } \tau) \text{ MeasureTheory.volume lowerEndpoint } u$
21. assumption: $\text{StronglyMeasurableAtFilter (fun } (\tau : \mathbb{R}) \Rightarrow \text{GN21DriverSurgePricing.gn21SwitchProb switch12 switch21 } \tau * \text{density } \tau) (\text{nhds } u) \text{ MeasureTheory.volume}$
22. assumption: $\text{ContinuousAt (fun } (\tau : \mathbb{R}) \Rightarrow \text{GN21DriverSurgePricing.gn21SwitchProb switch12 switch21 } \tau * \text{density } \tau) u$
23. assumption: $\text{IntervalIntegrable (fun } (\tau : \mathbb{R}) \Rightarrow \text{GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 } \tau * \text{density } \tau) \text{ MeasureTheory.volume lowerEndpoint } u$
24. assumption: $\text{StronglyMeasurableAtFilter (fun } (\tau : \text{GN21DriverSurgePricing.TripLength}) \Rightarrow \text{GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 } \tau * \text{density } \tau) (\text{nhds } u) \text{ MeasureTheory.volume}$
25. assumption: $\text{ContinuousAt (fun } (\tau : \text{GN21DriverSurgePricing.TripLength}) \Rightarrow \text{GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21 } \tau * \text{density } \tau) u$
26. assumption: $\text{IntervalIntegrable (fun } (\tau : \mathbb{R}) \Rightarrow \tau * \text{density } \tau) \text{ MeasureTheory.volume lowerEndpoint } u$
27. assumption: $\text{StronglyMeasurableAtFilter (fun } (\tau : \mathbb{R}) \Rightarrow \tau * \text{density } \tau) (\text{nhds } u) \text{ MeasureTheory.volume}$
28. assumption: $\text{ContinuousAt (fun } (\tau : \mathbb{R}) \Rightarrow \tau * \text{density } \tau) u$
29. assumption: $\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch12 lowerEndpoint density (GN21DriverSurgePricing.gn21SwitchProb switch12 switch21) } u = Q1$
30. assumption: $\text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u = T1$
31. assumption: $\text{GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density (GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21) } u = R2 * (T1 - 1) + z * (Q1 - \text{switch12})$
32. assumption: $\text{GN21DriverSurgePricing.lemma10StructuredBounds ratio T2 Q2 Tbar1 Qbar1 switch12}$
33. assumption: $z \leq 0$
34. assumption: $z = \text{ratio} * R2$
35. assumption: $0 < R2$
36. assumption: $0 < \text{switch12}$
37. assumption: $0 < \text{switch12} + \text{switch21}$
38. assumption: $0 < u$
39. assumption: $0 < T2 * \text{switch12} + Q2$
40. assumption: $0 \leq \text{switch12} * T1 - Q1$
41. assumption: $\text{switch12} * T1 - Q1 \leq \text{switch12} * Tbar1 - Qbar1$

42. assumption: $\text{switch12} < Q1$
 43. assumption: $Q1 \leq Qbar1$
 44. conclusion: $\text{Real.lt} \uparrow 0$
 (GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths arrivalRate switch12 lowerEndpoint u Q2
 T2 (R2 * T2) density (GN21DriverSurgePricing.gn21SwitchProb switch12 switch21)
 (GN21DriverSurgePricing.ctmcStructuredSurgePrice R2 z switch12 switch21) harrival_pos hdensity_pos hQ2_pos hden
 hq_int hq_meas hq_cont hw_int hw_meas hw_cont ht_int ht_meas ht_cont).derivativeValue

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma10_nonsurge_derivative_positive_of_acceptAll_bounds

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 10's fixed surge quantities, displayed non-surge accept-all bounds, and positive non-surge endpoint derivative are exactly the target's price, integral, endpoint, bound, and conclusion fields.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims

Review row 2

Source locator: cited publication, lines 324-367

Verbatim source input

Lemma 1. In the dynamic model, the earnings rate can be decomposed into each state i earnings rate $R_i(w_i, \sigma_i)$ and fraction of time $\mu_i(\sigma)$ spent in state i :
 $R(w, \sigma) = \mu_1(\sigma)R_1(w_1, \sigma_1) + \mu_2(\sigma)R_2(w_2, \sigma_2)$

with probability 1.

$i(\sigma_i)$
As in the single-state model, $R_i(w_i, \sigma_i) = W$
 $T_i(\sigma_i)$, where

$$\int_0^{\infty} W_i(\sigma_i) = \int_0^{\infty} w_i(\tau) dF_i(\tau),$$
$$F_i(\sigma_i) = \tau \in \sigma_i$$

$$\int_0^{\infty} T_i(\sigma_i) = \int_0^{\infty} \lambda_i F_i(\sigma_i) F_i(\sigma_i)$$

$$\int_0^{\infty} \tau dF_i(\tau) = \int_0^{\infty} \tau \in \sigma_i$$

We prove the result by defining a new renewal process, in which a single reward renewal cycle is: the time between the driver is open in state 1 to the next time the driver is open in state 1 after being open in state 2 at least once. In other words, each renewal cycle is composed of some number (potentially zero) of sub-cycles in which the driver is open in state 1 and then is open in state 1 again after a completed trip; one sub-cycle starting with the driver open in state 1 and ending with being open in state 2 (either after a completed trip or a state transition while open); some number (potentially zero) of sub-cycles in which the driver is open in state 2 and then is open in state 2 again after a completed trip; and finally one sub-cycle starting in state 2 and ending with the driver open in state 1.

Given the number of such renewal reward cycles completed up to time t , the total earnings on trips starting in each state (earnings in each sub-cycle) are independent of each other, and then we use Wald's identity (Wald, 1973) to separate $\mu_i(\sigma)$ and $R_i(\sigma_i)$.

Note that $T_i(\sigma_i)$ is not exactly the expected length of time in a single sub-cycle in a state given σ_i , but rather is proportional to it; the multiplicative constant $\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$ cancels out with the same constant in the expected earnings in a single sub-cycle in a state given σ_i . This constant emerges from the primitives: when the driver is open in state i , there are two competing exponential clocks (with rates $\lambda_i F_i(\sigma_i)$ and $\lambda_{i \rightarrow j}$, respectively) that determine whether the driver will accept a request before the world state changes.

Expanded PaperInterface specification

```
∀ {Omega : Type u_1} [inst : MeasurableSpace Omega] {POmega : MeasureTheory.Measure Omega}
{mul muj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchJ switchJI : ℝ}
{wl wJ : GN21DriverSurgePricing.PricingFunction} {sigmaJ sigmaJ : GN21DriverSurgePricing.TripPolicy}
(C :
  GN21DriverSurgePricing.GN21DynamicIIDCycleModel POmega mul muj arrival arrivalJ switchJ switchJI wl wJ sigmaJ
  sigmaJ),
∀m (omega : Omega) ∂POmega,
Filter.Tendsto
  (fun (n : ℕ) =>
    (∑ k ∈ Finset.range n, C.stateEarningI k omega + ∑ k ∈ Finset.range n, C.stateEarningJ k omega) /
    (∑ k ∈ Finset.range n, C.stateTimeI k omega + ∑ k ∈ Finset.range n, C.stateTimeJ k omega))
  Filter.atTop
  (nhds
    (GN21DriverSurgePricing.gn21MeasuredDynamicReward mul muj arrival arrivalJ switchJ switchJI wl wJ sigmaJ
    sigmaJ))
```

Retained governing declarations

- [GN21DriverSurgePricing.GN21DynamicIIDCycleModel](#)
- [GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21MeasuredDynamicReward](#)

Lean-elaborated claim roles

```

1. parameter: Type u_1
2. parameter: MeasurableSpace Omega
3. parameter: MeasureTheory.Measure Omega
4. parameter: MeasureTheory.Measure GN21DriverSurgePricing.TripLength
5. parameter: MeasureTheory.Measure GN21DriverSurgePricing.TripLength
6. parameter: ℝ
7. parameter: ℝ
8. parameter: ℝ
9. parameter: ℝ
10. parameter: GN21DriverSurgePricing.PricingFunction
11. parameter: GN21DriverSurgePricing.PricingFunction
12. parameter: GN21DriverSurgePricing.TripPolicy
13. parameter: GN21DriverSurgePricing.TripPolicy
14. parameter: GN21DriverSurgePricing.GN21DynamicIIDCycleModel POmega mul muj arrivall arrivalJ switchIJ switchJI wl wJ signal sigmaJ
15. conclusion: POmega
  {x : Omega |
    (fun (omega : Omega) =>
      Filter.Tendsto
        (fun (n : ℕ) =>
          (Σ k ∈ Finset.range n, C.stateEarningI k omega + Σ k ∈ Finset.range n, C.stateEarningJ k omega) /
          (Σ k ∈ Finset.range n, C.stateTimeI k omega + Σ k ∈ Finset.range n, C.stateTimeJ k omega))
        Filter.atTop
        (nhds
          (GN21DriverSurgePricing.gn21MeasuredDynamicReward mul muj arrivall arrivalJ switchIJ switchJI wl wJ
            signal sigmaJ)))
    x}ᶜ =
0

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma1_measured_dynamic_reward_decomposition

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 1's almost-sure state-fraction-weighted dynamic earning-rate decomposition is the target's renewal-cycle convergence statement. The current model prerequisite separately verifies that its IID observables are derived from the raw source calendar.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 657-670

Verbatim source input

Lemma 2. Suppose the world is in state i at time t . Let $q_{i \rightarrow j}(s)$ denote the probability that the world will be in state $j \neq i$ at time $t + s$. Then,

$$q_{i \rightarrow j}(s) = \frac{\lambda_{i \rightarrow j}}{\lambda_{i \rightarrow j} + \lambda_{j \rightarrow i}} (1 - e^{-(\lambda_{i \rightarrow j} + \lambda_{j \rightarrow i})s})$$

Note that $q_{i \rightarrow j}(s)$ is not just the probability that the world state transitions once during time $(t, t + s)$, but the probability that it transitions an odd number of times. This formulation emerges through a standard analysis of two-state CTMCs, in which this probability can be found through the inverse of the Laplace transform of the inverse of the resolvent of the Q -matrix for the system. Incorporating this value in closed form is the main hurdle in extending our results to general systems with more than two states. Using this formulation, the following lemma shows $\mu_i(\sigma)$.

Expanded PaperInterface specification

```
∀ (lambdaJ lambdaJl s : ℝ),
  GN21DriverSurgePricing.gn21SwitchProb lambdaJ lambdaJl s =
    lambdaJ / (lambdaJ + lambdaJl) * (1 - Real.exp (-(lambdaJ + lambdaJl) * s))
```

Retained governing declarations

- [GN21DriverSurgePricing.gn21SwitchProb](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. conclusion: $\text{GN21DriverSurgePricing.gn21SwitchProb } \lambda \lambda_l s = \lambda / (\lambda + \lambda_l) * (1 - \text{Real.exp } (-(\lambda + \lambda_l) * s))$

Lean proof endpoint (built separately)

`GN21DriverSurgePricing.PaperInterface.review_lemma2_switch_probability_formula`

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 2's two-state switch probability is exactly the target's rate ratio times one minus the exponential survival term for the same two rates and duration.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 671-696

Verbatim source input

Lemma 3. Let $T_i(\sigma_i)$ be as defined in Lemma 1. The fraction of time a driver following strategy $\sigma = \{\sigma_1, \sigma_2\}$ spends either open in state i or on a trip started in state i is

$$\lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j) + \lambda_j F_j(\sigma_j) T_j(\sigma_j) Q_i(\sigma_i) + \lambda_i F_i(\sigma_i) T_i(\sigma_i) Q_j(\sigma_j)$$

$$Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$$

$$q_{i \rightarrow j}(\tau) dF_i(\tau)$$

$$\mu_i(\sigma) =$$

where

$$\tau \in \sigma_i$$

We prove this lemma by finding the expected number of sub-cycles in each state i , i.e., within a larger renewal reward cycle as defined, the expected number of sub-cycles that start with the driver

[form-feed in source extraction]

being open in state i . This expectation is the mean of a geometric random variable parameterized by the probability that the driver will next be open in state j , given the driver is currently open in state i . $Q_i(\sigma_i)$ is proportional to this probability. (As with $T_i(\sigma_i)$, there is a normalizing constant

$\lambda_i F_i(\sigma_i) + \lambda_{i \rightarrow j}$); the larger it is, the fewer sub-cycles spent in state i . It has two components: the first is the probability that the state changes before the driver accepts a trip request; the second is the probability that the world state is j when the driver completes a trip. Thus, the numerator in $\mu_i(\sigma)$ is proportional to the length of a sub-cycle in state i , times the fraction of sub-cycles that are started in state i . The larger $Q_j(\sigma_j)$ or $T_i(\sigma_i)$, the more time the driver spends in state i .

Expanded PaperInterface specification

```
∀ {Omega : Type u_1} [inst : MeasurableSpace Omega] {POmega : MeasureTheory.Measure Omega}
{mul muj : MeasureTheory.Measure GN21DriverSurgePricing.TripLength} {arrival arrivalJ switchIJ switchJI : ℝ}
{sigmaI sigmaJ : GN21DriverSurgePricing.TripPolicy}
(C :
  GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel POmega mul muj arrival arrivalJ switchIJ switchJI sigmaI
  sigmaJ),
∀m (omega : Omega) ∂POmega,
Filter.Tendsto
  (fun (n : ℕ) =>
    (Σ k ∈ Finset.range n, C.stateTimeI k omega) /
    (Σ k ∈ Finset.range n, C.stateTimeI k omega + Σ k ∈ Finset.range n, C.stateTimeJ k omega))
  Filter.atTop
  (nhds (GN21DriverSurgePricing.gn21MeasuredTimeFraction mul muj arrival arrivalJ switchIJ switchJI sigmaI sigmaJ))
```

Retained governing declarations

- [GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.gn21MeasuredTimeFraction](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: MeasurableSpace Omega
3. parameter: MeasureTheory.Measure Omega
4. parameter: MeasureTheory.Measure GN21DriverSurgePricing.TripLength
5. parameter: MeasureTheory.Measure GN21DriverSurgePricing.TripLength
6. parameter: ℝ
7. parameter: ℝ
8. parameter: ℝ
9. parameter: ℝ
10. parameter: GN21DriverSurgePricing.TripPolicy
11. parameter: GN21DriverSurgePricing.TripPolicy
12. parameter: GN21DriverSurgePricing.GN21TimeFractionIIDCycleModel POmega mul muj arrival arrivalJ switchIJ switchJI sigmaI sigmaJ
13. conclusion: POmega
  {x : Omega |
    (fun (omega : Omega) =>
      Filter.Tendsto
        (fun (n : ℕ) =>
          (Σ k ∈ Finset.range n, C.stateTimeI k omega) /
          (Σ k ∈ Finset.range n, C.stateTimeI k omega + Σ k ∈ Finset.range n, C.stateTimeJ k omega))
        Filter.atTop
        (nhds
          (GN21DriverSurgePricing.gn21MeasuredTimeFraction mul muj arrival arrivalJ switchIJ switchJI sigmaI
            sigmaJ))))
    x}ᶜ =
```

0

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma3_measured_time_fraction_formula

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 3's almost-sure empirical state-time fraction and displayed T,Q formula agree with the target. The IID time observations are separately derived from the literal raw calendar in the model prerequisite lane.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix source claims (continued)

Review row 5

Source locator: cited publication, lines 2841-2853

Verbatim source input

Lemma 4. Consider the single-state model. There exists an optimal policy σ^* of the form $\sigma^* =$
)
*
*
*
{ $\tau : w(\tau$
 $\tau \geq c$ } such that $R(\sigma^*) = c$. Furthermore, this policy is the unique optimal policy, up to
)
*
sets of measure 0 and up to modifications (subtractions) of sets { $\tau : w(\tau$
 $\tau = c$ }.
)
 $\geq c^*$ },

[Next verbatim source excerpt]

-10
-20
(699.0, 3598.0]
1

Expanded PaperInterface specification

$\forall (\mu : \text{MeasureTheory.Measure } \text{GN21DriverSurgePricing.TripLength}) (\text{arrivalRate} : \mathbb{R})$
($w : \text{GN21DriverSurgePricing.PricingFunction}$),
(Measurable fun ($\tau : \text{GN21DriverSurgePricing.TripLength}$) $\Rightarrow w \tau / \tau$) $\rightarrow$
($\forall (\tau : \text{GN21DriverSurgePricing.TripLength}), 0 < \tau \rightarrow 0 \leq w \tau / \tau$) $\rightarrow$
 $\mu \text{GN21DriverSurgePricing.acceptAllPolicy} \neq \top \rightarrow$
MeasureTheory.IntegrableOn $w \text{GN21DriverSurgePricing.acceptAllPolicy } \mu \rightarrow$
MeasureTheory.IntegrableOn (fun ($\tau : \text{GN21DriverSurgePricing.TripLength}$) $\Rightarrow \tau$)
GN21DriverSurgePricing.acceptAllPolicy $\mu \rightarrow$
 $0 < \text{arrivalRate} \rightarrow$
 $\exists (cstar : \mathbb{R}),$
 $0 \leq cstar \wedge$
 $\exists (\ostar : \text{GN21DriverSurgePricing.TripPolicy}),$
GN21DriverSurgePricing.thresholdRatePolicy $w \text{ cstar } \ostar \wedge$
GN21DriverSurgePricing.singleStateRenewalReward $\mu \text{ arrivalRate } w \ostar = \text{ cstar } \wedge$
GN21DriverSurgePricing.singleStateMeasurableOptimal
(GN21DriverSurgePricing.singleStateRenewalReward $\mu \text{ arrivalRate } w$) $\ostar \wedge$
 $\forall \rho \leq \text{GN21DriverSurgePricing.acceptAllPolicy},$
MeasurableSet $\rho \rightarrow$
GN21DriverSurgePricing.singleStateMeasurableOptimal
(GN21DriverSurgePricing.singleStateRenewalReward $\mu \text{ arrivalRate } w$) $\rho \rightarrow$
GN21DriverSurgePricing.singleStateTripMass μ
(GN21DriverSurgePricing.strictThresholdPolicy $w \text{ cstar } \setminus \rho$) =
 $0 \wedge$
GN21DriverSurgePricing.singleStateTripMass μ
($\rho \setminus \text{GN21DriverSurgePricing.completeThresholdPolicy } w \text{ cstar}$) =
 0

Retained governing declarations

- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.completeThresholdPolicy](#)
- [GN21DriverSurgePricing.singleStateMeasurableOptimal](#)
- [GN21DriverSurgePricing.singleStateRenewalReward](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)
- [GN21DriverSurgePricing.strictThresholdPolicy](#)
- [GN21DriverSurgePricing.thresholdRatePolicy](#)

Lean-elaborated claim roles

```
1. parameter: MeasureTheory.Measure GN21DriverSurgePricing.TripLength
2. parameter: ℝ
3. parameter: GN21DriverSurgePricing.PricingFunction
4. assumption: Measurable fun (τ : GN21DriverSurgePricing.TripLength) => w τ / τ
5. assumption: ∀ (τ : GN21DriverSurgePricing.TripLength), 0 < τ → 0 ≤ w τ / τ
6. assumption: μ GN21DriverSurgePricing.acceptAllPolicy ≠ T
7. assumption: MeasureTheory.IntegrableOn w GN21DriverSurgePricing.acceptAllPolicy μ
8. assumption: MeasureTheory.IntegrableOn (fun (τ : GN21DriverSurgePricing.TripLength) => τ) GN21DriverSurgePricing.acceptAllPolicy μ
9. assumption: 0 < arrivalRate
10. conclusion: ∃ (cstar : ℝ),
  0 ≤ cstar ∧
  ∃ (ostar : GN21DriverSurgePricing.TripPolicy),
    GN21DriverSurgePricing.thresholdRatePolicy w cstar ostar ∧
    GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w ostar = cstar ∧
    GN21DriverSurgePricing.singleStateMeasurableOptimal
      (GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w) ostar ∧
    ∀ p ⊆ GN21DriverSurgePricing.acceptAllPolicy,
      MeasurableSet p →
        GN21DriverSurgePricing.singleStateMeasurableOptimal
          (GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w) p →
          GN21DriverSurgePricing.singleStateTripMass μ
            (GN21DriverSurgePricing.strictThresholdPolicy w cstar \ p) =
              0 ∧
            GN21DriverSurgePricing.singleStateTripMass μ
              (p \ GN21DriverSurgePricing.completeThresholdPolicy w cstar) =
                0
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma4_single_state_threshold_uniqueness

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 4's threshold optimum and uniqueness up to the threshold indifference set agree with the target's reward equality, optimality, and two precise zero-mass set differences.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 3724-3767

Verbatim source input

Lemma 6. Let $R(w, \sigma)$ be as defined in Lemma 1. Then, ∂u

[U+0013 control character]

[U+0012 control character]

[U+0013 control character] [U+0012 control character]

$q_i \rightarrow j(u)$

Q_j

Q_i

$w_i(u) Q_i Q_j$

$r(u, i, w, \sigma),$

$+$

$-$

$R_j +$

R_i

$\Delta j_i +$

u

u

T_i

T_j

T_i

T_j

In other words, $r(u, i, w, \sigma)$ has the same sign as the derivative of the overall reward with respect to u (an upper endpoint of σ_i) at w, σ , but it is not necessarily monotonic with it.

Remark 1. Given assumptions on F_i, w_i :

- $R_i(\sigma), R(\sigma), \mu_i$ are continuous in σ

∂

- ∂u

$R(w, \sigma), r(u, i, w, \sigma)$ are both continuous in u (for fixed σ), and continuous in σ .

•

$q_i \rightarrow j(u)$

is strictly decreasing in u .

u

- If $\Delta j_i < 0$ ($i = 2$ the surge state) and $w_i(u)$ is non-decreasing in u , then $r(u, i, w, \sigma)$ is

∂

$R(w, \sigma)$ is negative up to a certain point

strictly increasing in u for a fixed σ . Thus, ∂u

$u \in (0, \infty) \cup \{\infty\}$ and then positive thereafter.

- If $\Delta j_i > 0$ ($i = 1$ the non-surge state) and $w_i(u)$ is non-increasing in u , then $r(u, i, w, \sigma)$

∂

is strictly decreasing in u for a fixed σ . Thus, ∂u

$R(w, \sigma)$ is positive up to a certain point

Expanded PaperInterface specification

$\forall$ (arrivalRate switchRate lowerEndpoint u $Q_j T_j W_j R_i R_j : \mathbb{R}$) (density switchProb payment : $\mathbb{R} \rightarrow \mathbb{R}$),

$0 < \text{arrivalRate} \rightarrow$

$0 < u \rightarrow$

$0 < Q_j \rightarrow$

$0 < \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u \rightarrow$

$0 < T_j \rightarrow$

$\text{GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density payment } u =$

$R_i * \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u \rightarrow$

$W_j = R_j * T_j \rightarrow$

$\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb } u *$

$T_j +$

$Q_j * \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u \neq$

$0 \rightarrow$

$\text{IntervalIntegrable (fun } (\tau : \mathbb{R}) \Rightarrow \text{switchProb } \tau * \text{density } \tau) \text{ MeasureTheory.volume lowerEndpoint } u \rightarrow$

$\text{StronglyMeasurableAtFilter (fun } (\tau : \mathbb{R}) \Rightarrow \text{switchProb } \tau * \text{density } \tau) (\text{nhds } u) \text{ MeasureTheory.volume} \rightarrow$

$\text{ContinuousAt (fun } (\tau : \mathbb{R}) \Rightarrow \text{switchProb } \tau * \text{density } \tau) u \rightarrow$

$\text{IntervalIntegrable (fun } (\tau : \mathbb{R}) \Rightarrow \text{payment } \tau * \text{density } \tau) \text{ MeasureTheory.volume lowerEndpoint } u \rightarrow$

$\text{StronglyMeasurableAtFilter (fun } (\tau : \mathbb{R}) \Rightarrow \text{payment } \tau * \text{density } \tau) (\text{nhds } u)$

$\text{MeasureTheory.volume} \rightarrow$

$\text{ContinuousAt (fun } (\tau : \mathbb{R}) \Rightarrow \text{payment } \tau * \text{density } \tau) u \rightarrow$

$\text{IntervalIntegrable (fun } (\tau : \mathbb{R}) \Rightarrow \tau * \text{density } \tau) \text{ MeasureTheory.volume lowerEndpoint } u \rightarrow$

$\text{StronglyMeasurableAtFilter (fun } (\tau : \mathbb{R}) \Rightarrow \tau * \text{density } \tau) (\text{nhds } u)$

$\text{MeasureTheory.volume} \rightarrow$

$\text{ContinuousAt (fun } (\tau : \mathbb{R}) \Rightarrow \tau * \text{density } \tau) u \rightarrow$

HasDerivAt

$(\text{fun } (x : \mathbb{R}) \Rightarrow$

$\text{GN21DriverSurgePricing.gn21AggregateDynamicReward}$

$(\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate}$

$\text{lowerEndpoint density switchProb } x)$

Q_j

$(\text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density}$

$x)$

T_j

$(\text{GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density}$

$\text{payment } x)$

$W_j)$

$(\text{arrivalRate} * \text{density } u * Q_j *$

$\text{GN21DriverSurgePricing.gn21DerivativeSignKernel (switchProb } u) u (\text{payment } u)$

$(\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate}$

```

lowerEndpoint density switchProb u)
Qj
(GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint
density u)
Tj
(GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint
density payment u)
Wj /
(GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate
lowerEndpoint density switchProb u *
Tj +
Qj *
GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint
density u) ^
2)
u ^
(0 < density u →
GN21DriverSurgePricing.sameStrictSign
(arrivalRate * density u * Qj *
GN21DriverSurgePricing.gn21DerivativeSignKernel (switchProb u) u
(payment u)
(GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate
lowerEndpoint density switchProb u)
Qj
(GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint
density u)
Tj
(GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint
density payment u)
Wj /
(GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate
lowerEndpoint density switchProb u *
Tj +
Qj *
GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint
density u) ^
2)
(GN21DriverSurgePricing.gn21Lemma6Response (switchProb u) u (payment u)
(GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate
lowerEndpoint density switchProb u)
Qj
(GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density
u)
Tj Ri Rj))

```

Retained governing declarations

- [GN21DriverSurgePricing.gn21AggregateDynamicReward](#)
- [GN21DriverSurgePricing.gn21DerivativeSignKernel](#)
- [GN21DriverSurgePricing.gn21EndpointQiPath](#)
- [GN21DriverSurgePricing.gn21EndpointTiPath](#)
- [GN21DriverSurgePricing.gn21EndpointWiPath](#)
- [GN21DriverSurgePricing.gn21Lemma6Response](#)
- [GN21DriverSurgePricing.sameStrictSign](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. parameter: $\mathbb{R}$
5. parameter: $\mathbb{R}$
6. parameter: $\mathbb{R}$
7. parameter: $\mathbb{R}$
8. parameter: $\mathbb{R}$
9. parameter: $\mathbb{R}$
10. parameter: $\mathbb{R} \rightarrow \mathbb{R}$
11. parameter: $\mathbb{R} \rightarrow \mathbb{R}$
12. parameter: $\mathbb{R} \rightarrow \mathbb{R}$
13. assumption: $0 < \text{arrivalRate}$
14. assumption: $0 < u$
15. assumption: $0 < Qj$
16. assumption: $0 < \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u$
17. assumption: $0 < Tj$
18. assumption: $\text{GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density payment } u = Ri * \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u$
19. assumption: $Wj = Rj * Tj$
20. assumption: $\text{GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb } u * Tj + Qj * \text{GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density } u \neq 0$
21. assumption: $\text{IntervalIntegrable (fun } (\tau : \mathbb{R}) \Rightarrow \text{switchProb } \tau * \text{density } \tau) \text{ MeasureTheory.volume lowerEndpoint } u$
22. assumption: $\text{StronglyMeasurableAtFilter (fun } (\tau : \mathbb{R}) \Rightarrow \text{switchProb } \tau * \text{density } \tau) \text{ (nhds } \tau) \text{ MeasureTheory.volume}$
23. assumption: $\text{ContinuousAt (fun } (\tau : \mathbb{R}) \Rightarrow \text{switchProb } \tau * \text{density } \tau) u$
24. assumption: $\text{IntervalIntegrable (fun } (\tau : \mathbb{R}) \Rightarrow \text{payment } \tau * \text{density } \tau) \text{ MeasureTheory.volume lowerEndpoint } u$
25. assumption: $\text{StronglyMeasurableAtFilter (fun } (\tau : \mathbb{R}) \Rightarrow \text{payment } \tau * \text{density } \tau) \text{ (nhds } u) \text{ MeasureTheory.volume}$
26. assumption: $\text{ContinuousAt (fun } (\tau : \mathbb{R}) \Rightarrow \text{payment } \tau * \text{density } \tau) u$


```

27. assumption: IntervalIntegrable (fun (τ : ℝ) => τ * density τ) MeasureTheory.volume lowerEndpoint u
28. assumption: StronglyMeasurableAtFilter (fun (τ : ℝ) => τ * density τ) (nhds u) MeasureTheory.volume
29. assumption: ContinuousAt (fun (τ : ℝ) => τ * density τ) u
30. conclusion: HasDerivAt
  (fun (x : ℝ) =>
    GN21DriverSurgePricing.gn21AggregateDynamicReward
      (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb x) Qj
      (GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density x) Tj
      (GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density payment x) Wj)
    (arrivalRate * density u * Qj *
      GN21DriverSurgePricing.gn21DerivativeSignKernel (switchProb u) u (payment u)
      (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb u) Qj
      (GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u) Tj
      (GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density payment u) Wj /
      (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb u * Tj +
      Qj * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u) ^
      2)
    u ^
  (0 < density u →
    GN21DriverSurgePricing.sameStrictSign
      (arrivalRate * density u * Qj *
        GN21DriverSurgePricing.gn21DerivativeSignKernel (switchProb u) u (payment u)
        (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb u) Qj
        (GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u) Tj
        (GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density payment u) Wj /
        (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb u * Tj +
        Qj * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u) ^
        2)
      (GN21DriverSurgePricing.gn21Lemma6Response (switchProb u) u (payment u)
      (GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switchRate lowerEndpoint density switchProb u) Qj
      (GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u) Tj Ri Rj))

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma6_upper_endpoint_derivative_formula

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 6's upper-endpoint derivative formula and its positive-density sign equivalence agree with the expanded target's integral paths, response, and exact strict-sign conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 3773-3774

Verbatim source input

Lemma 7. Suppose $w_i(\tau) = m\tau + a$, where $m, a > 0$. Then, $r(u, i, w, \sigma)$ is strictly quasi-convex in u , for each fixed σ where $\Delta_{ji} \leq 0$.

Expanded PaperInterface specification

```
∀ (m a Qi Qj Ti Tj Ri Rj lambdaJ lambdaJl : ℝ),
  0 < m →
  0 < a →
  Rj - Ri ≤ 0 →
  0 < Qi / Ti + Qj / Tj →
  0 < lambdaJl →
  0 < lambdaJl + lambdaJl →
  Ti ≠ 0 →
  Tj ≠ 0 →
  GN21DriverSurgePricing.strictQuasiConvexOnPositive fun (u : GN21DriverSurgePricing.TripLength) =>
    GN21DriverSurgePricing.gn21Lemma6Response
      (GN21DriverSurgePricing.gn21SwitchProb lambdaJl lambdaJl u) u (m * u + a) Qi Qj Ti Tj Ri Rj
```

Retained governing declarations

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21Lemma6Response](#)
- [GN21DriverSurgePricing.gn21SwitchProb](#)
- [GN21DriverSurgePricing.strictQuasiConvexOnPositive](#)

Lean-elaborated claim roles

```
1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. parameter: ℝ
6. parameter: ℝ
7. parameter: ℝ
8. parameter: ℝ
9. parameter: ℝ
10. parameter: ℝ
11. assumption: 0 < m
12. assumption: 0 < a
13. assumption: Rj - Ri ≤ 0
14. assumption: 0 < Qi / Ti + Qj / Tj
15. assumption: 0 < lambdaJl
16. assumption: 0 < lambdaJl + lambdaJl
17. assumption: Ti ≠ 0
18. assumption: Tj ≠ 0
19. parameter: ℝ
20. parameter: ℝ
21. parameter: ℝ
22. assumption: 0 < x
23. assumption: 0 < y
24. assumption: x ≠ y
25. assumption: 0 < θ
26. assumption: θ < 1
27. conclusion: (fun (u : GN21DriverSurgePricing.TripLength) =>
  GN21DriverSurgePricing.gn21Lemma6Response (GN21DriverSurgePricing.gn21SwitchProb lambdaJl lambdaJl u)
    (m * u + a) Qi Qj Ti Tj Ri Rj)
  (θ * x + (1 - θ) * y) <
max
  ((fun (u : GN21DriverSurgePricing.TripLength) =>
    GN21DriverSurgePricing.gn21Lemma6Response (GN21DriverSurgePricing.gn21SwitchProb lambdaJl lambdaJl u)
      (m * u + a) Qi Qj Ti Tj Ri Rj)
    x)
  ((fun (u : GN21DriverSurgePricing.TripLength) =>
    GN21DriverSurgePricing.gn21Lemma6Response (GN21DriverSurgePricing.gn21SwitchProb lambdaJl lambdaJl u)
      (m * u + a) Qi Qj Ti Tj Ri Rj)
    y)
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma7_affine_positive_additive_response_quasi_convex

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 7's positive-additive affine response is strictly quasi-convex under the displayed reward ordering, exactly as stated on the target's positive-length domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 3775-3809

Verbatim source input

Lemma 8. Suppose $w_i(\tau) = m\tau + a$, where $m > 0$ and $a < 0$. Then, $r(u, i, w, \sigma)$ is strictly quasi-concave in u , for each fixed σ where $\Delta_{ji} \geq 0$.

D.3.4

Lemmas for IC policy

Remark 2.

Let
Then

$$\begin{aligned} w_i(u) &= mu + z_{qi \rightarrow j}(u) \\ W_i &= m(T_i - 1) + z(Q_i - \lambda_{i \rightarrow j}) \\ \frac{\partial}{\partial u} R(w, \sigma) &\propto q_{i \rightarrow j}(u) [(R_j - m)T_j T_i + mT_j + zQ_j T_i + zT_j \lambda_{i \rightarrow j}] \\ &+ u [Q_i T_j (m - R_j) + Q_j (m - zQ_i + z\lambda_{i \rightarrow j})] \\ &+ Q_j \end{aligned}$$

Remark 3. $\lim_{u \rightarrow 0} i \rightarrow j u$

(u)

$= \lambda_{i \rightarrow j}$.

Remark 4. $\lambda_{i \rightarrow j} T_i - Q_i \geq 0$ and maximized when $\sigma_i = (0, \infty)$. Similarly, $Q_i \geq 0$ and maximized when $\sigma_i = (0, \infty)$.

$\frac{\partial}{\partial u}$
In the next lemma, we consider u an upper endpoint of σ_2 , and so $\frac{\partial u}{\partial \sigma_1}$
 $R(w = \{w_1, w_2\}, \sigma = \{\sigma_1, \sigma_2\})$ is a derivative with respect to an upper endpoint of σ_2 .

57

[form-feed in source extraction]

Lemma 9. Fix arbitrary σ_1 , and thus Q_1, T_1, R_1 . Let $\bar{Q}_2, \bar{T}_2$ be the respective values of Q_2, T_2 at

Expanded PaperInterface specification

```
∀ (m a Qi Qj Ti Tj Ri Rj lambdad lambdaj : ℝ),
  0 < m →
  a < 0 →
  0 ≤ Rj - Ri →
  0 < Qi / Ti + Qj / Tj →
  0 < lambdad →
  0 < lambdad + lambdaj →
  Ti ≠ 0 →
  Tj ≠ 0 →
  GN21DriverSurgePricing.strictQuasiConcaveOnPositive fun (u : GN21DriverSurgePricing.TripLength) =>
    GN21DriverSurgePricing.gn21Lemma6Response
    (GN21DriverSurgePricing.gn21SwitchProb lambdad lambdaj u) u (m * u + a) Qi Qj Ti Tj Ri Rj
```

Retained governing declarations

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.gn21Lemma6Response](#)
- [GN21DriverSurgePricing.gn21SwitchProb](#)
- [GN21DriverSurgePricing.strictQuasiConcaveOnPositive](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. parameter: $\mathbb{R}$
5. parameter: $\mathbb{R}$
6. parameter: $\mathbb{R}$
7. parameter: $\mathbb{R}$
8. parameter: $\mathbb{R}$
9. parameter: $\mathbb{R}$
10. parameter: $\mathbb{R}$
11. assumption: $0 < m$
12. assumption: $a < 0$
13. assumption: $0 \leq R_j - R_i$
14. assumption: $0 < Q_i / T_i + Q_j / T_j$
15. assumption: $0 < \lambda_d$
16. assumption: $0 < \lambda_d + \lambda_j$
17. assumption: $T_i \neq 0$

```

18. assumption:  $T_j \neq 0$ 
19. parameter:  $\mathbb{R}$ 
20. parameter:  $\mathbb{R}$ 
21. parameter:  $\mathbb{R}$ 
22. assumption:  $0 < x$ 
23. assumption:  $0 < y$ 
24. assumption:  $x \neq y$ 
25. assumption:  $0 < \theta$ 
26. assumption:  $\theta < 1$ 
27. conclusion: min
  ((fun (u : GN21DriverSurgePricing.TripLength) =>
    GN21DriverSurgePricing.gn21Lemma6Response (GN21DriverSurgePricing.gn21SwitchProb lambdaJ lambdaJl u) u
    (m * u + a) Qi Qj Ti Tj Ri Rj)
  x)
  ((fun (u : GN21DriverSurgePricing.TripLength) =>
    GN21DriverSurgePricing.gn21Lemma6Response (GN21DriverSurgePricing.gn21SwitchProb lambdaJ lambdaJl u) u
    (m * u + a) Qi Qj Ti Tj Ri Rj)
  y) <
  (fun (u : GN21DriverSurgePricing.TripLength) =>
    GN21DriverSurgePricing.gn21Lemma6Response (GN21DriverSurgePricing.gn21SwitchProb lambdaJ lambdaJl u) u
    (m * u + a) Qi Qj Ti Tj Ri Rj)
  ( $\theta * x + (1 - \theta) * y$ )

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma8_affine_negative_additive_response_quasi_concave

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 8's negative-additive affine response is strictly quasi-concave under the displayed reward ordering, exactly as stated on the target's positive-length domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 704-708

Verbatim source input

Lemma 9. Fix arbitrary σ_1 , and thus Q_1, T_1, R_1 . Let $\bar{Q}_2, \bar{T}_2$ be the respective values of Q_2, T_2 at $\sigma_2 = (0, \infty)$. Let $w_2(\tau) = m\tau + zq_{2 \rightarrow 1}(\tau)$, where $m > R_1$.

If
z
T1 ($\lambda_{2 \rightarrow 1} \bar{T}_2 - \bar{Q}_2$) - (Q1 + T1 $\lambda_{2 \rightarrow 1}$)
 $\bar{Q}_2 T_1 + Q_1$
[U+0001 control character]<
<
m
-
R
Q1 ($\bar{Q}_2 - \lambda_{2 \rightarrow 1}$)
Q1 ($\lambda_{2 \rightarrow 1} \bar{T}_2 - \bar{Q}_2$) + $\lambda_{2 \rightarrow 1}$ (Q1 + T1 $\lambda_{2 \rightarrow 1}$)
1
 ∂

Then ∂u
 $R(w, \sigma) > 0$, for all u, σ_2 . Furthermore, the constraint set is feasible regardless of the primitives.

We can now do the same thing for the first state, assuming that $w_1(\tau)$ is of the form $w_1(\tau) = m\tau + zq_{1 \rightarrow 2}(\tau)$, where now $z \leq 0$ and $m = R_2$. In the next lemma, we consider u an upper endpoint

∂
of σ_1 , and so ∂u
 $R(w = \{w_1, w_2\}, \sigma = \{\sigma_1, \sigma_2\})$ is a derivative with respect to an upper endpoint of σ_1 . Then,

[Next verbatim source excerpt]

Theorem 3. Let $R_1 < R_2$ be target earning rates during non-surged and surge states, respectively. There exist prices $w = \{w_1, w_2\}$ of the form $w_i(\tau) = m_i \tau + z_i q_{i \rightarrow j}(\tau)$, where $m_1, m_2, z_2 \geq 0$ (but z_1 may be either positive or negative), such that the optimal driver policy is to accept every trip in the surge state and all trips up to a certain length in the non-surge state.

[Next verbatim source excerpt]

Proof. Proof.
Suppose we have $w_2(u) = mu + zq_{2 \rightarrow 1}(u)$, for some $m > R_1, z \geq 0$.

Expanded PaperInterface specification

$\forall$ (arrivalRate lowerEndpoint u T1 Q1 T2 Q2 Tbar2 Qbar2 switch21 switch12 m R1 z ratio : $\mathbb{R}$) (density : $\mathbb{R} \rightarrow \mathbb{R}$)
(harrival_pos : $0 < \text{arrivalRate}$) (hdensity_pos : $0 < \text{density } u$) (hQ1_pos : $0 < Q_1$)
(hden :
 GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch21 lowerEndpoint density
 (GN21DriverSurgePricing.gn21SwitchProb switch21 switch12) u *
 T1 +
 Q1 * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u $\neq$
 0)
(hq_int :
 IntervalIntegrable (fun (τ : $\mathbb{R}$) => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ)
 MeasureTheory.volume lowerEndpoint u)
(hq_meas :
 StronglyMeasurableAtFilter (fun (τ : $\mathbb{R}$) => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ)
 (nhds u) MeasureTheory.volume)
(hq_cont : ContinuousAt (fun (τ : $\mathbb{R}$) => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ) u)
(hw_int :
 IntervalIntegrable
 (fun (τ : $\mathbb{R}$) => GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ)
 MeasureTheory.volume lowerEndpoint u)
(hw_meas :
 StronglyMeasurableAtFilter
 (fun (τ : GN21DriverSurgePricing.TripLength) =>
 GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ)
 (nhds u) MeasureTheory.volume)
(hw_cont :
 ContinuousAt
 (fun (τ : GN21DriverSurgePricing.TripLength) =>
 GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ)
 u)
(ht_int : IntervalIntegrable (fun (τ : $\mathbb{R}$) => τ * density τ) MeasureTheory.volume lowerEndpoint u)
(ht_meas : StronglyMeasurableAtFilter (fun (τ : $\mathbb{R}$) => τ * density τ) (nhds u) MeasureTheory.volume)
(ht_cont : ContinuousAt (fun (τ : $\mathbb{R}$) => τ * density τ) u),
GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch21 lowerEndpoint density
 (GN21DriverSurgePricing.gn21SwitchProb switch21 switch12) u =
 Q2 $\rightarrow$
 GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u = T2 $\rightarrow$
 GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density
 (GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12) u =
 m * (T2 - 1) + z * (Q2 - switch21) $\rightarrow$
 GN21DriverSurgePricing.lemma9StructuredBounds ratio T1 Q1 Tbar2 Qbar2 switch21 $\rightarrow$
 0 $\leq z$ $\rightarrow$
 z = ratio * (m - R1) $\rightarrow$
 0 < m - R1 $\rightarrow$
 0 $\leq R_1$ $\rightarrow$
 0 $\leq T_1$ $\rightarrow$

```

0 < switch21 →
0 < switch21 + switch12 →
0 < u →
0 ≤ switch21 * T2 - Q2 →
switch21 * T2 - Q2 ≤ switch21 * Tbar2 - Qbar2 →
switch21 < Q2 →
Q2 ≤ Qbar2 →
0 <
(GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths
arrivalRate switch21 lowerEndpoint u Q1 T1 (R1 * T1) density
(GN21DriverSurgePricing.gn21SwitchProb switch21 switch12)
(GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12)
harrival_pos hdensity_pos hQ1_pos hden hq_int hq_meas hq_cont hw_int hw_meas
hw_cont ht_int ht_meas ht_cont).derivativeValue

```

Retained governing declarations

- [GN21DriverSurgePricing.Lemma6EndpointDerivativeData](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.ctmcStructuredSurgePrice](#)
- [GN21DriverSurgePricing.gn21EndpointQiPath](#)
- [GN21DriverSurgePricing.gn21EndpointTiPath](#)
- [GN21DriverSurgePricing.gn21EndpointWiPath](#)
- [GN21DriverSurgePricing.gn21SwitchProb](#)
- [GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths](#)
- [GN21DriverSurgePricing.lemma9StructuredBounds](#)

Lean-elaborated claim roles

```

1. parameter: ℝ
2. parameter: ℝ
3. parameter: ℝ
4. parameter: ℝ
5. parameter: ℝ
6. parameter: ℝ
7. parameter: ℝ
8. parameter: ℝ
9. parameter: ℝ
10. parameter: ℝ
11. parameter: ℝ
12. parameter: ℝ
13. parameter: ℝ
14. parameter: ℝ
15. parameter: ℝ
16. parameter: ℝ → ℝ
17. assumption: 0 < arrivalRate
18. assumption: 0 < density u
19. assumption: 0 < Q1
20. assumption: GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch21 lowerEndpoint density
    (GN21DriverSurgePricing.gn21SwitchProb switch21 switch12) u *
    T1 +
    Q1 * GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u ≠
    0
21. assumption: IntervalIntegrable (fun (τ : ℝ) => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ)
    MeasureTheory.volume lowerEndpoint u
22. assumption: StronglyMeasurableAtFilter (fun (τ : ℝ) => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ)
    (nhds u) MeasureTheory.volume
23. assumption: ContinuousAt (fun (τ : ℝ) => GN21DriverSurgePricing.gn21SwitchProb switch21 switch12 τ * density τ) u
24. assumption: IntervalIntegrable (fun (τ : ℝ) => GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ)
    MeasureTheory.volume lowerEndpoint u
25. assumption: StronglyMeasurableAtFilter
    (fun (τ : GN21DriverSurgePricing.TripLength) =>
      GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ)
    (nhds u) MeasureTheory.volume
26. assumption: ContinuousAt
    (fun (τ : GN21DriverSurgePricing.TripLength) =>
      GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12 τ * density τ)
    u
27. assumption: IntervalIntegrable (fun (τ : ℝ) => τ * density τ) MeasureTheory.volume lowerEndpoint u
28. assumption: StronglyMeasurableAtFilter (fun (τ : ℝ) => τ * density τ) (nhds u) MeasureTheory.volume
29. assumption: ContinuousAt (fun (τ : ℝ) => τ * density τ) u
30. assumption: GN21DriverSurgePricing.gn21EndpointQiPath arrivalRate switch21 lowerEndpoint density
    (GN21DriverSurgePricing.gn21SwitchProb switch21 switch12) u =
    Q2
31. assumption: GN21DriverSurgePricing.gn21EndpointTiPath arrivalRate lowerEndpoint density u = T2
32. assumption: GN21DriverSurgePricing.gn21EndpointWiPath arrivalRate lowerEndpoint density
    (GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12) u =
    m * (T2 - 1) + z * (Q2 - switch21)
33. assumption: GN21DriverSurgePricing.lemma9StructuredBounds ratio T1 Q1 Tbar2 Qbar2 switch21
34. assumption: 0 ≤ z
35. assumption: z = ratio * (m - R1)
36. assumption: 0 < m - R1
37. assumption: 0 ≤ R1

```

38. assumption: $0 \leq T1$
 39. assumption: $0 < \text{switch21}$
 40. assumption: $0 < \text{switch21} + \text{switch12}$
 41. assumption: $0 < u$
 42. assumption: $0 \leq \text{switch21} * T2 - Q2$
 43. assumption: $\text{switch21} * T2 - Q2 \leq \text{switch21} * Tbar2 - Qbar2$
 44. assumption: $\text{switch21} < Q2$
 45. assumption: $Q2 \leq Qbar2$
 46. conclusion: $\text{Real.lt} \uparrow 0$
 (GN21DriverSurgePricing.lemma6EndpointDerivativeData_of_interval_density_paths arrivalRate switch21 lowerEndpoint u Q1
 T1 (R1 * T1) density (GN21DriverSurgePricing.gn21SwitchProb switch21 switch12)
 (GN21DriverSurgePricing.ctmcStructuredSurgePrice m z switch21 switch12) harrival_pos hdensity_pos hQ1_pos hden
 hq_int hq_meas hq_cont hw_int hw_meas hw_cont ht_int ht_meas ht_cont).derivativeValue

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_lemma9_surge_derivative_positive_of_acceptAll_bounds

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 9's fixed non-surge quantities, structured surge price, accept-all bounds, and positive surge endpoint derivative agree with the target's exact fields and conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims (continued)

Review row 10

Source locator: cited publication, lines 498-548

Verbatim source input

Theorem 1 establishes that, in a single-state model with Poisson job arrivals, the length of the job is not important, only the hourly rate while busy on the job. The optimal cw in the policy is

This practice is not consistent across marketplaces and locations. For example, in California as of January 2020, Uber shows the driver the destination and payment estimate at request time. Incentive compatible pricing is an important stepping stone to showing this information.

We note that destination discrimination is against Uber’s guidelines and could lead to deactivation (Uber, 2019a).

Lyft has recently experimented with paying drivers for the time it takes to pick up the rider (Auerbach, 2019).

[form-feed in source extraction]

not necessarily $cw = \sup w(\tau)$
 τ : drivers must trade off the earnings rate while on a trip with their utilization rate; the more trips that a driver rejects, the longer the wait for an acceptable trip. In the appendix we prove the result by, starting at an arbitrary policy σ , making changes to the policy that increase the earnings rate while on a job without decreasing the utilization rate. Thus, each such change improves the reward $R(w, \sigma)$, and the sequence of changes results in a policy of the above form, for some threshold c_0 . Then, this threshold c_0 can be optimized, leading to an optimal policy of this form.

An immediate corollary of Theorem 1 is that $w(\tau) = m\tau$, for $m > 0$, is IC. In other words, if the

platform pays a constant rate $w(\tau) = m$ to busy drivers, then in the single-state model it is in the driver’s best interest to accept every trip. This result is driven by the following insight for Poisson arrivals: while receiving long trip requests is more beneficial to drivers in the single-state setting as they increase one’s utilization rate (the driver is busy for a longer time until the next open period), rejecting short trips to cherry-pick long trips decreases utilization by the same amount. Further note that, given an earnings rate target R , calculating the multiplier m and thus an IC pricing policy is trivial.

On the other hand, affine pricing may not be incentive compatible because short trips are worth

more per unit time than are long trips: $w(\tau) = m + \tau$. The optimal policy may thus be to accept

trips in $\sigma = (0, T)$ for some T . However, our next proposition establishes that affine pricing is incentive compatible if the additive component stays small enough as a function of the request arrival rate:

With a single state, $w(\tau) = m\tau + a$ is incentive compatible if $0 \leq a \leq m$

The sufficient condition has a simple intuition: when open, the expected amount of time the driver must wait for the next request is λ^{-1} ; if on-trip time is valued at m per unit-time, then with

$a = m$ the additive component can be interpreted as paying for the driver’s expected waiting time. Thus, while a driver may earn more per hour for a short trip than a long trip with affine pricing, such a short trip is not worth the time the driver must wait for the next trip request. We further note that the condition in the proposition is not a necessary one; however, deriving necessary and sufficient conditions in closed form requires specifying the trip distribution F .

Expanded PaperInterface specification

```
∀ (mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate m a : ℝ),
  GN21DriverSurgePricing.singleStateTripMass mu GN21DriverSurgePricing.acceptAllPolicy = 1 →
  MeasureTheory.IntegrableOn (fun (τ : GN21DriverSurgePricing.TripLength) => τ) GN21DriverSurgePricing.acceptAllPolicy
  mu →
  0 < arrivalRate →
  0 ≤ a →
  a ≤ m / arrivalRate →
  GN21DriverSurgePricing.singleStateMeasurableIncentiveCompatible
  (GN21DriverSurgePricing.affineSingleStateRenewalReward mu arrivalRate m a)
```

Retained governing declarations

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.affineSingleStateRenewalReward](#)
- [GN21DriverSurgePricing.singleStateMeasurableIncentiveCompatible](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)

Lean-elaborated claim roles

1. parameter: MeasureTheory.Measure GN21DriverSurgePricing.TripLength
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. parameter: $\mathbb{R}$
5. assumption: GN21DriverSurgePricing.singleStateTripMass mu GN21DriverSurgePricing.acceptAllPolicy = 1
6. assumption: MeasureTheory.IntegrableOn (fun (τ : GN21DriverSurgePricing.TripLength) => τ) GN21DriverSurgePricing.acceptAllPolicy mu
7. assumption: 0 < arrivalRate
8. assumption: 0 ≤ a
9. assumption: a ≤ m / arrivalRate
10. parameter: GN21DriverSurgePricing.TripPolicy
11. assumption: σ ≤ GN21DriverSurgePricing.acceptAllPolicy
12. assumption: MeasurableSet σ
13. conclusion: GN21DriverSurgePricing.affineSingleStateRenewalReward mu arrivalRate m a σ ≤
GN21DriverSurgePricing.affineSingleStateRenewalReward mu arrivalRate m a GN21DriverSurgePricing.acceptAllPolicy

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_proposition3_1_affine_single_state_ic

Recorded source-to-Spec screening Verdict: matches

Reason: Proposition 3.1's affine price parameter range and single-state accept-all incentive compatibility agree with the target's stated bounds and measurable-policy domination conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 455-520

Verbatim source input

freely reject it without penalty. Drivers often cannot see the rider's destination or the trip length until they pick up the rider (but they can reject a request based on the pick-up time to the rider, without penalty).⁷ Some drivers call ahead to find out the rider's destination or even cancel the trip at the pick-up location, creating negative experiences for both the rider and the driver.⁸ Our notion of incentive compatibility is ex-post, in which drivers would accept all trips even knowing the trip length. This notion is stronger than an ex-ante setting in which the trip length is not revealed to drivers. Furthermore, in practice, jobs have two components: the time it takes to pick up the rider, and the time while the rider is in the driver's vehicle – and the former component is typically unpaid.⁹ Our model combines these two components into an overall trip length, which determines payments.

Markovian surge and model limitations. In practice, surge has strong intra-day patterns – for example, rush hours have higher average surge values, cf. Appendix Figure 9b. However, evolution of surge on finer time scales, on the level of drivers' individual trip decisions, is more volatile and believably Markovian, cf. Appendix Figure 9c. Our theoretical model assumes that surge is Markovian and binary and the response of a single driver, and further ignores spatial effects. We discuss such issues in Sections A.3 and B.1, and our empirical analysis in Section 6 provides evidence that our insights extend to practice despite these theoretical limitations.

3

Incentive compatibility with affine pricing

In this section, we study the incentive compatibility of affine pricing. In Section 3.1, we first characterize the driver's best-response strategy with respect to any pricing function w in the singlestate model. We then observe that multiplicative pricing, a special case of affine pricing where $w(\tau) = m\tau$, is incentive compatible. In contrast, in Section 3.2, we show that in the dynamic model, multiplicative pricing may no longer be incentive compatible. We further derive the structure of optimal driver policies in each state with respect to affine or multiplicative pricing, which will enable numerical study of the incentive compatibility properties of additive and multiplicative surge in Section 5. Section 3.3 discusses the key differences in the two models, setting up Section 4 where we derive incentive compatible pricing functions for the dynamic model.

3.1

Single-state model: multiplicative pricing is incentive compatible

Our first result is a simple optimal driver policy in the single-state model.

Theorem

1. With σ

a single state, for each w there exists a constant $c_w \in \mathbb{R}_+$ such that the policy

σ

$\sigma(\tau) = m\tau$

*

$\sigma = \tau : \tau \geq c_w$ is optimal for the driver with respect to w .

Theorem 1 establishes that, in a single-state model with Poisson job arrivals, the length of the job is not important, only the hourly rate while busy on the job. The optimal c_w in the policy is

7

This practice is not consistent across marketplaces and locations. For example, in California as of January 2020, Uber shows the driver the destination and payment estimate at request time. Incentive compatible pricing is an important stepping stone to showing this information.

8

We note that destination discrimination is against Uber's guidelines and could lead to deactivation (Uber, 2019a).

9

Lyft has recently experimented with paying drivers for the time it takes to pick up the rider (Auerbach, 2019).

9

[form-feed in source extraction]

)

not necessarily $c_w = \sup w(\tau)$

τ : drivers must trade off the earnings rate while on a trip with their

utilization rate; the more trips that a driver rejects, the longer the wait for an acceptable trip. In the appendix we prove the result by, starting at an arbitrary policy σ , making changes to the policy that increase the earnings rate while on a job without decreasing the utilization rate. Thus, each such change improves the reward $R(w, \sigma)$, and the sequence of changes results in a policy of the above form, for some threshold c_0 . Then, this threshold c_0 can be optimized, leading to an optimal policy of this form.

An immediate corollary of Theorem 1 is that $w(\tau) = m\tau$, for $m > 0$, is IC. In other words, if the

Settled source reading The source-facing single-state payout argument is represented by 'GN21PayoutFunction', whose on-trip payments are nonnegative.

Expanded PaperInterface specification

```
∀ (μ : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrivalRate : ℝ)
(w : GN21DriverSurgePricing.ProofBridge.GN21PayoutFunction),
(Measurable fun (τ : GN21DriverSurgePricing.TripLength) => w.toPricingFunction τ / τ) →
μ GN21DriverSurgePricing.acceptAllPolicy ≠ T →
MeasureTheory.IntegrableOn w.toPricingFunction GN21DriverSurgePricing.acceptAllPolicy μ →
MeasureTheory.IntegrableOn (fun (τ : GN21DriverSurgePricing.TripLength) => τ)
GN21DriverSurgePricing.acceptAllPolicy μ →
0 < arrivalRate →
∃ (c : ℝ),
```

```

0 ≤ c ∧
  ∃ (σ : GN21DriverSurgePricing.TripPolicy),
    GN21DriverSurgePricing.thresholdRatePolicy w.toPricingFunction c σ ∧
    GN21DriverSurgePricing.singleStateMeasurableOptimal
      (GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w.toPricingFunction) σ

```

Retained governing declarations

- [GN21DriverSurgePricing.ProofBridge.GN21PayoutFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.singleStateMeasurableOptimal](#)
- [GN21DriverSurgePricing.singleStateRenewalReward](#)
- [GN21DriverSurgePricing.thresholdRatePolicy](#)

Lean-elaborated claim roles

```

1. parameter: MeasureTheory.Measure GN21DriverSurgePricing.TripLength
2. parameter: ℝ
3. parameter: GN21DriverSurgePricing.ProofBridge.GN21PayoutFunction
4. assumption: Measurable fun (τ : GN21DriverSurgePricing.TripLength) => w.toPricingFunction τ / τ
5. assumption: μ GN21DriverSurgePricing.acceptAllPolicy ≠ ⊥
6. assumption: MeasureTheory.IntegrableOn w.toPricingFunction GN21DriverSurgePricing.acceptAllPolicy μ
7. assumption: MeasureTheory.IntegrableOn (fun (τ : GN21DriverSurgePricing.TripLength) => τ) GN21DriverSurgePricing.acceptAllPolicy μ
8. assumption: 0 < arrivalRate
9. conclusion: ∃ (c : ℝ),
  0 ≤ c ∧
    ∃ (σ : GN21DriverSurgePricing.TripPolicy),
      GN21DriverSurgePricing.thresholdRatePolicy w.toPricingFunction c σ ∧
      GN21DriverSurgePricing.singleStateMeasurableOptimal
        (GN21DriverSurgePricing.singleStateRenewalReward μ arrivalRate w.toPricingFunction) σ

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem1_single_state_threshold_best_response

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 1's payment-rate threshold optimizer matches the target's nonnegative cutoff, threshold-membership equivalence, and optimality conclusion; the source payout domain supplies the hourly-rate sign internally.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 560-656

Verbatim source input

Theorem 2. If $w = \{w_1, w_2\}$, there exists an optimal policy $\sigma = \{\sigma_1, \sigma_2\}$ (i.e., that maximizes $R(w, \sigma)$), defined with parameters $t_1, t_2, t_3, t_4, t_5, t_6 \in [0, \infty) \cup \{\infty\}$, such that

- Non-surge state driver optimal policy σ_1 :
 - If w_1 is multiplicative or positive affine, σ_1 rejects long trips, i.e., $\sigma_1 = (0, t_1)$.
 - If w_1 is negative affine, σ_1 rejects short and long trips, i.e., $\sigma_1 = (t_2, t_3)$.

This insight is similar to a result of Kamble (2019); however, in our setting the driver's strategy σ is a subset of R_+ denoting the job requests accepted, as opposed to a discrete set of prices charged. Further, in our settings the driver responds to the platform's prices instead of setting prices, enabling a wider range of IC pricing mechanisms.

[form-feed in source extraction]

- Surge state driver optimal policy σ_2 :
 - If w_2 is multiplicative or negative affine, σ_2 rejects short trips, i.e., $\sigma_2 = (t_4, \infty)$.

- If w_2 is positive affine, σ_2 rejects medium length trips, i.e., $\sigma_2 = (0, t_5) \cup (t_6, \infty)$.

Furthermore, there exist settings where t_i 's take positive finite values, and in which multiplicative pricing is not incentive compatible in either state. Finally, only policies of the appropriate form as indicated (up to differences of measure 0) can be optimal.

We discuss the intuition in the next section. In the appendix, we prove the result for each case as follows: fixing σ_j for $j = i$, we start with an arbitrary open set $\sigma_i = \cup_{k \in \mathbb{N}} (k, u_k)$, recalling that

open sets can be written as a countable union of such disjoint intervals. Then, we find $\partial_u R(w, \sigma)$, the derivative of the set function $R(w, \sigma)$ with respect to one of the interval upper end-points of σ_i , i.e., u_k . This derivative is the infinitesimal change in the overall reward if σ_i is expanded by increasing u_k , and it has useful properties. In the surge state with multiplicative pricing, for

example, ∂_u

$R(w, \sigma)$ has the same sign as a function that is increasing in u , for each fixed σ . With affine pricing, it has the same sign as a quasi-convex (positive affine in the surge state) or quasiconcave (negative affine in the non-surge state) function $\hookrightarrow$ in u , for a fixed σ . Such properties enable

constructing a sequence of changes to σ_i that each do not decrease the reward $R(w, \sigma)$, with the limit being a policy of the appropriate form. In particular, we can show that any policy that is not of the appropriate form above has $\partial_u R(w, \sigma) > 0$ for some u_k , allowing local improvements until adjacent intervals $(k, u_k), (k+1, u_{k+1})$ can be combined or expanded to infinity. The numerics in Section 5 provide examples in which multiplicative pricing is not incentive compatible, i.e., where policies of the form above with positive finite constants strictly increase driver earnings over the driver policy that accepts all trip requests.

The results of rejecting long trips in non-surge (and short trips in surge) extend to arbitrary functions where $w_1(\tau)$ is non-increasing (respectively, $w_2(\tau)$ is non-decreasing). The other two results do not hold with such generality, as the behavior of the derivative may be arbitrarily complex.

3.3

Why is multiplicative surge pricing not incentive compatible?

"I thoroughly dislike short trips ESPECIALLY when I'm picking up in a waning surge zone"

Anonymous driver

What explains the difference between multiplicative pricing being incentive compatible in the singlestate model but not in the dynamic model? In the

$\hookrightarrow$ latter, a driver's policy affects not just their earnings while they are busy, but also the fraction of time during which they are busy during the lucrative surge state. In particular, it turns out, accepting short trips during surge may reduce the amount of time that a driver is on a surge trip! Appendix Figure 7 shows in an example how the fraction of time in the surge state $\mu_2(\sigma)$ changes as a function of how many short trips the driver rejects.

The anonymous driver we quote above identifies the key effect: when surge is short-lived, a driver may only have the chance to complete one surge trip before it ends. Thus, the driver may be better off waiting to receive a longer trip request, as with multiplicative surge they are paid a higher rate for the full duration of the longer trip. (Of course, there is a trade-off as rejecting too many trip requests risks not receiving any acceptable request before surge ends). In the surge state, then, multiplicative pricing does not compensate drivers enough to accept short trips that may reduce their future surge earnings. In the non-surge state, analogously, multiplicative pricing under-values long trips that may prevent taking advantage of a future surge.

11

[form-feed in source extraction]

Affine pricing is a first, reasonable attempt at fixing these issues. In the surge state, the additive value makes the previously under-valued short trips comparatively more valuable, as the earnings per unit time $w_2(\tau) = m_2 + a\tau^2$ (with $a > 0$) are now higher for short trips. Unfortunately, with such pricing the structure for the surge optimal policy becomes $\sigma_2 = (0, t_5) \cup (t_6, \infty)$ - if the values m_2, a are not balanced correctly, the additive value is enough to make accepting extremely short trips $(0, t_5)$ profitable; for medium-length trips $\tau \in (t_5, t_6)$, however, the additive value is not large enough to make up for the fact that accepting the trip prevents accepting another surged trip before surge ends. Similarly, negative affine pricing in the non-surge state, $w_1(\tau) = m_1\tau + a_1$, (with $a_1 < 0$) is now too harsh on very short trips but potentially not enticing enough for long trips. Next, we fix these issues and construct incentive compatible pricing schemes for our dynamic model. Then, in Section 5 we leverage structural results derived here to numerically compare the incentive compatibility of additive and multiplicative surge.

4

Incentive Compatible Surge Pricing

We now present our main result, regarding the structure of incentive compatible pricing in the

dynamic model. To this aim, in Section 4.1, we characterize $\mu_i(\sigma)$, how much time the driver spends in each state. In Section 4.2, we present incentive compatible prices, under a condition R1 on the ratio of per-state earning rate constraints, R2. Section 4.3 discusses an intuition of the IC pricing structure in terms of the driver's opportunity cost.

4.1

Transition probabilities and expected time spent in each state

The expected fraction of time spent in each state, $\mu_i(\sigma)$, depends both on the evolution of the world state and the trips a driver accepts. To quantify the effects previewed in Section 3.3, we first analyze the evolution of the world state CTMC.

Expanded PaperInterface specification

```
((0 < 2 ^
  GN21DriverSurgePricing.rejectsLongTrips 2
  (GN21DriverSurgePricing.theorem2BothStatesContinuousNonsurgeDeviation 0)) ^
GN21DriverSurgePricing.dynamicProfitableDeviation
(GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional
 GN21DriverSurgePricing.theorem2BothStatesContinuousMu
 GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun (i : Fin 2) =>
  GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i))
GN21DriverSurgePricing.theorem2BothStatesContinuousNonsurgeDeviation) ^
(0 < 2 ^
  GN21DriverSurgePricing.rejectsShortTrips 2
  (GN21DriverSurgePricing.theorem2BothStatesContinuousSurgeDeviation 1)) ^
GN21DriverSurgePricing.dynamicProfitableDeviation
(GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional
 GN21DriverSurgePricing.theorem2BothStatesContinuousMu
 GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun (i : Fin 2) =>
  GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i))
GN21DriverSurgePricing.theorem2BothStatesContinuousSurgeDeviation) ^
¬GN21DriverSurgePricing.dynamicIncentiveCompatible
(GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional GN21DriverSurgePricing.theorem2BothStatesContinuousMu
 GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun (i : Fin 2) =>
  GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i)))
```

Retained governing declarations

- [GN21DriverSurgePricing.dynamicIncentiveCompatible](#)
- [GN21DriverSurgePricing.dynamicProfitableDeviation](#)
- [GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional](#)
- [GN21DriverSurgePricing.multiplicativePricing](#)
- [GN21DriverSurgePricing.rejectsLongTrips](#)
- [GN21DriverSurgePricing.rejectsShortTrips](#)
- [GN21DriverSurgePricing.theorem2BothStatesContinuousArrival](#)
- [GN21DriverSurgePricing.theorem2BothStatesContinuousM](#)
- [GN21DriverSurgePricing.theorem2BothStatesContinuousMu](#)
- [GN21DriverSurgePricing.theorem2BothStatesContinuousNonsurgeDeviation](#)
- [GN21DriverSurgePricing.theorem2BothStatesContinuousSurgeDeviation](#)

Lean-elaborated claim roles

```
1. conclusion: (((0 < 2 ^
  GN21DriverSurgePricing.rejectsLongTrips 2
  (GN21DriverSurgePricing.theorem2BothStatesContinuousNonsurgeDeviation 0)) ^
GN21DriverSurgePricing.dynamicProfitableDeviation
(GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional
 GN21DriverSurgePricing.theorem2BothStatesContinuousMu
 GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun (i : Fin 2) =>
  GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i))
GN21DriverSurgePricing.theorem2BothStatesContinuousNonsurgeDeviation) ^
(0 < 2 ^
  GN21DriverSurgePricing.rejectsShortTrips 2
  (GN21DriverSurgePricing.theorem2BothStatesContinuousSurgeDeviation 1)) ^
GN21DriverSurgePricing.dynamicProfitableDeviation
(GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional
 GN21DriverSurgePricing.theorem2BothStatesContinuousMu
 GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun (i : Fin 2) =>
  GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i))
GN21DriverSurgePricing.theorem2BothStatesContinuousSurgeDeviation) ^
¬GN21DriverSurgePricing.dynamicIncentiveCompatible
(GN21DriverSurgePricing.gn21MeasuredDynamicRewardFunctional GN21DriverSurgePricing.theorem2BothStatesContinuousMu
 GN21DriverSurgePricing.theorem2BothStatesContinuousArrival (1 / 4) (1 / 4) fun (i : Fin 2) =>
  GN21DriverSurgePricing.multiplicativePricing (GN21DriverSurgePricing.theorem2BothStatesContinuousM i)))
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem2_multiplicative_positive_finite_cutoff_not_ic_both_states

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 2's counterexample to two positive finite cutoffs being incentive compatible is represented by the target's concrete continuous instance, two profitable deviations, and failure of accept-all optimality.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix source claims (continued 2)

Review row 13

Source locator: cited publication, lines 3859-3880

Verbatim source input

Theorem 4. Consider pricing function $w = \{w_1, w_2\}$, where $i = 2$ is the surge state as defined. Then, there exists an optimal policy $\sigma = \{\sigma_1, \sigma_2\}$ that maximizes $R(w, \sigma)$, with the following properties.

- Non-surge state driver optimal policy σ_1 :
 - If $w_1(\tau) = m_1 \tau + a_1$, for $a_1 \geq 0$, then $\sigma_1 = (0, t_1)$, for some $t_1 \in [0, \infty) \cup \{\infty\}$.
 - If $w_1(\tau) = m_1 \tau - a_1$, for $a_1 > 0$, then $\sigma_1 = (t_2, t_3)$, for some $t_2, t_3 \in [0, \infty) \cup \{\infty\}$.

∂
- If w_1 such that ∂u
 $R(w, \sigma \mid 0 = \{\sigma_{10}, \sigma_{20}\}) > 0$ for all $\sigma \mid 0$, where u is an upper endpoint of an interval that makes up σ_{10} , then $\sigma_1 = (0, \infty)$.

- Surge state driver optimal policy σ_2 :
 - If $w_2(\tau) = m_2 \tau - a_2$, for $a_2 \geq 0$, then $\sigma_1 = (t_4, \infty)$, for some $t_4 \in [0, \infty)$.
 - If $w_2(\tau) = m_2 \tau + a_2$, for $a_2 > 0$, then $\sigma_1 = (0, t_5) \cup (t_6, \infty)$, for some $t_5, t_6 \in [0, \infty) \cup \{\infty\}$.

∂
- If w_2 such that ∂u
 $R(w, \sigma \mid 0 = \{\sigma_{10}, \sigma_{20}\}) > 0$ for all $\sigma \mid 0$, where u is an upper endpoint of an interval that makes up σ_{20} , then $\sigma_2 = (0, \infty)$.

Furthermore, only policies of the given forms can be optimal.

[Next verbatim source excerpt]

[form-feed in source extraction]

- Start with some arbitrary policy $\sigma = \{\sigma_1, \sigma_2\}$.
- With assumption on the surge state providing higher potential earnings, replace σ_2 with a policy that provides higher earnings in state 2 than σ_1 does in state 1, without decreasing total reward.
- Using Lemma 5, replace σ_1 with policy of the appropriate form, without decreasing total reward.
- Using Lemma 5, replace σ_2 with policy of the appropriate form, without decreasing total reward.

∂
Let $r(u, i, w, \sigma)$ be as defined in Lemma 6, a function that has the same sign as ∂u
 $R(w, \sigma)$,
where u is an upper endpoint of an interval that is part of σ_i . Recall that, above, we show

- (Remark 1). $\Delta(\sigma_i, \sigma_j) > 0$ and $w(\tau)$ non-decreasing implies $r(u, i, w, \sigma)$ strictly increasing in $u \in \sigma_i$.
- (Remark 1). $\Delta(\sigma_i, \sigma_j) < 0$ and $w(\tau)$ non-increasing implies $r(u, i, w, \sigma)$ strictly decreasing in $u \in \sigma_i$.
- (Lemma 7). $w(\tau) = m\tau + a$ for $m, a > 0$ and $\Delta(\sigma_i, \sigma_{-i}) \geq 0$ implies $r(u, i, w, \sigma)$ is strictly quasi-convex in $u \in \sigma_i$.
- (Lemma 8). $w(\tau) = m\tau - a$ for $m, a > 0$ and $\Delta(\sigma_i, \sigma_{-i}) \leq 0$ implies $r(u, i, w, \sigma)$ is strictly quasi-concave in $u \in \sigma_i$.

We need to show that there exists a σ of the appropriate form such that $R(w, \sigma) \geq R(w, \sigma \mid 0)$, for all $\sigma \mid 0$.

Start with arbitrary $\sigma \mid 0 = \{\sigma_{10}, \sigma_{20}\}$ where $\sigma_{10}, \sigma_{20} \subseteq \mathbb{R}^+$ are open, measurable sets, but not of the correct form in the theorem statement. Invoking Lemma 5 as appropriate, we construct a sequence of changes to $\sigma \mid 0$ such that the overall reward does not decrease with each change, and the sequence ends with a policy consistent with the theorem statement.

Step A First, if $R_2(\sigma_{20}) < R_1(\sigma_{10})$, then we replace σ_{20} with a policy σ_{2A} such that $R_2(\sigma_{2A}) > R_1(\sigma_1)$, $\forall \sigma_1$.

Let σ_{2A} be such that $\Delta(\sigma_{2A}, \sigma_{100}) > 0$, for all σ_{100} open and measurable. Such σ_{2A} exists by the definition of the surge state (we in fact define the surge state so that such σ_{2A} exists). Then, let $\sigma_A, \{\sigma_{1A} = \sigma_{10}, \sigma_{2A}\}$.

Note that $R(w, \sigma_A) \geq R(w, \sigma \mid 0)$: time spent earning reward at the rate of $R_2(w_2, \sigma_{20})$ is replaced by time spent earning at rate $R_1(w_1, \sigma_{10})$ or earning at rate $R_2(w_2, \sigma_{2A})$; time spent earning at $R_1(w_1, \sigma_{10})$ may be replaced by time earning at rate $R_2(w_2, \sigma_{2A})$.

Step B Now, we replace σ_{1A} with a policy that is of the appropriate form.

Let $\hat{R}(\sigma_1)$, $R(w, \{\sigma_1, \sigma_{2A}\})$. By Lemma 5, there exists σ_{1B} such that $R(w, \{\sigma_{1B}, \sigma_{2A}\}) \geq R(w, \{\sigma_{1A}, \sigma_{2A}\})$, and σ_{1B} is of the required form. The inequality is strict if σ_{1A} is not of the required form.

Note that all the assumptions of Lemma 5 are met for each appropriate case: σ_{2B} such that $\Delta(\sigma_{2B}, \sigma_{10}) > 0, \forall \sigma_{10}$, and so $r(u, i, w, \sigma)$ remains decreasing or strictly quasi-concave as necessary.

Let $\sigma_B, \{\sigma_{1B}, \sigma_{2B} = \sigma_{2A}\}$.

59

[form-feed in source extraction]

Step C Now, we replace σ_{2B} with a policy that is of the appropriate form.

Let $\hat{R}(\sigma_2)$, $R(w, \{\sigma_{1B}, \sigma_2\})$.

By Lemma 5, there exists σ_{2C} such that $R(w, \{\sigma_{1B}, \sigma_{2C}\}) \geq R(w, \{\sigma_{1B}, \sigma_{2B}\})$, and σ_{2C} is of the required form according to the table. The inequality is strict if σ_{2B} is not of the required form.

As before, all the assumptions of Lemma 5 are met for each appropriate case. σ_{2B} such that $\Delta(\sigma_{2B}, \sigma_{10}) > 0, \forall \sigma_{10}$, and so $r(u, i, w, \sigma)$ remains strictly increasing / strictly quasi-convex in u for a fixed σ .

Let $\sigma_C, \{\sigma_{1C} = \sigma_{1B}, \sigma_{2C}\}$.

Thus, we have constructed $\sigma * = \{\sigma_1 * = \sigma_1 C, \sigma_2 * = \sigma_2 C\}$ such that $\sigma_1 *$, $\sigma_2 *$ correspond to theorem statement for the appropriate cases, respectively, and $R(w, \sigma *) \geq R(w, \sigma)$, for all $\sigma = \{\sigma_1, \sigma_2\}$ where $\sigma_1, \sigma_2 \subseteq R+$ are open, measurable sets, with the inequality strict if σ is not of the required form.

Settled source reading Theorem 4 uses the paper’s assumed policy continuity on the acceptance-set domain of the formalization.

Expanded PaperInterface specification

```

∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) [MeasureTheory.NoAtoms (mu 0)]
[MeasureTheory.NoAtoms (mu 1)] [MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)]
[(mu 0).InnerRegularCompactLTop] [(mu 1).InnerRegularCompactLTop] (arrival : Fin 2 → ℝ) (switch12 switch21 : ℝ)
(w : Fin 2 → GN21DriverSurgePricing.PricingFunction) (shape : Fin 2 → GN21DriverSurgePricing.Lemma5DerivativeShape),
0 < arrival 0 →
0 < arrival 1 →
0 < switch12 →
0 < switch21 →
Measurable (w 0) →
Measurable (w 1) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
MeasureTheory.IntegrableOn (w 0) GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
MeasureTheory.IntegrableOn (w 1) GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
GN21DriverSurgePricing.gn21SourceSurgeStateDominance mu arrival w →
GN21DriverSurgePricing.gn21Theorem4NonsurgeLiteralEndpointPriceCase
(GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21 w)
w shape →
GN21DriverSurgePricing.gn21Theorem4SurgeLiteralEndpointPriceCase
(GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21
w)
w shape →
(∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
GN21DriverSurgePricing.dynamicFeasibleOpenPolicy rho →
∀ (i : Fin 2) (tau : GN21DriverSurgePricing.TripPolicy),
GN21DriverSurgePricing.GN21SymmDiffContinuousAt (mu i)
(fun (policy : GN21DriverSurgePricing.TripPolicy) =>
GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12
switch21 w (Function.update rho i policy))
tau) →
(∃ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
GN21DriverSurgePricing.dynamicOpenOptimal
(GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12
switch21 w)
rho ∧
GN21DriverSurgePricing.lemma5SourcePolicyForm (shape 0) (rho 0) ∧
GN21DriverSurgePricing.lemma5SourcePolicyForm (shape 1) (rho 1)) ∧
∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
GN21DriverSurgePricing.dynamicOpenOptimal
(GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12
switch21 w)
rho →
GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere (mu 0) (shape 0)
(rho 0) ∧
GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere (mu 1) (shape 1)
(rho 1))

```

Retained governing declarations

- [GN21DriverSurgePricing.GN21SymmDiffContinuousAt](#)
- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.PricingFunction](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.dynamicFeasibleOpenPolicy](#)
- [GN21DriverSurgePricing.dynamicOpenOptimal](#)
- [GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional](#)
- [GN21DriverSurgePricing.gn21SourceSurgeStateDominance](#)
- [GN21DriverSurgePricing.gn21Theorem4NonsurgeLiteralEndpointPriceCase](#)
- [GN21DriverSurgePricing.gn21Theorem4SurgeLiteralEndpointPriceCase](#)
- [GN21DriverSurgePricing.lemma5SourcePolicyForm](#)
- [GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere](#)

Lean-elaborated claim roles

```
1. parameter: Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength
2. assumption: MeasureTheory.NoAtoms (mu 0)
3. assumption: MeasureTheory.NoAtoms (mu 1)
4. assumption: MeasureTheory.IsFiniteMeasure (mu 0)
5. assumption: MeasureTheory.IsFiniteMeasure (mu 1)
6. assumption: (mu 0).InnerRegularCompactLTTop
7. assumption: (mu 1).InnerRegularCompactLTTop
8. parameter: Fin 2 → ℝ
9. parameter: ℝ
10. parameter: ℝ
11. parameter: Fin 2 → GN21DriverSurgePricing.PricingFunction
12. parameter: Fin 2 → GN21DriverSurgePricing.Lemma5DerivativeShape
13. assumption: 0 < arrival 0
14. assumption: 0 < arrival 1
15. assumption: 0 < switch12
16. assumption: 0 < switch21
17. assumption: Measurable (w 0)
18. assumption: Measurable (w 1)
19. assumption: MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 0)
20. assumption: MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 1)
21. assumption: MeasureTheory.IntegrableOn (w 0) GN21DriverSurgePricing.acceptAllPolicy (mu 0)
22. assumption: MeasureTheory.IntegrableOn (w 1) GN21DriverSurgePricing.acceptAllPolicy (mu 1)
23. assumption: GN21DriverSurgePricing.gn21SourceSurgeStateDominance mu arrival w
24. assumption: GN21DriverSurgePricing.gn21Theorem4NonsurgeLiteralEndpointPriceCase
   (GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21 w) w shape
25. assumption: GN21DriverSurgePricing.gn21Theorem4SurgeLiteralEndpointPriceCase
   (GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21 w) w shape
26. assumption: ∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
   GN21DriverSurgePricing.dynamicFeasibleOpenPolicy rho →
   ∀ (i : Fin 2) (tau : GN21DriverSurgePricing.TripPolicy),
   GN21DriverSurgePricing.GN21SymmDiffContinuousAt (mu i)
   (fun (policy : GN21DriverSurgePricing.TripPolicy) =>
     GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21 w
     (Function.update rho i policy))
   tau
27. conclusion: (∃ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
   GN21DriverSurgePricing.dynamicOpenOptimal
   (GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21 w) rho ∧
   GN21DriverSurgePricing.lemma5SourcePolicyForm (shape 0) (rho 0) ∧
   GN21DriverSurgePricing.lemma5SourcePolicyForm (shape 1) (rho 1)) ∧
   ∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
   GN21DriverSurgePricing.dynamicOpenOptimal
   (GN21DriverSurgePricing.gn21AggregateDynamicRewardFunctional mu arrival switch12 switch21 w) rho →
   GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere (mu 0) (shape 0) (rho 0) ∧
   GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere (mu 1) (shape 1) (rho 1)
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem4_full_structural_policy_forms_direct

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 4's existence of an open optimum and universal statewise structural forms up to null sets agree with the target's literal price/response cases, continuity domain, and universal conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims (continued 2)

Review row 14

Source locator: cited publication, lines 264-285

Verbatim source input

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq R^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of R^+ . When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is

$$R(w, \sigma) = \liminf_{t \rightarrow \infty} \frac{R(w, \sigma, t)}{t}$$

$R(w, \sigma, t)$

t

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

where $w_i(\tau) = m_i \tau + a_i$, with $m_i \geq 0$ (in the single-state model: $w(\tau) = m\tau + a$, with $m \geq 0$; we refer to the case with $a_i > 0$ ($a_i < 0$) as positive (negative) affine pricing). Such pricing functions

[Next verbatim source excerpt]

Theorem 2. If $w = \{w_1, w_2\}$, there exists an optimal policy $\sigma = \{\sigma_1, \sigma_2\}$ (i.e., that maximizes $R(w, \sigma)$), defined with parameters $t_1, t_2, t_3, t_4, t_5, t_6 \in [0, \infty) \cup \{\infty\}$, such that

- Non-surge state driver optimal policy σ_1 :
 - If w_1 is multiplicative or positive affine, σ_1 rejects long trips, i.e., $\sigma_1 = (0, t_1)$.
 - If w_1 is negative affine, σ_1 rejects short and long trips, i.e., $\sigma_1 = (t_2, t_3)$.

10

This insight is similar to a result of Kamble (2019); however, in our setting the driver's strategy σ is a subset of R^+ denoting the job requests accepted, as opposed to a discrete set of prices charged. Further, in our settings the driver responds to the platform's prices instead of setting prices, enabling a wider range of IC pricing mechanisms.

10

[form-feed in source extraction]

- Surge state driver optimal policy σ_2 :
 - If w_2 is multiplicative or negative affine, σ_2 rejects short trips, i.e., $\sigma_2 = (t_4, \infty)$.

- If w_2 is positive affine, σ_2 rejects medium length trips, i.e., $\sigma_2 = (0, t_5) \cup (t_6, \infty)$.

Furthermore, there exist settings where t_i 's take positive finite values, and in which multiplicative pricing is not incentive compatible in either state. Finally, only policies of the appropriate form as indicated (up to differences of measure 0) can be optimal.

We discuss the intuition in the next section. In the appendix, we prove the result for each case as follows: fixing σ_j for $j \neq i$, we start with an arbitrary open set $\sigma_i = \cup_{k \in \mathbb{N}} (t_k, u_k)$, recalling that

open sets can be written as a countable union of such disjoint intervals. Then, we find $\partial u \partial_k R(w, \sigma)$, the derivative of the set function $R(w, \sigma)$ with respect to one of the interval upper end-points of σ_i , i.e., u_k . This derivative is the infinitesimal change in the overall reward if σ_i is expanded by increasing u_k , and it has useful properties. In the surge state with multiplicative pricing, for

example, ∂u

$R(w, \sigma)$ has the same sign as a function that is increasing in u , for each fixed σ . With affine pricing, it has the same sign as a quasi-convex (positive affine in the surge state) or quasiconcave (negative affine in the non-surge state) function

$\hookrightarrow$ in u , for a fixed σ . Such properties enable constructing a sequence of changes to σ_i that each do not decrease the reward $R(w, \sigma)$, with the limit being a policy of the appropriate form. In particular, we can show that any policy that is not of the appropriate form above has $\partial u \partial_k R(w, \sigma) > 0$ for some u_k , allowing local improvements until adjacent intervals (t_k, u_k) , (t_{k+1}, u_{k+1}) can be combined or expanded to infinity. The numerics in Section 5 provide examples in which multiplicative pricing is not incentive compatible, i.e., where policies of the form above with positive finite constants strictly increase driver earnings over the driver policy that accepts all trip requests.

The results of rejecting long trips in non-surge (and short trips in surge) extend to arbitrary functions where $w_1(\tau)$ is non-increasing (respectively, $w_2(\tau)$ is non-decreasing). The other two results do not hold with such generality, as the behavior of the derivative may be arbitrarily complex.

3.3

Why is multiplicative surge pricing not incentive compatible?

"I thoroughly dislike short trips ESPECIALLY when I'm picking up in a waning surge zone"

Anonymous driver

What explains the difference between multiplicative pricing being incentive compatible in the singlestate model but not in the dynamic model? In the

$\hookrightarrow$ latter, a driver's policy affects not just their earnings while they are busy, but also the fraction of time during which they are busy during the lucrative surge state. In particular, it turns out, accepting short trips during surge may reduce the

amount of time that a driver is on a surge trip! Appendix Figure 7 shows in an example how the fraction of time in the surge state $\mu_2(\sigma)$ changes as a function of how many short trips the driver rejects.

The anonymous driver we quote above identifies the key effect: when surge is short-lived, a driver may only have the chance to complete one surge trip before it ends. Thus, the driver may be better off waiting to receive a longer trip request, as with multiplicative surge they are paid a higher rate for the full duration of the longer trip. (Of course, there is a trade-off as rejecting too many trip requests risks not receiving any acceptable request before surge ends). In the surge state, then, multiplicative pricing does not compensate drivers enough to accept short trips that may reduce their future surge earnings. In the non-surge state, analogously, multiplicative pricing under-values long trips that may prevent taking advantage of a future surge.

11

[form-feed in source extraction]

Affine pricing is a first, reasonable attempt at fixing these issues. In the surge state, the additive value makes the previously under-valued short trips comparatively more valuable, as the earnings per unit time $w_2(\tau) = m_2 + a\tau$ (with $a > 0$) are now higher for short trips. Unfortunately, with such pricing the structure for the surge optimal policy becomes $\sigma_2 = (0, t_5) \cup (t_6, \infty)$ – if the values m_2, a are not balanced correctly, the additive value is enough to make accepting extremely short trips $(0, t_5)$ profitable; for medium-length trips $\tau \in (t_5, t_6)$, however, the additive value is not large enough to make up for the fact that accepting the trip prevents accepting another surged trip before surge ends. Similarly, negative affine pricing in the non-surge state, $w_1(\tau) = m_1\tau + a_1$, (with $a_1 < 0$) is now too harsh on very short trips but potentially not enticing enough for long trips. Next, we fix these issues and construct incentive compatible pricing schemes for our dynamic model. Then, in Section 5 we leverage structural results derived here to numerically compare the incentive compatibility of additive and multiplicative surge.

4

Incentive Compatible Surge Pricing

We now present our main result, regarding the structure of incentive compatible pricing in the dynamic model. To this aim, in Section 4.1, we characterize $\mu_i(\sigma)$, how much time the driver spends in each state. In Section 4.2, we present incentive compatible prices, under a condition

R1 on the ratio of per-state earning rate constraints, R

. Section 4.3 discusses an intuition of the IC

2

pricing structure in terms of the driver's opportunity cost.

4.1

Transition probabilities and expected time spent in each state

The expected fraction of time spent in each state, $\mu_i(\sigma)$, depends both on the evolution of the world state and the trips a driver accepts. To quantify the effects previewed in Section 3.3, we first analyze the evolution of the world state CTMC.

[Next verbatim source excerpt]

We assume throughout:

- Distribution of jobs F, F_i is a continuous probability measure, i.e., f, f_i bounded.
- There exists a policy in state 2 that dominates state 1: $\exists \sigma_2$ such that $\Delta(\sigma_2, \sigma_1) > 0, \forall \sigma_1 \in (0, \infty)$.
- σ, σ_i constrained to be measurable with respect to F, F_i , and σ_i are open.

Expanded PaperInterface specification

```

∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) [MeasureTheory.NoAtoms (mu 0)]
[MeasureTheory.NoAtoms (mu 1)] [MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)]
(arrival : Fin 2 → ℝ) (switch12 switch21 : ℝ) (m : Fin 2 → ℝ),
0 ≤ m 0 →
0 ≤ m 1 →
0 < arrival 0 →
0 < arrival 1 →
0 < switch12 →
0 < switch21 →
GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1 →
GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1 →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
(GN21DriverSurgePricing.gn21SourceSurgeStateDominance mu arrival fun (i : Fin 2) =>
GN21DriverSurgePricing.multiplicativePricing (m i)) →
(∃ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
GN21DriverSurgePricing.dynamicOpenOptimal
(GN21DriverSurgePricing.gn21AggregateMultiplicativeDynamicReward mu arrival switch12
switch21 m)
rho) ∧
GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 0)
(rho 0) ∧
GN21DriverSurgePricing.rejectsShortTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 1)
(rho 1)) ∧
∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
GN21DriverSurgePricing.dynamicOpenOptimal
(GN21DriverSurgePricing.gn21AggregateMultiplicativeDynamicReward mu arrival switch12
switch21 m)
rho →
GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 0)
(rho 0) ∧
GN21DriverSurgePricing.rejectsShortTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 1)
(rho 1)

```

Retained governing declarations

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.dynamicOpenOptimal](#)
- [GN21DriverSurgePricing.gn21AggregateMultiplicativeDynamicReward](#)
- [GN21DriverSurgePricing.gn21SourceSurgeStateDominance](#)
- [GN21DriverSurgePricing.multiplicativePricing](#)
- [GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoffAlmostEverywhere](#)
- [GN21DriverSurgePricing.rejectsShortTripsFiniteOrInfiniteCutoffAlmostEverywhere](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)

Lean-elaborated claim roles

```
1. parameter: Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength
2. assumption: MeasureTheory.NoAtoms (mu 0)
3. assumption: MeasureTheory.NoAtoms (mu 1)
4. assumption: MeasureTheory.IsFiniteMeasure (mu 0)
5. assumption: MeasureTheory.IsFiniteMeasure (mu 1)
6. parameter: Fin 2 → ℝ
7. parameter: ℝ
8. parameter: ℝ
9. parameter: Fin 2 → ℝ
10. assumption: 0 ≤ m 0
11. assumption: 0 ≤ m 1
12. assumption: 0 < arrival 0
13. assumption: 0 < arrival 1
14. assumption: 0 < switch12
15. assumption: 0 < switch21
16. assumption: GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1
17. assumption: GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1
18. assumption: MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 0)
19. assumption: MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau) GN21DriverSurgePricing.acceptAllPolicy (mu 1)
20. assumption: GN21DriverSurgePricing.gn21SourceSurgeStateDominance mu arrival fun (i : Fin 2) =>
  GN21DriverSurgePricing.multiplicativePricing (m i)
21. conclusion: (∃ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
  GN21DriverSurgePricing.dynamicOpenOptimal
    (GN21DriverSurgePricing.gn21AggregateMultiplicativeDynamicReward mu arrival switch12 switch21 m) rho ∧
    GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 0) (rho 0) ∧
    GN21DriverSurgePricing.rejectsShortTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 1) (rho 1)) ∧
  ∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
  GN21DriverSurgePricing.dynamicOpenOptimal
    (GN21DriverSurgePricing.gn21AggregateMultiplicativeDynamicReward mu arrival switch12 switch21 m) rho →
    GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 0) (rho 0) ∧
    GN21DriverSurgePricing.rejectsShortTripsFiniteOrInfiniteCutoffAlmostEverywhere (mu 1) (rho 1))
```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem2_multiplicative_policy_shape_source_claim

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 2's multiplicative-pricing open optimum and universal non-surge reject-long/surge reject-short forms up to null sets agree with the target's existence and necessity clauses.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Appendix source claims (continued 3)

Review row 15

Source locator: cited publication, lines 3343-3386

Verbatim source input

Lemma 5. Consider a function $\hat{R}(\sigma)$ that maps open, measurable subsets $\sigma = \cup_{k=1}^{\infty} (k, uk) \subseteq (0, \infty)$ to the non-negative reals, and probability measure F such that F is continuous, i.e. f is bounded.

Let $\partial_u \hat{R}(\sigma)$ denote the partial derivative of $\hat{R}$ with respect to an upper end-point u of the intervals that make up $\sigma = \cup_{k=1}^{\infty} (k, uk)$, i.e., it is the infinitesimal gain in the value of $\hat{R}(\sigma)$ by adding u to the set σ . Suppose,

1. $\hat{R}(\sigma)$ is continuous in σ , and $\partial_u \hat{R}(\sigma)$ exists, for all σ and u .
2. $\partial_u \hat{R}(\sigma)$ is continuous in u , for each fixed σ .
3. $\partial_u \hat{R}(\sigma)$ is continuous in σ , for each fixed u .

Suppose that there exists a function $r(u, \sigma)$ that has the same sign as $\partial_u \hat{R}(\sigma)$, for all σ, u where

$f(u) > 0$. Consider any open measurable subset $\sigma \subseteq (0, \infty)$, where $F(\sigma) > 0$, and $\hat{R}(\sigma \cap \sigma_0) > \hat{R}(\sigma_0)$.

50

[form-feed in source extraction]

Then the following statement holds for each of the below specific cases: "Suppose $\exists [U+000F \text{ control character}] > 0$ s.t. $r(u, \sigma)$ is [Property] in u (for a fixed σ), for all σ such that $\hat{R}(\sigma) \geq \hat{R}(\sigma \cap \sigma_0) - [U+000F \text{ control character}]$. Then, there exists a policy σ^* of the form [Form] such that $\hat{R}(\sigma^*) \geq \hat{R}(\sigma \cap \sigma_0)$, and the inequality is strict unless $\sigma \cap \sigma_0$ is also of the same form."

Property
 $r(u, \sigma) > 0$ (is positive)
 $r(u, \sigma)$ is strictly increasing
 $r(u, \sigma)$ is strictly decreasing
 $r(u, \sigma)$ is strictly quasi-convex
 $r(u, \sigma)$ is strictly quasi-concave

Form
 $\sigma^* = (0, \infty)$
 $\sigma^* = (k^*, \infty)$, for $k^* \in \mathbb{R}^+$
 $\sigma^* = (0, u^*)$, for $u^* \in \mathbb{R}^+ \cup \{\infty\}$
 $\sigma^* = (0, k^*) \cup (u^*, \infty)$, for $k^*, u^* \in \mathbb{R}^+ \cup \{\infty\}$
 $\sigma^* = (k^*, u^*)$, for $k^*, u^* \in \mathbb{R}^+ \cup \{\infty\}$

[Next verbatim source excerpt]

Proof. Proof. The facts that policies not of the appropriate form cannot be optimal follow directly from the respective structures of the derivatives, as we will show below. However, since the domain of the set function $\hat{R}$ is any subset of $\mathbb{R}^+$, we need to also prove existence of a maximizer of the appropriate form. The proof is structured around proving existence of a maximizer in each case, but we will point out where the given facts imply necessity of having the appropriate form.

The general approach is as follows: Start at subset $\sigma \cap \sigma_0 \subseteq (0, \infty) = \cup_{k=1}^{\infty} (k, uk) = \cup_{k=1}^{\infty} \zeta_k$, where the intervals are disjoint and $\zeta_k = (k, uk)$ denotes the k th interval. (recall that any open subset of $\mathbb{R}$ can be uniquely written as the countable union of such disjoint intervals).

Then, do the following:

1. Create a sequence $\sigma_{\delta} \rightarrow \sigma \cap \sigma_0$ (as $\delta \rightarrow 0$), where, for each δ , the set σ_{δ} is δ -close to $\sigma \cap \sigma_0$: $F((\sigma \cap \sigma_0) \setminus \sigma_{\delta}) \cup (\sigma_{\delta} \setminus \sigma \cap \sigma_0) < \delta$.
2. Show that there exists a σ^* of the appropriate form (according to the property that holds above), such that $\hat{R}(\sigma_{\delta}) \leq \hat{R}(\sigma^*)$, $\forall \delta$.

By continuity of the set function $\hat{R}$, this implies that $\hat{R}(\sigma \cap \sigma_0) \leq \hat{R}(\sigma^*)$.

The second step is the main proof step and the only one that depends substantially on the given property of the function $r(u, \sigma)$.

[U+0001 control character]

Step one: a sequence $\sigma_{\delta} \rightarrow \sigma \cap \sigma_0$. Each σ_{δ} will be of the form $\sigma_{\delta} = (0, L) \cup \cup_{k=1}^K (k, uk)$

$k=1 (k, uk) \cup (B, \infty)$,

for some K, B, L that depend on δ . We construct a σ_{δ} such that $F((\sigma \cap \sigma_0) \setminus \sigma_{\delta} \cup \sigma_{\delta} \setminus \sigma \cap \sigma_0) < \delta$ as follows:

- F is a finite (probability) measure, and so there exists K such that $F(\cup_{k=K+1}^{\infty} (k, uk)) < \delta/2$.

0

(Since $F(\sigma) \leq 1$, it follows by the Cauchy condition).

- Let $B \in \mathbb{R}$ s.t. $F((B, \infty)) < \delta/4$. Let $L \in \mathbb{R}$ s.t. $F((0, L)) < \delta/4$. Such B, L exist by condition on F .

[U+0001 control character]

- Set $\sigma_{\delta} = (0, L) \cup \cup_{k=1}^K (k, uk)$

$k=1 (k, uk) \cup (B, \infty)$.

- For convenience, we re-index the disjoint intervals $\{\zeta_k\}_{K+2}^{\infty}$ such that they are in increasing

order, i.e. $u_k > \bar{k} \geq u_{k-1}$, $\forall k > 1$, starting at $(0, L)$, with the last interval (B, ∞) . If there exist any intervals such that $\bar{k} = u_{k-1}$, replace them with the combined interval $(\bar{k}-1, u_k)$. If $\{B, \infty\}$ overlaps with the last interval, combine them. Note that, by the suppositions, in each case $\exists \delta_0$ small enough such that $r(u, \sigma)$ maintains the appropriate property for all σ such that $\hat{R}(\sigma) \geq \hat{R}(\sigma_{\delta_0})$, $\forall \delta < \delta_0$.

51

[form-feed in source extraction]

Step 2: showing that $\hat{R}(\sigma^*) \geq [U+0001 \text{ control character}]\hat{R}(\sigma_0)$, where σ^* is of the appropriate form. Now, starting at $\sigma = \sigma_{\delta_0} = (0, L) \cup u_K$

$k=1 (\bar{k}, u_k) \cup (B, \infty)$, we describe a sequence of modifications to σ , such that

each modification does not reduce the reward $\hat{R}(\sigma)$. The limit of this sequence of modifications is a policy σ^* of the appropriate form, regardless of the starting σ_{δ_0} .

We now carry out this step separately for each case. The general argument is that the properties force $r(u, \sigma)$ to be positive at certain points, which allows expanding the policy until a policy of the appropriate form is reached.

Let $r_L(\bar{\cdot}, \sigma) = -r(u, \sigma)$, i.e., it is a function that has the same sign as the derivative of a lower endpoint of σ (the same sign as the infinitesimal loss as removing the point $\bar{\cdot}$ from the set σ).

∂

$\hat{R}(\sigma)$ is positive in u , we can

Setting where $r(u, \sigma) > 0$ (is positive). By supposition that ∂u

increase u_1 (merging with other intervals) while increasing $\hat{R}(\sigma)$. Thus, we can keep increasing u_1 ,

and $u_1 \rightarrow B$, and so $\hat{R}((0, \infty)) \geq \hat{R}(\sigma_{\delta_0})$. For any set $\sigma_0 \in (0, \infty)$, we can increase the reward by expanding an interval, and so it cannot be optimal. Thus, $(0, \infty)$ is the unique optimal set.

Setting where $r(u, \sigma)$ is strictly increasing.

Now, starting at $\sigma = \sigma_{\delta_0}$, the limit of the sequence of modifications is a policy $\sigma^* = (\bar{\cdot}^*, \infty)$.

By the supposition that $r(u, \sigma)$ strictly increasing in u , we have:

$r_L(\bar{\cdot}, \sigma)$ strictly decreasing

$r_L(\bar{1}, \sigma) \leq 0 \Rightarrow r(u_1, \sigma) > 0$

∂

$\hat{R}(\sigma)$

$\hat{R}(\sigma) \leq 0 \Rightarrow$

$\hat{R}(\sigma) > 0$

$\partial \bar{1}$

∂u_1

$\bar{1} < u_1$

$r_L(\bar{1}, \sigma) > 0 \Leftarrow r(u_1, \sigma) \leq 0$

∂

$\hat{R}(\sigma)$

$\hat{R}(\sigma) > 0 \Leftarrow$

$\hat{R}(\sigma) \leq 0$

$\partial \bar{1}$

∂u_1

Case 1: $\exists \bar{1}, \bar{2} \subset \sigma$ such that $\bar{2} > u_1$, $[\bar{1}, \bar{2}]$, i.e. there is more than one interval that makes up σ , and $\bar{1}, \bar{2}$ are the first and second such intervals, respectively, with positive mass.

Then we make the following sequence of changes (forming new σ), depending on $\partial \bar{1} \hat{R}(\sigma)$, $\partial \bar{2} \hat{R}(\sigma)$:

Subcase 1A, $\partial \bar{2} \hat{R}(\sigma) > 0$: Increase u_1 until $u_1 = \bar{2}$ (exit Case 1), or $\partial \bar{2} \hat{R}(\sigma) \leq 0$ (go to Case 1B).

Sub-subcase 1AA, $\partial \bar{1} \hat{R}(\sigma) < 0$, $\bar{1} > 0$:

Simultaneously, decrease $\bar{1}$.

Sub-subcase 1AB, $\partial \bar{1} \hat{R}(\sigma) \geq 0$ or $\bar{1} = 0$:

Hold $\bar{1}$ fixed.

Subcase 1B, $\partial \bar{2} \hat{R}(\sigma) \leq 0 \Rightarrow \partial \bar{1} \hat{R}(\sigma) > 0$: Increase $\bar{1}$ until $\bar{1} = u_1$ (exit Case 1), or $\partial \bar{1} \hat{R}(\sigma) \leq 0$ (which implies $\partial \bar{2} \hat{R}(\sigma) > 0$, i.e. go to Case 1A).

Each of these changes increases $\hat{R}(\sigma)$, due to the direction of the changes in u_1 , $\bar{1}$ and the signs of the appropriate derivatives. Note that these subcases are mutually-exclusive, and one is true as long as there is more than one disjoint interval, $\exists \bar{1}, \bar{2} \subset \sigma$, $\bar{2} > u_1$. Further, note that u_1 is increasing in Subcase 1A and constant in Subcase 1B. Thus, with $\bar{2}$ fixed and bounded, eventually:

52

[form-feed in source extraction]

• $\bar{1} \rightarrow u_1$, in Subcase 1B (i.e. the first interval collapses to mass 0). OR

• $u_1 \rightarrow \bar{2}$, in Subcase 1A (i.e. the first interval merges with the second).

Thus, this sequence of changes increases the reward, and results in there being one fewer interval than before (after combining the bottom 2 intervals by adding the point $u_1 = \bar{2}$ of 0 measure).

Case 1 can be iteratively applied until there is just a single interval $\sigma = (\bar{\cdot}^0, \infty)$.

That the changes can increase the reward for any other σ implies that such σ cannot be optimal.

Case 2: $\sigma = (\bar{\cdot}^0, \infty)$, i.e. there is a single interval that makes up σ

By supposition, $\hat{R}(\sigma_0) > \hat{R}(\emptyset)$ and so $\hat{R}((\bar{\cdot}^0, \infty)) > \hat{R}(\emptyset)$. Further $\hat{R}((\bar{\cdot}, \infty))$ is a continuous

function in $\bar{\cdot}$. Thus, there exists L such that $\forall \bar{\cdot} > L$, $\hat{R}((\bar{\cdot}^0, \infty)) > \hat{R}((\bar{\cdot}, \infty))$.

Thus, there exists $\bar{*} \in [0, L]$ such that $\hat{R}((\bar{*}, \infty)) \geq \hat{R}((\bar{\cdot}, \infty))$, $\forall \bar{\cdot} \in \mathbb{R}^+ \cup \{\infty\}$ (continuous

functions in a compact domain have a maximum).

Setting where $r(u, \sigma)$ is strictly decreasing.

The proof is extremely similar to the strictly increasing case, with two differences.

First we now need to [U+0001 control character]modify the starting σ_0 so it does not contain an interval (B, ∞) : set

σ_0

$\sigma_0 = (0, L) \cup u_K$

$k=1 (\bar{k}, u_k) \setminus (B, \infty)$.

Second, each case from above is duplicated but move the policy in the opposite direction to increase the reward. We omit the details of this case for brevity.

Setting where $r(u, \sigma)$ is strictly quasi-convex.

We show that there exists a $\sigma^* = (0, \bar{u}) \cup (u^*, \infty)$, for some $u^*, \bar{u} \in \mathbb{R}^+$, such that $\hat{R}(\sigma_0) \leq \hat{R}(\sigma^*)$, $\forall \sigma$.

The key is noting that quasi-convexity of $r(u, \sigma)$ in u implies that any σ with three intervals $\zeta_1, \zeta_2, \zeta_3$ can be improved by eliminating the middle interval (or joining it with one of the others).

Case 1: $\exists$ disjoint $\zeta_1 = (0, u_1)$, $\zeta_2 = (\bar{u}_2, u_2)$, $\zeta_3 = (\bar{u}_3, u_3)$, s.t. $|\zeta_1|, |\zeta_2|, |\zeta_3| > 0$, i.e. σ is composed of at least three intervals, and $\zeta_1, \zeta_2, \zeta_3$ are the first three such intervals with positive mass. (u_3 may be ∞).

By supposition, $r(u_k, \sigma)$, is strictly quasi-convex in u , and so $r_L(\bar{u}_k, \sigma)$, is strictly quasi-concave in $\bar{u}$.

Then, we have:

$$\begin{aligned} \frac{\partial}{\partial \bar{u}_2} \hat{R}(\sigma) &\leq 0 \text{ and} \\ \hat{R}(\sigma) &\geq 0 \implies \\ \frac{\partial}{\partial \bar{u}_2} \hat{R}(\sigma) &> 0 \text{ and} \\ \hat{R}(\sigma) &< 0 \end{aligned}$$

$$\begin{aligned} \frac{\partial}{\partial u_1} \hat{R}(\sigma) &\leq 0 \text{ or} \\ \hat{R}(\sigma) &\geq 0 \implies \\ \frac{\partial}{\partial u_1} \hat{R}(\sigma) &> 0 \text{ or} \\ \hat{R}(\sigma) &< 0 \end{aligned}$$

$$\begin{aligned} \frac{\partial}{\partial u_2} \hat{R}(\sigma) &\leq 0 \text{ or} \\ \hat{R}(\sigma) &\geq 0 \implies \\ \frac{\partial}{\partial u_2} \hat{R}(\sigma) &> 0 \text{ or} \\ \hat{R}(\sigma) &< 0 \end{aligned}$$

Then we make the following sequence of changes (forming new σ):

Subcase 1A, $\frac{\partial}{\partial u_1} \hat{R}(\sigma) \leq 0$ and $\frac{\partial}{\partial \bar{u}_2} \hat{R}(\sigma) \geq 0 \implies \frac{\partial}{\partial \bar{u}_2} \hat{R}(\sigma) > 0$ and $\frac{\partial}{\partial u_2} \hat{R}(\sigma) < 0$: Increase $\bar{u}_2$ and decrease u_2 simultaneously until $\bar{u}_2 = u_2$ (exit Case 1), $\frac{\partial}{\partial u_2} \hat{R}(\sigma) \geq 0$, or $\frac{\partial}{\partial \bar{u}_2} \hat{R}(\sigma) \leq 0$ (go to 1B or 1C).

Subcase 1B, $\frac{\partial}{\partial u_1} \hat{R}(\sigma) > 0$: Increase u_1 until $u_1 = \bar{u}_2$ (exit Case 1), or $\frac{\partial}{\partial u_1} \hat{R}(\sigma) \leq 0$ (go to 1A or 1C)

Subcase 1C, $\frac{\partial}{\partial \bar{u}_2} \hat{R}(\sigma) < 0$: Decrease $\bar{u}_2$ until $u_2 = \bar{u}_3$ (exit Case 1), or $\frac{\partial}{\partial \bar{u}_2} \hat{R}(\sigma) \geq 0$ (go to 1B or 1A).

Thus, this sequence of changes cannot decrease the reward, and results in there being one fewer interval than before. Case 1 can be iteratively applied until there are just two intervals $\sigma = (0, t_1) \cup (t_2, \infty)$.

That the changes can increase the reward for any other σ implies that such σ cannot be optimal.

Case 2: $\sigma = (0, t_1) \cup (t_2, \infty)$. We need to show that there exists a maxima (t^*_1, t^*_2) . Specifically, we need to eliminate the possible cases where t_1 or t_2 increase to infinity, but the asymptotic values at ∞ produce lower rewards, which would have implied that the maximum is not achieved.

By supposition, $\hat{R}(\sigma_0) > \hat{R}(\emptyset)$ and so $\hat{R}((0, t_1) \cup (t_2, \infty)) > \hat{R}(\emptyset)$ for $t_1 > 0$ or $t_2 < \infty$.

Further $\hat{R}((0, t_1) \cup (t_2, \infty))$ is a continuous function in t_1, t_2 . Thus $\hat{R}((0, t_1) \cup (t_2, \infty)) \rightarrow \hat{R}(\emptyset)$ as $t_1 \rightarrow 0, t_2 \rightarrow \infty$ together.

Further, $\hat{R}((0, t_1) \cup (t_2, \infty)) \rightarrow \hat{R}((0, \infty))$ as $t_1 \rightarrow \infty$, regardless of how t_2 behaves. Similarly, fixing t_1 , $\hat{R}((0, t_1) \cup (t_2, \infty)) \rightarrow \hat{R}((0, t_1))$ as $t_2 \rightarrow \infty$.

• If $\hat{R}((0, t_1) \cup (t^*_2(t_1), \infty))$ is increasing for $t_1 > T_1$, for however $t^*_2(t_1)$ behaves as a function of t_1 then $\hat{R}((0, \infty)) \geq \hat{R}((0, t_1) \cup (t_2, \infty)), \forall t_1 > T_1, t_2$.

• For any fixed t_1 , if $\hat{R}((0, t_1) \cup (t_2, \infty))$ is increasing for $t_2 > T_2$, then $\hat{R}((0, t_1)) \geq \hat{R}((0, t_1) \cup (t_2, \infty)), \forall t_2$.

Thus, the maximum is achieved: either

1. $\exists t^*_1 \in (0, \infty) : \hat{R}((0, t^*_1)) \geq \hat{R}((0, t_1) \cup (t_2, \infty)), \forall t_1, t_2$
2. $\exists t^*_1, t^*_2 \in [0, \infty) : \hat{R}((0, t^*_1) \cup (t^*_2, \infty)) \geq \hat{R}((0, t_1) \cup (t_2, \infty)), \forall t_1, t_2$

Setting where $r(u, \sigma)$ is strictly quasi-concave. The proof is extremely similar to the strictly quasi-convex case. However, we now need to modify the starting σ_0 so it does not contain an intervals $(0, L)$ or (B, ∞) , and the subsequent modifications also differ directionally.

[U+0001 control character]

Let $\sigma_0 = u_k$

$k=1(\bar{u}_k, u_k) \setminus (0, L) \setminus (B, \infty)$. Now, the key step is noting that strict quasi-concavity of $r(u, \sigma)$ implies that any σ with two intervals ζ_1, ζ_2 can be improved by eliminating one (or joining the two).

Case 1: $\exists$ disjoint $\zeta_1 = (\bar{u}_1, u_1)$, $\zeta_2 = (\bar{u}_2, u_2)$, s.t. $|\zeta_1|, |\zeta_2| > 0$, i.e. σ is composed of at least two intervals with positive mass, and ζ_1, ζ_2 are the first two such intervals.

By supposition, $r(u_k, \sigma)$, is strictly quasi-concave in u . Then, $r_L(\bar{u}_k, \sigma)$, is strictly quasi-convex in $\bar{u}$.

Then, we have:

$$\begin{aligned} \frac{\partial}{\partial \bar{u}_2} \hat{R}(\sigma) &\leq 0 \text{ and} \\ \hat{R}(\sigma) &\leq 0 \implies \end{aligned}$$

$\dot{R}(\sigma) > 0$
 ∂^1
 ∂^2
 ∂u_1
 ∂
 ∂
 ∂
 $\dot{R}(\sigma) \leq 0 \implies$
 $\dot{R}(\sigma) > 0$ or
 $\dot{R}(\sigma) > 0$
 ∂u_1
 ∂^1
 ∂^2
54

[form-feed in source extraction]

Then we make the following sequence of changes (forming new σ):

Subcase 1A, $\partial^1 \dot{R}(\sigma) \leq 0$ and $\partial^2 \dot{R}(\sigma) \leq 0 \implies \partial u \partial^1 \dot{R}(\sigma) > 0$: Increase u_1 until $u_1 = \partial^2$ (exit Case 1) or $\partial u \partial^1 \dot{R}(\sigma) \leq 0$ (go to 1B or 1C).

Subcase 1B, $\partial^1 \dot{R}(\sigma) > 0$: Increase ∂^1 until $u_1 = \partial^1$ (exit Case 1), or $\partial^1 \dot{R}(\sigma) \leq 0$ (go to 1A or 1C)

Subcase 1C, $\partial^2 \dot{R}(\sigma) > 0$: Increase ∂^2 until $u_2 = \partial^2$ (exit Case 1), or $\partial^2 \dot{R}(\sigma) \leq 0$ (go to 1B or 1A).

Each of these changes strictly increase $R(\sigma)$. 1B and 1C may both be true, in which case arbitrarily decide between them. At least one of the three subcases is true as long as the Case 1 condition holds. Thus, eventually:

- $\partial^2 = u_1$, in Subcase 1A (i.e. the intervals combine). OR
- $u_1 = \partial^1$, in Subcase 1B (i.e. the first interval collapses to mass 0). OR
- $u_2 = \partial^2$, in Subcase 1C (i.e. the second interval collapses to mass 0).

Thus, this sequence of changes cannot decrease the reward, and result in there being one fewer interval than before. Case 1 can be iteratively applied until there is just one interval $\sigma_i = (t_1, t_2)$.

That the changes can increase the reward for any other σ implies that such σ cannot be optimal. Case 2: $\sigma_i = (t_1, t_2)$. As in the same case in the quasi-convex setting. We need to values eliminate the possible cases where t_1 or t_2 increasing to infinity, but the asymptotic values at ∞ produce lower rewards, which would imply that the maximum is not achieved.

Note that $\dot{R}((t_1, t_2))$ is a continuous function in t_1, t_2 . Thus $\dot{R}((t_1, t_2)) \rightarrow \dot{R}(\emptyset)$ as $t_1 \rightarrow t_2$.

Further, $\dot{R}((t_1, t_2)) \rightarrow \dot{R}(\emptyset)$ as $t_1 \rightarrow \infty$, regardless of how $t_2 \geq t_1$ behaves. Similarly, fixing t_1 ,

$\dot{R}((t_1, t_2)) \rightarrow \dot{R}((t_1, \infty))$ as $t_2 \rightarrow \infty$.

- If $\dot{R}((t_1, t_2))$ is increasing for $t_1 > T_1$, for however $t_2(t_1)$ behaves as a function of t_1 then $\dot{R}(\emptyset) \geq \dot{R}((t_1, t_2))$, $\forall t_1 > T_1, t_2$.

- For any fixed t_1 , if $\dot{R}((t_1, t_2))$ is increasing for $t_2 > T_2$, then $\dot{R}((t_1, \infty)) \geq \dot{R}((t_1, t_2))$, $\forall t_2 > T_2$.

Thus, either

1. $\dot{R}(\emptyset) \geq \dot{R}((t_1, t_2))$, $\forall t_1, t_2$
2. $\exists t^* \in [0, \infty) : \dot{R}((t^*, \infty)) \geq \dot{R}((t_1, t_2))$, $\forall t_1, t_2$
3. $\exists t^* \in [0, \infty) : \dot{R}(t^*, t^*) \geq \dot{R}((t_1, t_2))$, $\forall t_1, t_2$

[Next verbatim source excerpt]

Recall in the dynamic model that we constrain σ_i to be measurable, open, subsets of the R^+ . Then, σ_i can be written as a countable union of disjoint subsets of R^+ , i.e. $\sigma_i = \bigcup_{k=0}^{\infty} (k, u_k)$. We further

assume that $u_k = m$, for any k, m ; we can do so without loss of generality by making a measure 0 change to σ_i , by adding $u_k = m$ to σ_i .

Suppose u is an upper-endpoint of σ_i , i.e. $\exists k$ such that $u = u_k$. Then, we use ∂u $H(\sigma_i)$ to denote

the derivative of the set function H with respect to u at σ_i . Similarly, $\partial^- H(\sigma_i)$ is the derivative of H at σ_i with respect to a lower-endpoint of σ_i .

∂

We derive ∂u

$R(w, \{\sigma_1, \sigma_2\})$, ∂^-

$R(w, \{\sigma_1, \sigma_2\})$. We will make it clear in each instance whether u

∂

refers to the derivative

or ∂^- is an endpoint of σ_1 or σ_2 . For all the derivatives in this subsection ∂u

∂

with respect to an upper endpoint in σ_i , and ∂^- refers to a derivative at a lower endpoint of σ_i .

Note that

∂

$R(w, \{\sigma_1, \sigma_2\}) = -R(w, \{\sigma_1, \sigma_2\})$.

∂u

∂^-

Furthermore:

- We use σ in the function argument when the function depends on policies in both states, and σ_i when it only depends on the policy in state i .
- We use $\propto$ to denote that "two functions of u have the same sign except where $f(u) = 0$ ", rather than proportional to.
- All policy equalities are up to measure 0.

Settled source reading The generic Lemma 5 statement explicitly assumes continuity on each compact endpoint-parameter domain. Continuity on the source's open policy domain alone is not treated as a proof of extension to the limiting endpoints.

Settled source reading The generic Lemma 5 continuity premise quantifies over all TripPolicy centers and comparison policies. This is broader than the source-open domain; the added quantification is a current-proof premise, not evidence that the source theorem requires it.

Settled source reading The generic Lemma 5 shape premises quantify over every policy in the near-optimal reward band, beyond the source-open domain, and explicitly state the quasi-shape comparisons involving zero trip length. The memo gives their precise scope.

Expanded PaperInterface specification

```

∀ (mu : MeasureTheory.Measure GN21DriverSurgePricing.TripLength) [MeasureTheory.IsFiniteMeasure mu]
[mu.InnerRegularCompactLTTOP] [MeasureTheory.NoAtoms mu] (Rhat : GN21DriverSurgePricing.SingleStateReward)
(response : GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ)
(shape : GN21DriverSurgePricing.Lemma5DerivativeShape) {sigma : GN21DriverSurgePricing.TripPolicy},
IsOpen sigma →
sigma ⊆ GN21DriverSurgePricing.acceptAllPolicy →
Rhat 0 < Rhat sigma →
  ∀ {margin : ℝ},
  0 < margin →
    (∀ (tau : GN21DriverSurgePricing.TripPolicy), GN21DriverSurgePricing.GN21SymmDiffContinuousAt mu Rhat tau) →
      (∀ (extra : ℕ),
        ContinuousOn
          (fun (endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra) =>
            Rhat (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints))
            (GN21DriverSurgePricing.gn21Lemma5EndpointDomain shape extra)) →
        (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.positive →
          ∀ (tau : GN21DriverSurgePricing.TripPolicy),
            Rhat sigma - margin ≤ Rhat tau →
              ∀ (u : GN21DriverSurgePricing.TripLength), 0 < u → 0 < response tau u) →
            (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyIncreasing →
              ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                Rhat sigma - margin ≤ Rhat tau → StrictMonoOn (response tau) (Set.Ici 0)) →
            (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyDecreasing →
              ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                Rhat sigma - margin ≤ Rhat tau → StrictAntiOn (response tau) (Set.Ici 0)) →
            (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex →
              ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                Rhat sigma - margin ≤ Rhat tau →
                  GN21DriverSurgePricing.strictQuasiConvexOnPositive (response tau)) →
            (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave →
              ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                Rhat sigma - margin ≤ Rhat tau →
                  ∀ {middleLower middleUpper right : ℝ},
                    0 < middleLower →
                      middleLower < middleUpper →
                        middleUpper < right →
                          response tau middleLower < max (response tau 0) (response tau right)) →
            (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave →
              ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                Rhat sigma - margin ≤ Rhat tau →
                  GN21DriverSurgePricing.strictQuasiConcaveOnPositive (response tau)) →
            (shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave →
              ∀ (tau : GN21DriverSurgePricing.TripPolicy),
                Rhat sigma - margin ≤ Rhat tau →
                  ∀ {middle right : ℝ},
                    0 < middle →
                      middle < right →
                        min (response tau 0) (response tau right) < response tau middle) →
              ∀
                (H :
                  GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation mu Rhat response
                    shape sigma margin),
                ∃ (policy : GN21DriverSurgePricing.TripPolicy),
                  GN21DriverSurgePricing.lemma5SourcePolicyForm shape policy ∧
                    Rhat sigma ≤ Rhat policy ∧
                    (¬GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere mu shape sigma →
                      Rhat sigma < Rhat policy))
          )
    )
  )

```

Retained governing declarations

- [GN21DriverSurgePricing.GN21Lemma5EndpointVector](#)
- [GN21DriverSurgePricing.GN21SymmDiffContinuousAt](#)
- [GN21DriverSurgePricing.Lemma5DerivativeShape](#)
- [GN21DriverSurgePricing.ProofBridge.GN21Lemma5AENullEndpointVariation](#)
- [GN21DriverSurgePricing.SingleStateReward](#)
- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.gn21EndpointVectorPolicy](#)
- [GN21DriverSurgePricing.gn21Lemma5EndpointDomain](#)
- [GN21DriverSurgePricing.lemma5SourcePolicyForm](#)
- [GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere](#)
- [GN21DriverSurgePricing.strictQuasiConcaveOnPositive](#)
- [GN21DriverSurgePricing.strictQuasiConvexOnPositive](#)

Lean-elaborated claim roles

1. parameter: MeasureTheory.Measure GN21DriverSurgePricing.TripLength
2. assumption: MeasureTheory.IsFiniteMeasure mu
3. assumption: mu.InnerRegularCompactLTTop
4. assumption: MeasureTheory.NoAtoms mu
5. parameter: GN21DriverSurgePricing.SingleStateReward
6. parameter: GN21DriverSurgePricing.TripPolicy → GN21DriverSurgePricing.TripLength → ℝ
7. parameter: GN21DriverSurgePricing.Lemma5DerivativeShape
8. parameter: GN21DriverSurgePricing.TripPolicy
9. assumption: IsOpen sigma
10. assumption: sigma ⊆ GN21DriverSurgePricing.acceptAllPolicy
11. assumption: Rhat ∅ < Rhat sigma
12. parameter: ℝ
13. assumption: 0 < margin
14. assumption: ∀ (tau : GN21DriverSurgePricing.TripPolicy), GN21DriverSurgePricing.GN21SymmDiffContinuousAt mu Rhat tau
15. assumption: ∀ (extra : ℕ), ContinuousOn
 (fun (endpoints : GN21DriverSurgePricing.GN21Lemma5EndpointVector extra) =>
 Rhat (GN21DriverSurgePricing.gn21EndpointVectorPolicy endpoints))
 (GN21DriverSurgePricing.gn21Lemma5EndpointDomain shape extra)
16. assumption: shape = GN21DriverSurgePricing.Lemma5DerivativeShape.positive →
 ∀ (tau : GN21DriverSurgePricing.TripPolicy),
 Rhat sigma - margin ≤ Rhat tau → ∀ (u : GN21DriverSurgePricing.TripLength), 0 < u → 0 < response tau u
17. assumption: shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyIncreasing →
 ∀ (tau : GN21DriverSurgePricing.TripPolicy), Rhat sigma - margin ≤ Rhat tau → StrictMonoOn (response tau) (Set.Ici 0)
18. assumption: shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyDecreasing →
 ∀ (tau : GN21DriverSurgePricing.TripPolicy), Rhat sigma - margin ≤ Rhat tau → StrictAntiOn (response tau) (Set.Ici 0)
19. assumption: shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex →
 ∀ (tau : GN21DriverSurgePricing.TripPolicy),
 Rhat sigma - margin ≤ Rhat tau → GN21DriverSurgePricing.strictQuasiConvexOnPositive (response tau)
20. assumption: shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConvex →
 ∀ (tau : GN21DriverSurgePricing.TripPolicy),
 Rhat sigma - margin ≤ Rhat tau →
 ∀ {middleLower middleUpper right : ℝ},
 0 < middleLower →
 middleLower < middleUpper →
 middleUpper < right → response tau middleLower < max (response tau 0) (response tau right)
21. assumption: shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave →
 ∀ (tau : GN21DriverSurgePricing.TripPolicy),
 Rhat sigma - margin ≤ Rhat tau → GN21DriverSurgePricing.strictQuasiConcaveOnPositive (response tau)
22. assumption: shape = GN21DriverSurgePricing.Lemma5DerivativeShape.strictlyQuasiConcave →
 ∀ (tau : GN21DriverSurgePricing.TripPolicy),
 Rhat sigma - margin ≤ Rhat tau →
 ∀ {middle right : ℝ},
 0 < middle → middle < right → min (response tau 0) (response tau right) < response tau middle
23. parameter: GN21DriverSurgePricing.ProofBridge.GN21Lemma5AEndpointVariation mu Rhat response shape sigma margin
24. conclusion: ∃ (policy : GN21DriverSurgePricing.TripPolicy),
 GN21DriverSurgePricing.lemma5SourcePolicyForm shape policy ∧
 Rhat sigma ≤ Rhat policy ∧
 (¬GN21DriverSurgePricing.lemma5SourcePolicyFormAlmostEverywhere mu shape sigma → Rhat sigma < Rhat policy)

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.lemma5_full_variational_policy_forms

Recorded source-to-Spec screening Verdict: matches

Reason: Lemma 5's five response-shape cases and its no-worse/strict-improvement policy-form conclusion agree with the expanded target, whose endpoint, continuity, shape, and null-set premises make the source proof domain explicit.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims (continued 3)

Review row 16

Source locator: cited publication, lines 264-285

Verbatim source input

follows policy $\sigma = \{\sigma_1, \sigma_2\}$, where $\sigma_i \subseteq R^+$ indicates the jobs accepted in state i . We assume that driver policies are measurable with respect to F (corresponding F_i in dynamic model); for technical reasons, in the dynamic model we also assume that σ_i consist of a union of open intervals, i.e., are open subsets of R^+ . When we write equalities with policies σ , we mean equality up to changes of measure 0.

The driver is long-lived and aims to maximize their own lifetime average hourly earnings on the platform, including both open and busy times. Let $R(w, \sigma, t)$ denote the (random) total earnings from jobs accepted from time 0 up to time t if the driver follows policy σ and the payout function is w . Then, the driver's lifetime earnings rate is $R(w, \sigma), \liminf_{t \rightarrow \infty}$

$R(w, \sigma, t)$
.

This earnings rate is a deterministic (non-random) quantity, and is a function of the driver policy σ , pricing function w , and the primitives.

A driver policy σ^* is optimal (best-response) with respect to pricing function w if it maximizes the lifetime earnings rate of the driver among all policies: $R(w, \sigma^*) \geq R(w, \sigma)$, for all valid policies σ (i.e., measurable with respect to F or F_i , with σ_i open sets). Then, pricing function w is incentive compatible (IC) if accepting all job requests is optimal with respect to w , i.e., $\sigma = (0, \infty)$ in the single-state model or $\sigma = \{(0, \infty), (0, \infty)\}$ in the dynamic model is optimal with respect to w .

[Next verbatim source excerpt]

Theorem 3. Let $R_1 < R_2$ be target earning rates during non-surged and surge states, respectively.

There exist prices $w = \{w_1, w_2\}$ of the form

$w_i(\tau) = m_i \tau + z_i q_{i \rightarrow j}(\tau)$,

where $m_1, m_2, z_2 \geq 0$ (but z_1 may be either positive or negative), such that the optimal driver policy

is to accept every trip in the surge state and all trips up to a certain length in the non-surge state.

R_1

Furthermore, for R

$\in (C, 1)$, there exist fully incentive compatible prices of this form, where

2

$C = 1 -$

$1 Q_2 (\lambda_{12} T_1 - Q_1) + Q_1 (T_2 \lambda_{1 \rightarrow 2} + Q_2)$

$\in [0, 1)$,

$T_1 Q_2 (\lambda_{1 \rightarrow 2} T_1 - Q_1) + \lambda_{1 \rightarrow 2} (T_2 \lambda_{1 \rightarrow 2} + Q_2)$

and $T_i = \lambda_i F_i(\sigma_i) T_i((0, \infty))$, and $Q_i = Q_i((0, \infty))$. For such prices, the driver policy to accept all requests is the unique optimal driver policy (up to differences of measure 0).

[Next verbatim source excerpt]

$q_{i \rightarrow j}(\tau) dF_i(\tau)$

$Q_i, Q_i(\sigma_i) = \lambda_{i \rightarrow j} + \lambda_i$

σ_i

Z

$\tau dF_i(\tau)$

$T_i, \lambda_i F_i(\sigma_i) T_i(\sigma_i) = 1 + \lambda_i$

$\tau \in \sigma_i$

Z

$w_i(\tau) dF_i(\tau)$

$W_i, \lambda_i F_i(\sigma_i) W_i(\sigma_i) = \lambda_i$

$\tau \in \sigma_i$

$\Delta_{ji}, \Delta(\sigma_j, \sigma_i) = R_j(w_j, \sigma_j) - R_i(w_i, \sigma_i)$

We assume throughout:

- Distribution of jobs F, F_i is a continuous probability measure, i.e., f, f_i bounded.
- There exists a policy in state 2 that dominates state 1: $\exists \sigma_2$ such that $\Delta(\sigma_2, \sigma_1) > 0, \forall \sigma_1 \in (0, \infty)$.
- σ, σ_i constrained to be measurable with respect to F, F_i , and σ_i are open.

Expanded PaperInterface specification

```
(∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ) (R1 : NNReal)
(R2 switch12 switch21 : ℝ) [MeasureTheory.NoAtoms (mu 0)] [MeasureTheory.NoAtoms (mu 1)]
[MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)],
↑R1 < R2 →
0 < arrival 0 →
0 < arrival 1 →
0 < switch12 →
0 < switch21 →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1 →
GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1 →
∃ (m : Fin 2 → ℝ) (z : Fin 2 → ℝ) (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
(0 ≤ m 0 ∧ 0 ≤ m 1 ∧ 0 ≤ z 1) ∧
GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 0) (arrival 0)
(GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 0)
```

```

GN21DriverSurgePricing.acceptAllPolicy =
  ↑ R1 ∧
GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 1) (arrival 1)
  (GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 1)
  GN21DriverSurgePricing.acceptAllPolicy =
  R2 ∧
  rho 1 = GN21DriverSurgePricing.acceptAllPolicy ∧
  GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoff (rho 0) ∧
  GN21DriverSurgePricing.dynamicOpenOptimal
  (GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival switch12
    switch21 m z)
  rho) ∧
∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ)
(R1 R2 switch12 switch21 : ℝ) [MeasureTheory.NoAtoms (mu 0)] [MeasureTheory.NoAtoms (mu 1)]
[MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)],
0 < R2 →
GN21DriverSurgePricing.theorem3FeasibilityThresholdC
  (GN21DriverSurgePricing.gn21AcceptAllScaledStateTime (mu 0) (arrival 0))
  (GN21DriverSurgePricing.gn21AcceptAllScaledStateTime (mu 1) (arrival 1))
  (GN21DriverSurgePricing.gn21AcceptAllExitWeightIntegral (mu 0) (arrival 0) switch12 switch21)
  (GN21DriverSurgePricing.gn21AcceptAllExitWeightIntegral (mu 1) (arrival 1) switch21 switch12) switch12 <
  R1 / R2 →
R1 / R2 < 1 →
0 < arrival 0 →
0 < arrival 1 →
0 < switch12 →
0 < switch21 →
  MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
  GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
  MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
  GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
  GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1 →
  GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1 →
  ∃ (m : Fin 2 → ℝ) (z : Fin 2 → ℝ),
  (0 ≤ m 0 ∧ 0 ≤ m 1 ∧ 0 ≤ z 1) ∧
  GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 0) (arrival 0)
  (GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 0)
  GN21DriverSurgePricing.acceptAllPolicy =
  R1 ∧
  GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 1) (arrival 1)
  (GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 1)
  GN21DriverSurgePricing.acceptAllPolicy =
  R2 ∧
  GN21DriverSurgePricing.dynamicOpenOptimal
  (GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival
    switch12 switch21 m z)
  GN21DriverSurgePricing.acceptAllDynamicPolicy ∧
  ∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
  GN21DriverSurgePricing.dynamicOpenOptimal
  (GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival
    switch12 switch21 m z)
  rho →
  GN21DriverSurgePricing.dynamicAcceptAllAlmostEverywhere mu rho

```

Retained governing declarations

- [GN21DriverSurgePricing.TripLength](#)
- [GN21DriverSurgePricing.TripPolicy](#)
- [GN21DriverSurgePricing.acceptAllDynamicPolicy](#)
- [GN21DriverSurgePricing.acceptAllPolicy](#)
- [GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice](#)
- [GN21DriverSurgePricing.dynamicAcceptAllAlmostEverywhere](#)
- [GN21DriverSurgePricing.dynamicOpenOptimal](#)
- [GN21DriverSurgePricing.gn21AcceptAllExitWeightIntegral](#)
- [GN21DriverSurgePricing.gn21AcceptAllScaledStateTime](#)
- [GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward](#)
- [GN21DriverSurgePricing.gn21MeasuredStateRewardRate](#)
- [GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoff](#)
- [GN21DriverSurgePricing.singleStateTripMass](#)
- [GN21DriverSurgePricing.theorem3FeasibilityThresholdC](#)

Lean-elaborated claim roles

```

1. conclusion: (∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ) (R1 : NNReal)
(R2 switch12 switch21 : ℝ) [MeasureTheory.NoAtoms (mu 0)] [MeasureTheory.NoAtoms (mu 1)]
[MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)],
↑R1 < R2 →
0 < arrival 0 →
0 < arrival 1 →
0 < switch12 →
0 < switch21 →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1 →
GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1 →
∃ (m : Fin 2 → ℝ) (z : Fin 2 → ℝ) (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
(0 ≤ m 0 ∧ 0 ≤ m 1 ∧ 0 ≤ z 1) ∧
GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 0) (arrival 0)
(GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 0)
GN21DriverSurgePricing.acceptAllPolicy =
↑R1 ∧
GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 1) (arrival 1)
(GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 1)
GN21DriverSurgePricing.acceptAllPolicy =
R2 ∧
rho 1 = GN21DriverSurgePricing.acceptAllPolicy ∧
GN21DriverSurgePricing.rejectsLongTripsFiniteOrInfiniteCutoff (rho 0) ∧
GN21DriverSurgePricing.dynamicOpenOptimal
(GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival switch12
switch21 m z)
rho) ∧
∀ (mu : Fin 2 → MeasureTheory.Measure GN21DriverSurgePricing.TripLength) (arrival : Fin 2 → ℝ)
(R1 R2 switch12 switch21 : ℝ) [MeasureTheory.NoAtoms (mu 0)] [MeasureTheory.NoAtoms (mu 1)]
[MeasureTheory.IsFiniteMeasure (mu 0)] [MeasureTheory.IsFiniteMeasure (mu 1)],
0 < R2 →
GN21DriverSurgePricing.theorem3FeasibilityThresholdC
(GN21DriverSurgePricing.gn21AcceptAllScaledStateTime (mu 0) (arrival 0))
(GN21DriverSurgePricing.gn21AcceptAllScaledStateTime (mu 1) (arrival 1))
(GN21DriverSurgePricing.gn21AcceptAllExitWeightIntegral (mu 0) (arrival 0) switch12 switch21)
(GN21DriverSurgePricing.gn21AcceptAllExitWeightIntegral (mu 1) (arrival 1) switch21 switch12) switch12 <
R1 / R2 →
R1 / R2 < 1 →
0 < arrival 0 →
0 < arrival 1 →
0 < switch12 →
0 < switch21 →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 0) →
MeasureTheory.IntegrableOn (fun (tau : GN21DriverSurgePricing.TripLength) => tau)
GN21DriverSurgePricing.acceptAllPolicy (mu 1) →
GN21DriverSurgePricing.singleStateTripMass (mu 0) GN21DriverSurgePricing.acceptAllPolicy = 1 →
GN21DriverSurgePricing.singleStateTripMass (mu 1) GN21DriverSurgePricing.acceptAllPolicy = 1 →
∃ (m : Fin 2 → ℝ) (z : Fin 2 → ℝ),
(0 ≤ m 0 ∧ 0 ≤ m 1 ∧ 0 ≤ z 1) ∧
GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 0) (arrival 0)
(GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 0)
GN21DriverSurgePricing.acceptAllPolicy =
R1 ∧
GN21DriverSurgePricing.gn21MeasuredStateRewardRate (mu 1) (arrival 1)
(GN21DriverSurgePricing.ctmcStructuredDynamicSurgePrice m z switch12 switch21 1)
GN21DriverSurgePricing.acceptAllPolicy =
R2 ∧
GN21DriverSurgePricing.dynamicOpenOptimal
(GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival
switch12 switch21 m z)
GN21DriverSurgePricing.acceptAllDynamicPolicy ∧
∀ (rho : Fin 2 → GN21DriverSurgePricing.TripPolicy),
GN21DriverSurgePricing.dynamicOpenOptimal
(GN21DriverSurgePricing.gn21AggregateCTMCStructuredDynamicReward mu arrival
switch12 switch21 m z)
rho →
GN21DriverSurgePricing.dynamicAcceptAllAlmostEverywhere mu rho

```

Lean proof endpoint (built separately)

GN21DriverSurgePricing.PaperInterface.review_theorem3_structured_pricing

Recorded source-to-Spec screening Verdict: matches

Reason: Theorem 3's ordered-target structured-price construction and stricter ratio-interval fully-IC/accept-all uniqueness clause agree with the expanded conjunction and stated policy forms; the first target remains in the source's nonnegative rate domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation